

nsbaci

1.0

Generated by Doxygen 1.16.1

Chapter 1

Main Page

Chapter 2

Download

Chapter 3

Namespace Index

3.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

nsbaci	Root namespace for the nsbaci application	??
nsbaci::compiler	Compiler namespace containing all compilation-related stuff	??
nsbaci::factories	Factories namespace for nsbaci	??
nsbaci::services	Services namespace containing all backend service implementations	??
nsbaci::services::runtime	Runtime services namespace for nsbaci	??
nsbaci::types	Type definitions namespace for nsbaci (runtime-specific)	??
nsbaci::ui	User interface namespace for nsbaci	??

Chapter 4

Hierarchical Index

4.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

nsbaci::BaseResult	??
FileResult	??
LoadResult	??
saveResult	??
InterpreterResult	??
nsbaci::compiler::CompilerResult	??
nsbaci::services::RuntimeResult	??
nsbaci::types::CanvasConfig	??
nsbaci::types::Circle	??
nsbaci::types::Color	??
nsbaci::types::CompileError	??
nsbaci::compiler::Compiler	??
nsbaci::compiler::NsbaciCompiler	??
nsbaci::services::CompilerService	??
nsbaci::factories::CompilerServiceFactory	??
nsbaci::factories::DefaultDrawingBackend	??
nsbaci::factories::DefaultFileSystem	??
nsbaci::types::Drawable	??
nsbaci::types::DrawCommand	??
nsbaci::factories::DrawingServiceFactory	??
nsbaci::types::DrawText	??
nsbaci::types::Ellipse	??
nsbaci::Error	??
nsbaci::types::ErrorBase	??
nsbaci::ui::ErrorDialogFactory	??
nsbaci::ui::ExampleEntry	??
nsbaci::services::FileService	??
nsbaci::factories::FileServiceFactory	??
nsbaci::compiler::Instruction	??
nsbaci::services::runtime::Interpreter	??
nsbaci::services::runtime::NsbaciInterpreter	??
nsbaci::types::Line	??
nsbaci::types::LoadError	??
nsbaci::factories::NsbaciCompiler	??

nsbaci::factories::NsbaciRuntime	??
nsbaci::types::Pixel	??
nsbaci::types::Point	??
nsbaci::ui::Preferences	??
nsbaci::services::runtime::Program	??
QDialog	
nsbaci::ui::PreferencesDialog	??
nsbaci::ui::UserGuideDialog	??
QMainWindow	
MainWindow	??
QObject	
nsbaci::Controller	??
nsbaci::services::DrawingService	??
QPlainTextEdit	
CodeEditor	??
QWidget	
LineNumberArea	??
nsbaci::ui::DrawingWidget	??
nsbaci::ui::RuntimeView	??
nsbaci::types::Rectangle	??
nsbaci::types::RuntimeError	??
nsbaci::services::RuntimeService	??
nsbaci::factories::RuntimeServiceFactory	??
nsbaci::types::SaveError	??
nsbaci::services::runtime::Scheduler	??
nsbaci::services::runtime::NsbaciScheduler	??
nsbaci::services::runtime::Semaphore	??
nsbaci::types::Size	??
nsbaci::types::SymbolInfo	??
Text	??
nsbaci::services::runtime::Thread	??
nsbaci::ui::ThreadInfo	??
nsbaci::types::Triangle	??
nsbaci::UIError	??
nsbaci::ui::VariableInfo	??
yyFlexLexer	
nsbaci::compiler::Lexer	??

Chapter 5

Class Index

5.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

nsbaci::BaseResult	Base result structure for all service operations	??
nsbaci::types::CanvasConfig	Configuration for the drawing canvas	??
nsbaci::types::Circle	Circle shape with center and radius	??
CodeEditor		??
nsbaci::types::Color	RGB color representation with values from 0-255	??
nsbaci::types::CompileError	Error payload for compilation errors	??
nsbaci::compiler::Compiler	Abstract base class for all compilers	??
nsbaci::compiler::CompilerResult	Result of a compilation operation	??
nsbaci::services::CompilerService	Service for compiling nsbaci (or maybe other stuff in the future) source code	??
nsbaci::factories::CompilerServiceFactory	Factory for creating CompilerService instances	??
nsbaci::Controller	Central coordinator between UI and backend services	??
nsbaci::factories::DefaultDrawingBackend		??
nsbaci::factories::DefaultFileSystem		??
nsbaci::types::Drawable	Complete drawable object with shape, color, and visibility	??
nsbaci::types::DrawCommand	Represents a single drawing command to be executed	??
nsbaci::services::DrawingService	Adapter service for graphical output backends	??
nsbaci::factories::DrawingServiceFactory	Factory for creating DrawingService instances	??
nsbaci::ui::DrawingWidget	Qt widget that provides a drawing canvas	??
nsbaci::types::DrawText		??
nsbaci::types::Ellipse	Ellipse shape with center and radii	??

nsbaci::Error	Represents an error with a message and optional code	??
nsbaci::types::ErrorBase	Base structure containing common error properties	??
nsbaci::ui::ErrorDialogFactory	Factory for creating error dialogs from UIError objects	??
nsbaci::ui::ExampleEntry	Data for a single example entry in the User Guide	??
FileResult	Base result type for file operations	??
nsbaci::services::FileService	Service for handling file system operations on BACI source files	??
nsbaci::factories::FileServiceFactory	Factory for creating FileService instances	??
nsbaci::compiler::Instruction	Represents a single instruction in the virtual machine	??
nsbaci::services::runtime::Interpreter	Executes instructions for threads within a program context	??
InterpreterResult		??
nsbaci::compiler::Lexer	Flex-based lexer for BACI source code	??
nsbaci::types::Line	Line segment from start to end point	??
LineNumberArea		??
nsbaci::types::LoadError	Error payload for file load errors	??
LoadResult	Result type for file load operations	??
MainWindow		??
nsbaci::compiler::NsbaciCompiler	Nsbaci compiler implementation using flex and bison	??
nsbaci::factories::NsbaciCompiler		??
nsbaci::services::runtime::NsbaciInterpreter	BACI-specific implementation of the Interpreter	??
nsbaci::factories::NsbaciRuntime		??
nsbaci::services::runtime::NsbaciScheduler	BACI-specific implementation of the Scheduler	??
nsbaci::types::Pixel	Single pixel at a position	??
nsbaci::types::Point	2D point/position representation	??
nsbaci::ui::Preferences	Holds all user-configurable preferences	??
nsbaci::ui::PreferencesDialog	A dialog that allows the user to configure application preferences	??
nsbaci::services::runtime::Program	Represents a compiled program ready for execution	??
nsbaci::types::Rectangle	Rectangle shape with position and size	??
nsbaci::types::RuntimeError	Error payload for runtime errors	??
nsbaci::services::RuntimeResult	Result of a runtime operation (step, run, etc.)	??
nsbaci::services::RuntimeService	Service that manages program execution	??
nsbaci::factories::RuntimeServiceFactory	Factory for creating RuntimeService instances	??

nsbaci::ui::RuntimeView	
Widget displaying runtime execution state	??
nsbaci::types::SaveError	
Error payload for file save errors	??
saveResult	
Result type for file save operations	??
nsbaci::services::runtime::Scheduler	
Manages thread scheduling and state transitions	??
nsbaci::services::runtime::Semaphore	
Counting semaphore for process synchronization	??
nsbaci::types::Size	
2D size representation	??
nsbaci::types::SymbolInfo	
Information about a variable/symbol	??
Text	
Text to be drawn at a position	??
nsbaci::services::runtime::Thread	
Represents a thread in the runtime service	??
nsbaci::ui::ThreadInfo	
Information about a thread for display	??
nsbaci::types::Triangle	
Triangle shape defined by three vertices	??
nsbaci::UIError	
UI-ready error representation for display in dialogs	??
nsbaci::ui::UserGuideDialog	
A dialog that provides an integrated user guide with code examples	??
nsbaci::ui::VariableInfo	
Information about a variable for display	??

Chapter 6

File Index

6.1 File List

Here is a list of all documented files with brief descriptions:

controller.cpp	Implementation of the Controller class for nsbaci	??
controller.h	Controller class declaration for nsbaci	??
error.h	Error class declaration for nsbaci	??
uiError.cpp	Implementation of UIError utilities	??
uiError.h	UI Error type definitions for nsbaci	??
main.cpp	Application entry point for nsbaci	??
services/compilerService/compiler/nsbaci/tools/main.cpp	CLI tool for testing the nsbaci compiler	??
compilerServiceFactory.cpp	Implementation unit for the CompilerServiceFactory	??
compilerServiceFactory.h	CompilerServiceFactory class declaration for nsbaci	??
drawingServiceFactory.cpp	Implementation unit for the DrawingServiceFactory	??
drawingServiceFactory.h	DrawingServiceFactory class declaration for nsbaci	??
fileServiceFactory.cpp	Implementation unit for the FileServiceFactory	??
fileServiceFactory.h	FileServiceFactory class declaration for nsbaci	??
runtimeServiceFactory.cpp	Implementation unit for the RuntimeServiceFactory	??
runtimeServiceFactory.h	RuntimeServiceFactory class declaration for nsbaci	??
serviceFactories.h	Aggregate header for all service factories	??
baseResult.h	Base result class declaration for nsbaci services	??
compiler.h	Abstract Compiler class declaration for nsbaci	??

instruction.cpp	Instruction implementation for nsbaci compiler	??
instruction.h	Instruction definitions for nsbaci compiler	??
lexer.h	Lexer class declaration for nsbaci compiler	??
nsbaciCompiler.cpp	NsbaciCompiler class implementation for nsbaci	??
nsbaciCompiler.h	NsbaciCompiler class declaration for nsbaci	??
compilerService.cpp	Implementation of the CompilerService class for nsbaci	??
compilerService.h	CompilerService class declaration for nsbaci	??
drawingService.cpp	DrawingService class implementation for nsbaci	??
drawingService.h	DrawingService class declaration for nsbaci	??
fileService.cpp	Implementation of the FileService class for nsbaci	??
fileService.h	FileService class declaration for nsbaci	??
interpreter.h	Interpreter class declaration for nsbaci runtime service	??
nsbaciInterpreter.cpp	NsbaciInterpreter class implementation for nsbaci runtime service	??
nsbaciInterpreter.h	NsbaciInterpreter class declaration for nsbaci runtime service	??
program.cpp	Program class implementation for nsbaci runtime service	??
program.h	Program class declaration for nsbaci runtime service	??
runtimeService.cpp	Implementation unit for the RuntimeService	??
runtimeService.h	RuntimeService class declaration for nsbaci	??
nsbaciScheduler.cpp	NsbaciScheduler class implementation for nsbaci runtime service	??
nsbaciScheduler.h	NsbaciScheduler class declaration for nsbaci runtime service	??
scheduler.h	Scheduler class declaration for nsbaci runtime service	??
semaphore.h	Semaphore implementation for process synchronization	??
thread.cpp	Thread class implementation for nsbaci runtime service	??
thread.h	Thread class declaration for nsbaci runtime service	??
compilerTypes.h	Type definitions for compiler-related operations	??
drawingTypes.h	Type definitions for drawing-related operations	??
errorTypes.h	Type definitions for error-related structures	??
fileTypes.h	Type definitions for file-related operations	??
runtimeTypes.h	Type definitions for runtime-related operations	??

drawingWidget.cpp	DrawingWidget class implementation for nsbaci	??
drawingWidget.h	DrawingWidget class declaration for nsbaci	??
errorDialogFactory.cpp	Implementation of ErrorDialogFactory	??
errorDialogFactory.h	Factory for creating error dialogs from UIError objects	??
codeeditor.cpp	Implementation of the CodeEditor class with line numbers	??
codeeditor.h	CodeEditor class with line numbers for nsbaci	??
mainwindow.cpp	Implementation of the MainWindow class	??
mainwindow.h	Main window class declaration for nsbaci	??
preferencesDialog.cpp	Implementation of the PreferencesDialog class	??
preferencesDialog.h	Preferences dialog for nsbaci application settings	??
runtimeView.cpp	Implementation of the RuntimeView widget	??
runtimeView.h	RuntimeView widget declaration for nsbaci	??
userGuideDialog.cpp	Implementation of the UserGuideDialog class	??
userGuideDialog.h	User Guide dialog with integrated code examples for nsbaci	??

Chapter 7

Namespace Documentation

7.1 nsbaci Namespace Reference

Root namespace for the nsbaci application.

Namespaces

- namespace [factories](#)
Factories namespace for nsbaci.
- namespace [compiler](#)
Compiler namespace containing all compilation-related stuff.
- namespace [services](#)
Services namespace containing all backend service implementations.
- namespace [types](#)
Type definitions namespace for nsbaci (runtime-specific).
- namespace [ui](#)
User interface namespace for nsbaci.

Classes

- class [Controller](#)
Central coordinator between UI and backend services.
- class [Error](#)
Represents an error with a message and optional code.
- struct [UIError](#)
UI-ready error representation for display in dialogs.
- struct [BaseResult](#)
Base result structure for all service operations.

7.1.1 Detailed Description

Root namespace for the nsbaci application.

Main namespace for the nsbaci interpreter application.

Contains all components of the BACI concurrent programming interpreter including the controller, services, UI components, and type definitions.

7.2 nsbaci::compiler Namespace Reference

[Compiler](#) namespace containing all compilation-related stuff.

Classes

- struct [CompilerResult](#)
Result of a compilation operation.
- class [Compiler](#)
Abstract base class for all compilers.
- struct [Instruction](#)
Represents a single instruction in the virtual machine.
- class [Lexer](#)
Flex-based lexer for BACI source code.
- class [NsbaciCompiler](#)
nsbaci compiler implementation using flex and bison.

Typedefs

- using [Operand](#)
[Operand](#) types that an instruction can have.
- using **InstructionStream** = std::vector<[Instruction](#)>
Vector of instructions representing a compiled program.

Enumerations

- enum class [Opcode](#) : uint8_t {
LoadValue , **LoadAddress** , **LoadIndirect** , **StoreIndirect** ,
LoadBlock , **Store** , **StoreKeep** , **PushLiteral** ,
Swap , **RotateDown3** , **Index** , **CopyBlock** ,
ValueAt , **MarkStack** , **UpdateDisplay** , **Add** ,
Sub , **Mult** , **Div** , **Mod** ,
Negate , **Complement** , **And** , **Or** ,
TestEQ , **TestNE** , **TestLT** , **TestLE** ,
TestGT , **TestGE** , **TestEqualKeep** , **Jump** ,
JumpZero , **Call** , **ShortCall** , **ShortReturn** ,
ExitProc , **ExitFunction** , **EnterFrame** , **LeaveFrame** ,
Halt , **BeginFor** , **EndFor** , **Cobegin** ,
Coend , **Create** , **ThreadEnd** , **Suspend** ,
Revive , **WhichProc** , **Wait** , **Signal** ,
StoreSemaphore , **EnterMonitor** , **ExitMonitor** , **CallMonitorInit** ,
ReturnMonitorInit , **WaitCondition** , **SignalCondition** , **Empty** ,
Read , **ReadIn** , **Write** , **WriteIn** ,
WriteString , **WriteRawString** , **EolEof** , **Sprintf** ,
Sscanf , **CopyString** , **CopyRawString** , **ConcatString** ,
ConcatRawString , **CompareString** , **CompareRawString** , **LengthString** ,
DrawClear , **DrawRefresh** , **DrawSetColor** , **DrawSetColorAlpha** ,
DrawSetLineWidth , **DrawSetPosition** , **DrawCircle** , **DrawRectangle** ,
DrawTriangle , **DrawLine** , **DrawEllipse** , **DrawPixel** ,
DrawText , **FillCircle** , **FillRectangle** , **FillTriangle** ,
FillEllipse , **MoveTo** , **MoveBy** , **ChangeColor** ,
MakeVisible , **Remove** , **Random** , **Test** ,
_Count }
Opcodes for the BACI virtual machine instruction set.

Functions

- `const char * opcodeName (Opcode op)`
Get the string name of an opcode (for debugging/display).

7.2.1 Detailed Description

`Compiler` namespace containing all compilation-related stuff.

`Compiler` namespace for nsbaci.

7.2.2 Typedef Documentation

7.2.2.1 Operand

using `nsbaci::compiler::Operand`

Initial value:

```
std::variant<std::monostate,
             int32_t,
             uint32_t,
             std::string
             >
```

`Operand` types that an instruction can have.

7.2.3 Function Documentation

7.2.3.1 opcodeName()

```
const char * nsbaci::compiler::opcodeName (
    Opcode op)
```

Get the string name of an opcode (for debugging/display).

Parameters

<i>op</i>	The opcode to convert.
-----------	------------------------

Returns

String representation of the opcode.

Here is the caller graph for this function:



7.3 nsbaci::factories Namespace Reference

Factories namespace for nsbaci.

Classes

- struct [NsbaciCompiler](#)
- class [CompilerServiceFactory](#)
Factory for creating CompilerService instances.
- struct [DefaultDrawingBackend](#)
- class [DrawingServiceFactory](#)
Factory for creating DrawingService instances.
- struct [DefaultFileSystem](#)
- class [FileServiceFactory](#)
Factory for creating FileService instances.
- struct [NsbaciRuntime](#)
- class [RuntimeServiceFactory](#)
Factory for creating RuntimeService instances.

Variables

- constexpr [NsbaciCompiler](#) **nsbaciCompiler** {}
- constexpr [DefaultDrawingBackend](#) **defaultDrawingBackend** {}
- constexpr [DefaultFileSystem](#) **defaultFileSystem** {}
- constexpr [NsbaciRuntime](#) **nsbaciRuntime** {}

7.3.1 Detailed Description

Factories namespace for nsbaci.

7.4 nsbaci::services Namespace Reference

Services namespace containing all backend service implementations.

Namespaces

- namespace [runtime](#)
Runtime services namespace for nsbaci.

Classes

- class [CompilerService](#)
service for compiling nsbaci (or maybe other stuff in the future) source code.
- class [DrawingService](#)
Adapter service for graphical output backends.
- class [FileService](#)
Service for handling file system operations on BACI source files.
- struct [RuntimeResult](#)
Result of a runtime operation (step, run, etc.).
- class [RuntimeService](#)
Service that manages program execution.

Enumerations

- enum class `RuntimeState` { `Idle` , `Running` , `Paused` , `Halted` }

Possible states of the runtime service.

7.4.1 Detailed Description

Services namespace containing all backend service implementations.

Services namespace for nsbaci.

This namespace contains service classes that encapsulate specific functionality areas such as file I/O, compilation, and runtime execution.

7.4.2 Enumeration Type Documentation

7.4.2.1 RuntimeState

```
enum class nsbaci::services::RuntimeState [strong]
```

Possible states of the runtime service.

Represents the lifecycle states of program execution:

- Idle: Initial state, no program loaded or execution completed
- Running: Active execution in progress
- Paused: Execution suspended, can be resumed or stepped
- Halted: Program has finished (reached Halt instruction)

Enumerator

Idle	No program loaded or ready to start.
Running	Program is actively executing.
Paused	Execution paused, can step or continue.
Halted	Program has finished execution.

7.5 nsbaci::services::runtime Namespace Reference

Runtime services namespace for nsbaci.

Classes

- class [Interpreter](#)
Executes instructions for threads within a program context.
- class [NsbaciInterpreter](#)
BACI-specific implementation of the [Interpreter](#).
- class [Program](#)
Represents a compiled program ready for execution.
- class [NsbaciScheduler](#)
BACI-specific implementation of the [Scheduler](#).
- class [Scheduler](#)
Manages thread scheduling and state transitions.
- class [Semaphore](#)
Counting semaphore for process synchronization.
- class [Thread](#)
Represents a thread in the runtime service.

Typedefs

- using **OutputCallback** = std::function<void(const std::string&)>
Callback type for output operations.
- using **InputRequestCallback** = std::function<void(const std::string&)>
Callback type for input requests.
- using **DrawingCallback** = std::function<void(const [nsbaci::types::DrawCommand](#)&)>
Callback type for drawing operations.

7.5.1 Detailed Description

Runtime services namespace for nsbaci.

7.6 nsbaci::types Namespace Reference

Type definitions namespace for nsbaci (runtime-specific).

Classes

- struct [SymbolInfo](#)
Information about a variable/symbol.
- struct [Color](#)
RGB color representation with values from 0-255.
- struct [Point](#)
2D point/position representation.
- struct [Size](#)
2D size representation.
- struct [Circle](#)
[Circle](#) shape with center and radius.
- struct [Rectangle](#)

- [Rectangle](#) shape with position and size.
- struct [Triangle](#)
 - [Triangle](#) shape defined by three vertices.
- struct [Line](#)
 - [Line](#) segment from start to end point.
- struct [Ellipse](#)
 - [Ellipse](#) shape with center and radii.
- struct [Pixel](#)
 - Single pixel at a position.
- struct [DrawText](#)
- struct [Drawable](#)
 - Complete drawable object with shape, color, and visibility.
- struct [DrawCommand](#)
 - Represents a single drawing command to be executed.
- struct [CanvasConfig](#)
 - Configuration for the drawing canvas.
- struct [ErrorBase](#)
 - Base structure containing common error properties.
- struct [CompileError](#)
 - [Error](#) payload for compilation errors.
- struct [SaveError](#)
 - [Error](#) payload for file save errors.
- struct [LoadError](#)
 - [Error](#) payload for file load errors.
- struct [RuntimeError](#)
 - [Error](#) payload for runtime errors.

Typedefs

- using **StackValue** = int32_t
 - [Stack](#) value type (can hold int or address).
- using **Stack** = std::vector<[StackValue](#)>
 - Runtime stack.
- using **Memory** = std::vector<int32_t>
 - [Memory](#) block for runtime data.
- using [SemaphoreTable](#)
 - Maps memory addresses to semaphores.
- using **ThreadQueue** = std::queue<[nsbaci::services::runtime::Thread](#)>
 - Queue of threads for scheduler operations.
- using **VarName** = std::string
 - Type alias for variable names.
- using **MemoryAddr** = uint32_t
 - Type alias for memory addresses.
- using **SymbolTable** = std::unordered_map<[VarName](#), [SymbolInfo](#)>
 - Lookup table mapping variable names to their symbol info.
- using [Shape](#)
 - Variant type for all drawable shapes.
- using **ErrorMessage** = std::string
- using [ErrorPayload](#)
 - Variant type for all possible error payloads.

- using **Text** = std::string
Alias for text content (file contents, source code, etc.).
- using **File** = fs::path
Alias for file system paths.
- using **ThreadID** = unsigned long long int
- using **Priority** = unsigned long int

Enumerations

- enum class **DrawCommandType** {
Clear , **SetColor** , **SetPosition** , **DrawShape** ,
Fill , **SetLineWidth** , **Refresh** }
Types of drawing commands that can be executed.
- enum class **StandardPosition** {
TopLeft , **TopCenter** , **TopRight** , **CenterLeft** ,
Center , **CenterRight** , **BottomLeft** , **BottomCenter** ,
BottomRight }
- enum class **ErrSeverity** { **Warning** , **Error** , **Fatal** }
Severity levels for errors.
- enum class **ErrType** {
emptyPath , **invalidPath** , **invalidExtension** , **directoryNotFound** ,
fileNotFound , **notARegularFile** , **permissionDenied** , **openFailed** ,
readFailed , **writeFailed** , **compilationError** , **unknown** }
Types of errors that can occur in the application.
- enum class **ThreadState** {
Ready , **Running** , **Blocked** , **Waiting** ,
IO , **Terminated** }

Functions

- **Point resolvePosition** (StandardPosition pos, **Size** canvasSize)
Resolve a standard position to actual coordinates.

7.6.1 Detailed Description

Type definitions namespace for nsbaci (runtime-specific).

Type definitions namespace for nsbaci.

7.6.2 Typedef Documentation

7.6.2.1 ErrorPayload

```
using nsbaci::types::ErrorPayload
```

Initial value:

```
std::variant<SaveError, LoadError, CompileError, RuntimeError>
```

Variant type for all possible error payloads.

Can be used to create a more specific error message in the controller, for example when an error is of type compile↵ Error, the controller can specify the line, col, what might have happened...

7.6.2.2 SemaphoreTable

```
using nsbaci::types::SemaphoreTable
```

Initial value:

```
std::unordered_map<MemoryAddr, nsbaci::services::runtime::Semaphore>
```

Maps memory addresses to semaphores.

- Compile time: Compiler emits StoreSemaphore instructions with addresses
- Runtime: Executing StoreSemaphore adds entry to this table
- Wait/Signal: Look up semaphore by address

7.6.2.3 Shape

```
using nsbaci::types::Shape
```

Initial value:

```
std::variant<Circle, Rectangle, Triangle, Line, Ellipse, Pixel, DrawText>
```

Variant type for all drawable shapes.

7.6.3 Enumeration Type Documentation

7.6.3.1 ThreadState

```
enum class nsbaci::types::ThreadState [strong]
```

Enumerator

Ready	Thread is ready to run.
Running	Thread is currently executing.
Blocked	Thread is blocked (e.g., waiting on semaphore).
Waiting	Thread is waiting for I/O.
IO	Thread is performing I/O.
Terminated	Thread has finished execution.

7.6.4 Function Documentation

7.6.4.1 resolvePosition()

```
Point nsbaci::types::resolvePosition (
    StandardPosition pos,
    Size canvasSize) [inline]
```

Generated by Doxygen

Resolve a standard position to actual coordinates.

Parameters

<i>pos</i>	The standard position enum value.
<i>canvasSize</i>	The size of the canvas.

Returns

The resolved [Point](#) coordinates.

7.7 nsbaci::ui Namespace Reference

User interface namespace for nsbaci.

Classes

- class [DrawingWidget](#)
Qt widget that provides a drawing canvas.
- class [ErrorDialogFactory](#)
Factory for creating error dialogs from [UIError](#) objects.
- struct [Preferences](#)
Holds all user-configurable preferences.
- class [PreferencesDialog](#)
A dialog that allows the user to configure application preferences.
- struct [ThreadInfo](#)
Information about a thread for display.
- struct [VariableInfo](#)
Information about a variable for display.
- class [RuntimeView](#)
Widget displaying runtime execution state.
- struct [ExampleEntry](#)
Data for a single example entry in the User Guide.
- class [UserGuideDialog](#)
A dialog that provides an integrated user guide with code examples.

Enumerations

- enum class [Theme](#) { **Dark** , **Light** }
Enumerates the available application themes.

7.7.1 Detailed Description

User interface namespace for nsbaci.

Chapter 8

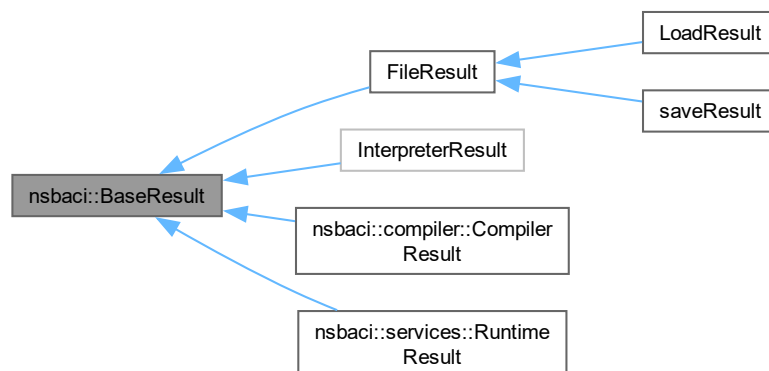
Class Documentation

8.1 nsbaci::BaseResult Struct Reference

Base result structure for all service operations.

```
#include <baseResult.h>
```

Inheritance diagram for nsbaci::BaseResult:



Public Member Functions

- **BaseResult** ()
Default constructor creates a successful result.
- **BaseResult** (std::vector< nsbaci::Error > errs)
Constructs a result from a vector of errors.
- **BaseResult** (nsbaci::Error error)
Constructs a failed result from a single error.
- **BaseResult** (BaseResult &&) noexcept=default
- **BaseResult** & operator= (BaseResult &&) noexcept=default
- **BaseResult** (const BaseResult &)=default
- **BaseResult** & operator= (const BaseResult &)=default

Public Attributes

- `bool ok`
True if the operation succeeded.
- `std::vector< nsbaci::Error > errors`
Errors encountered (empty if ok is true).

8.1.1 Detailed Description

Base result structure for all service operations.

`BaseResult` provides a common interface for service operation results, encapsulating success/failure status and any associated error information. All service-specific result types should inherit from this base to ensure consistent error handling patterns throughout the application.

Invariant

If `ok` is false, errors vector contains at least one error.

If `ok` is true, errors vector is empty.

Usage example:

```
BaseResult result = someService.doOperation();
if (!result.ok) {
    for (const auto& err : result.errors) {
        handleError(err);
    }
}
```

8.1.2 Constructor & Destructor Documentation

8.1.2.1 BaseResult() [1/2]

```
nsbaci::BaseResult::BaseResult (
    std::vector< nsbaci::Error > errs) [inline], [explicit]
```

Constructs a result from a vector of errors.

If the error vector is empty, the result is considered successful.

Parameters

<code>errs</code>	Vector of errors encountered during the operation.
-------------------	--

8.1.2.2 BaseResult() [2/2]

```
nsbaci::BaseResult::BaseResult (
    nsbaci::Error error) [inline], [explicit]
```

Constructs a failed result from a single error.

Parameters

<i>error</i>	The error that caused the operation to fail.
--------------	--

The documentation for this struct was generated from the following file:

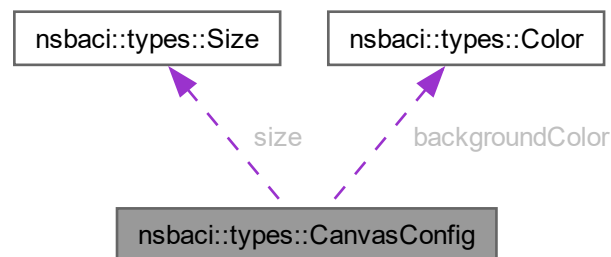
- [baseResult.h](#)

8.2 nsbaci::types::CanvasConfig Struct Reference

Configuration for the drawing canvas.

```
#include <drawingTypes.h>
```

Collaboration diagram for nsbaci::types::CanvasConfig:



Public Attributes

- [Size](#) **size** {800, 600}
- [Color](#) **backgroundColor** {255, 255, 255}
- `std::string` **title** = "NSBACI Canvas"

8.2.1 Detailed Description

Configuration for the drawing canvas.

The documentation for this struct was generated from the following file:

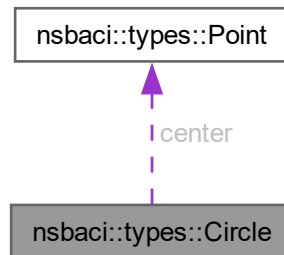
- [drawingTypes.h](#)

8.3 nsbaci::types::Circle Struct Reference

[Circle](#) shape with center and radius.

```
#include <drawingTypes.h>
```

Collaboration diagram for nsbaci::types::Circle:



Public Member Functions

- **Circle** ([Point](#) c, `int32_t` r, `bool` fill=false)

Public Attributes

- [Point](#) **center**
- `int32_t` **radius** = 0
- `bool` **filled** = false

8.3.1 Detailed Description

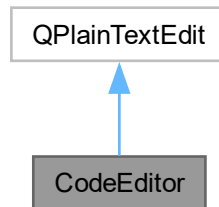
[Circle](#) shape with center and radius.

The documentation for this struct was generated from the following file:

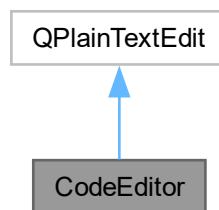
- [drawingTypes.h](#)

8.4 CodeEditor Class Reference

Inheritance diagram for CodeEditor:



Collaboration diagram for CodeEditor:



Public Member Functions

- **CodeEditor** (QWidget *parent=nullptr)
- void **lineNumberAreaPaintEvent** (QPaintEvent *event)
- int **lineNumberAreaWidth** ()
- void **setLineNumbersVisible** (bool visible)
Show or hide the line number gutter.
- bool **lineNumbersVisible** () const
- void **setLightTheme** (bool light)
Set whether to use light theme colors for line numbers and highlights.

Protected Member Functions

- void **resizeEvent** (QResizeEvent *event) override
- void **changeEvent** (QEvent *event) override

The documentation for this class was generated from the following files:

- [codeeditor.h](#)
- [codeeditor.cpp](#)

8.5 nsbaci::types::Color Struct Reference

RGB color representation with values from 0-255.

```
#include <drawingTypes.h>
```

Public Member Functions

- constexpr **Color** (uint8_t red, uint8_t green, uint8_t blue, uint8_t alpha=255)
- constexpr bool **operator==** (const Color &other) const

Public Attributes

- uint8_t **r** = 0
- uint8_t **g** = 0
- uint8_t **b** = 0
- uint8_t **a** = 255

8.5.1 Detailed Description

RGB color representation with values from 0-255.

The documentation for this struct was generated from the following file:

- [drawingTypes.h](#)

8.6 nsbaci::types::CompileError Struct Reference

[Error](#) payload for compilation errors.

```
#include <errorTypes.h>
```

Public Attributes

- int **line** = 0
- int **column** = 0

8.6.1 Detailed Description

[Error](#) payload for compilation errors.

The documentation for this struct was generated from the following file:

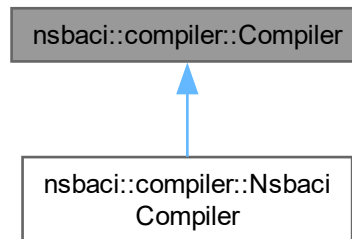
- [errorTypes.h](#)

8.7 nsbaci::compiler::Compiler Class Reference

Abstract base class for all compilers.

```
#include <compiler.h>
```

Inheritance diagram for nsbaci::compiler::Compiler:



Public Member Functions

- **Compiler** ()=default
Default constructor.
- virtual **~Compiler** ()=default
Virtual destructor.
- virtual **CompilerResult compile** (const std::string &source)=0
Compiles source code from a string.
- virtual **CompilerResult compile** (std::istream &input)=0
Compiles source code from an input stream.

8.7.1 Detailed Description

Abstract base class for all compilers.

The **Compiler** interface defines the contract for compiling source code into p-code instructions. Implementations handle lexical analysis, parsing, semantic analysis, and code generation.

The compilation process produces:

- An instruction stream (p-code) for the virtual machine
- A symbol table mapping variable names to their types and addresses
- Compilation errors, if there are

Subclasses must implement both **compile()** overloads to support compilation from both strings and input streams.

Parameters

<i>source</i>	The BACI source code to compile.
---------------	----------------------------------

Returns

[CompilerResult](#) containing instructions on success, or errors on failure.

Implemented in [nsbaci::compiler::NsbaciCompiler](#).

8.7.2.2 compile() [2/2]

```
virtual CompilerResult nsbaci::compiler::Compiler::compile (  
    std::istream & input) [pure virtual]
```

Compiles source code from an input stream.

Allows compilation from files or other stream sources.

Parameters

<i>input</i>	The input stream containing BACI source code.
--------------	---

Returns

[CompilerResult](#) containing instructions on success, or errors on failure.

Implemented in [nsbaci::compiler::NsbaciCompiler](#).

The documentation for this class was generated from the following file:

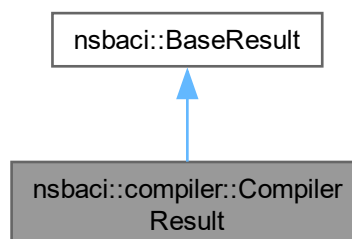
- [compiler.h](#)

8.8 nsbaci::compiler::CompilerResult Struct Reference

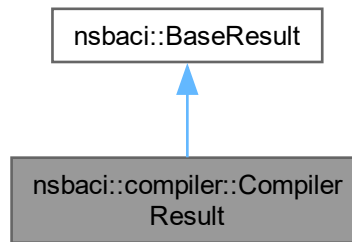
Result of a compilation operation.

```
#include <compiler.h>
```

Inheritance diagram for nsbaci::compiler::CompilerResult:



Collaboration diagram for nsbaci::compiler::CompilerResult:



Public Member Functions

- **CompilerResult** ()
Default constructor creates a successful empty result.
- **CompilerResult** (std::vector< nsbaci::Error > errs)
Constructs a failed result from a vector of errors.
- **CompilerResult** (nsbaci::Error error)
Constructs a failed result from a single error.
- **CompilerResult** (CompilerResult &&) noexcept=default
- **CompilerResult** & **operator=** (CompilerResult &&) noexcept=default
- **CompilerResult** (const CompilerResult &)=default
- **CompilerResult** & **operator=** (const CompilerResult &)=default

Public Member Functions inherited from nsbaci::BaseResult

- **BaseResult** ()
Default constructor creates a successful result.
- **BaseResult** (std::vector< nsbaci::Error > errs)
Constructs a result from a vector of errors.
- **BaseResult** (nsbaci::Error error)
Constructs a failed result from a single error.
- **BaseResult** (BaseResult &&) noexcept=default
- **BaseResult** & **operator=** (BaseResult &&) noexcept=default
- **BaseResult** (const BaseResult &)=default
- **BaseResult** & **operator=** (const BaseResult &)=default

Public Attributes

- **InstructionStream** instructions
Generated p-code instructions.
- **nsbaci::types::SymbolTable** symbols
Symbol table from compilation.

Public Attributes inherited from `nsbaci::BaseResult`

- `bool ok`
True if the operation succeeded.
- `std::vector< nsbaci::Error > errors`
Errors encountered (empty if ok is true).

8.8.1 Detailed Description

Result of a compilation operation.

Contains the outcome of a compilation attempt including success/failure status, any error messages, and on success, the generated instruction stream and symbol table.

Note

On failure, the instructions and symbols fields should not be used.

8.8.2 Constructor & Destructor Documentation

8.8.2.1 `CompilerResult()` [1/2]

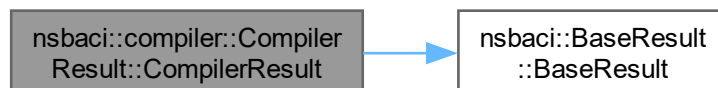
```
nsbaci::compiler::CompilerResult::CompilerResult (
    std::vector< nsbaci::Error > errs) [inline], [explicit]
```

Constructs a failed result from a vector of errors.

Parameters

<code>errs</code>	Vector of compilation errors.
-------------------	-------------------------------

Here is the call graph for this function:



8.8.2.2 `CompilerResult()` [2/2]

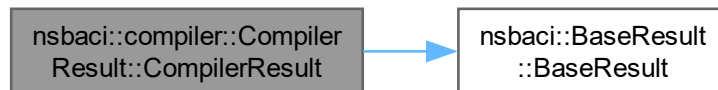
```
nsbaci::compiler::CompilerResult::CompilerResult (
    nsbaci::Error error) [inline], [explicit]
```

Constructs a failed result from a single error.

Parameters

<i>error</i>	The compilation error.
--------------	------------------------

Here is the call graph for this function:



The documentation for this struct was generated from the following file:

- [compiler.h](#)

8.9 nsbaci::services::CompilerService Class Reference

service for compiling nsbaci (or maybe other stuff in the future) source code.

```
#include <compilerService.h>
```

Public Member Functions

- [CompilerService](#) (std::unique_ptr< [nsbaci::compiler::Compiler](#) > c=std::make_unique< [nsbaci::compiler::NsbaciCompiler](#) >())
Constructs the [CompilerService](#) with a compiler implementation.
- [~CompilerService](#) ()=default
Default destructor.
- [CompilerService](#) (const [CompilerService](#) &)=delete
- [CompilerService](#) & [operator=](#) (const [CompilerService](#) &)=delete
- [CompilerService](#) ([CompilerService](#) &&)=default
- [CompilerService](#) & [operator=](#) ([CompilerService](#) &&)=default
- [nsbaci::compiler::CompilerResult](#) [compile](#) ([nsbaci::types::Text](#) raw)
Compiles nsbaci source code into p-code instructions.
- bool [hasProgramReady](#) () const
Checks if a compiled program is available for execution.
- [nsbaci::compiler::InstructionStream](#) [takeInstructions](#) ()
Retrieves and releases ownership of the compiled instructions.
- [nsbaci::types::SymbolTable](#) [takeSymbols](#) ()
Retrieves and releases ownership of the symbol table.

8.9.1 Detailed Description

service for compiling nsbaci (or maybe other stuff in the future) source code.

The service functions like the following:

1. Call `compile()` with source code
2. Check `hasProgramReady()` to verify success
3. Call `takeInstructions()` and `takeSymbols()` to retrieve compiled data

After taking the instructions, the program is no longer considered ready until a new successful compilation occurs.

Usage example:

```
CompilerService cs;
auto result = cs.compile(sourceCode);
if (result.ok && cs.hasProgramReady()) {
    auto instructions = cs.takeInstructions();
    auto symbols = cs.takeSymbols();
    // Load into runtime...
}
```

8.9.2 Constructor & Destructor Documentation

8.9.2.1 CompilerService()

```
nsbaci::services::CompilerService::CompilerService (
    std::unique_ptr< nsbaci::compiler::Compiler > c = std::make_unique<nsbaci::compiler::NsbaciCompi
```

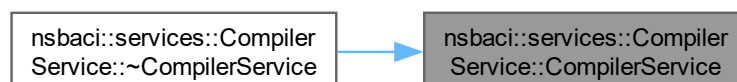
Constructs the `CompilerService` with a compiler implementation.

By default, creates an `NsbaciCompiler` instance. Custom compiler implementations can be injected for testing or alternative language support.

Parameters

<code>c</code>	Unique pointer to a Compiler implementation. Defaults to <code>NsbaciCompiler</code> .
----------------	--

Here is the caller graph for this function:



8.9.3 Member Function Documentation

8.9.3.1 compile()

```
nsbaci::compiler::CompilerResult nsbaci::services::CompilerService::compile (
```


Parameters

<i>raw</i>	The nsbaci source code to compile.
------------	------------------------------------

Returns

CompilerResult containing success status and any compilation errors.

8.9.3.2 hasProgramReady()

```
bool nsbaci::services::CompilerService::hasProgramReady () const
```

Checks if a compiled program is available for execution.

Returns true after a successful [compile\(\)](#) call and before [takeInstructions\(\)](#) is called. Used to verify that valid instructions are available before attempting to load them into the runtime.

Returns

True if compiled instructions are available, false otherwise.

8.9.3.3 takeInstructions()

```
nsbaci::compiler::InstructionStream nsbaci::services::CompilerService::takeInstructions ()
```

Retrieves and releases ownership of the compiled instructions.

Moves the instruction stream out of the service. After this call, [hasProgramReady\(\)](#) will return false until a new successful compilation.

Returns

The compiled instruction stream.

Warning

Only call when [hasProgramReady\(\)](#) returns true.

8.9.3.4 takeSymbols()

```
nsbaci::types::SymbolTable nsbaci::services::CompilerService::takeSymbols ()
```

Retrieves and releases ownership of the symbol table.

Moves the symbol table out of the service. The symbol table contains information about all declared variables including names, types, and memory addresses.

Returns

The symbol table from the last successful compilation.

The documentation for this class was generated from the following files:

- [compilerService.h](#)
- [compilerService.cpp](#)

8.10 nsbaci::factories::CompilerServiceFactory Class Reference

Factory for creating CompilerService instances.

```
#include <compilerServiceFactory.h>
```

Static Public Member Functions

- static [nsbaci::services::CompilerService](#) **createService** ([NsbaciCompiler](#))

8.10.1 Detailed Description

Factory for creating CompilerService instances.

The documentation for this class was generated from the following files:

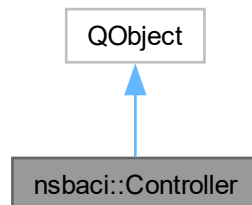
- [compilerServiceFactory.h](#)
- [compilerServiceFactory.cpp](#)

8.11 nsbaci::Controller Class Reference

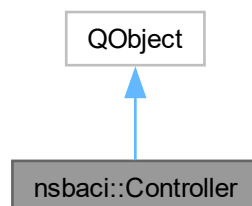
Central coordinator between UI and backend services.

```
#include <controller.h>
```

Inheritance diagram for nsbaci::Controller:



Collaboration diagram for nsbaci::Controller:



Public Slots

- void [onSaveRequested](#) (nsbaci::types::File file, nsbaci::types::Text contents)
Handles a request to save source code to a file.
- void [onOpenRequested](#) (nsbaci::types::File file)
Handles a request to open and load a source file.
- void [onCompileRequested](#) (nsbaci::types::Text contents)
Handles a request to compile source code.
- void [onRunRequested](#) ()
Handles a request to load and prepare a compiled program for execution.
- void [onStepRequested](#) ()
Executes a single instruction across any ready thread.
- void [onStepThreadRequested](#) (nsbaci::types::ThreadID threadId)
Executes a single instruction on a specific thread.
- void [onRunContinueRequested](#) ()
Starts or resumes continuous execution mode.
- void [onPauseRequested](#) ()
Pauses continuous execution.
- void [onResetRequested](#) ()
Resets the runtime to initial state.
- void [onStopRequested](#) ()
Stops execution and unloads the program.
- void [onInputProvided](#) (const QString &input)
Provides user input to the runtime.

Signals

- void [saveFailed](#) (std::vector< [UIError](#) > errors)
Emitted when a file save operation fails.
- void [saveSucceeded](#) ()
Emitted when a file save operation succeeds.
- void [loadFailed](#) (std::vector< [UIError](#) > errors)
Emitted when a file load operation fails.
- void [loadSucceeded](#) (const QString &contents)
Emitted when a file load operation succeeds.
- void [compileFailed](#) (std::vector< [UIError](#) > errors)
Emitted when compilation fails.
- void [compileSucceeded](#) ()
Emitted when compilation succeeds.
- void [runStarted](#) (const QString &programName)
Emitted when a program is loaded and ready for execution.
- void [runtimeStateChanged](#) (bool running, bool halted)
Emitted when the runtime execution state changes.
- void [threadsUpdated](#) (const std::vector< [nsbaci::ui::ThreadInfo](#) > &threads)
Emitted when thread information needs to be updated in the UI.
- void [variablesUpdated](#) (const std::vector< [nsbaci::ui::VariableInfo](#) > &variables)
Emitted when variable information needs to be updated in the UI.
- void [outputReceived](#) (const QString &output)
Emitted when the runtime produces output (cout, writeln, etc.).
- void [inputRequested](#) (const QString &prompt)
Emitted when the runtime needs user input (cin, read, etc.).

Public Member Functions

- [Controller](#) ([nsbaci::services::FileService](#) &&*f*, [nsbaci::services::CompilerService](#) &&*c*, [nsbaci::services::RuntimeService](#) &&*r*, `std::unique_ptr< nsbaci::services::DrawingService >` *d*, `QObject *parent=nullptr`)
Constructs the [Controller](#) with all required services.
- [~Controller](#) ()=default
Default destructor.

8.11.1 Detailed Description

Central coordinator between UI and backend services.

The [Controller](#) class is a `QObject` that serves as the main application controller, implementing the MVC pattern. It receives user actions through Qt slots, processes them using the specific backend services, and communicates results back to the UI through Qt signals.

The controller owns instances of all required services:

- `FileService`: Handles file system operations
- `CompilerService`: Compiles NsBaci source code to p-code
- `RuntimeService`: Executes compiled programs with thread scheduling
- `DrawingService`: Not yet implemented, but will be used as graphical API in the future

Execution modes supported:

- Single-step execution: Execute one instruction at a time
- Continuous execution: Run program with timer-based batching
- Thread-specific stepping: Execute a single thread's instruction

Note

The controller uses a `QTimer` for continuous execution to maintain UI responsiveness during long-running programs.

8.11.2 Constructor & Destructor Documentation

8.11.2.1 Controller()

```
nsbaci::Controller::Controller (
    nsbaci::services::FileService && f,
    nsbaci::services::CompilerService && c,
    nsbaci::services::RuntimeService && r,
    std::unique_ptr< nsbaci::services::DrawingService > d,
    QObject * parent = nullptr) [explicit]
```

Constructs the [Controller](#) with all required services.

Takes ownership of the provided services via move semantics. Initializes the internal `QTimer` used for continuous execution mode.

Parameters

<i>f</i>	FileService instance for file operations.
<i>c</i>	CompilerService instance for compilation.
<i>r</i>	RuntimeService instance for program execution.
<i>d</i>	DrawingService instance for graphical output (unique_ptr).
<i>parent</i>	Optional parent QObject for Qt memory management.

8.11.2.2 ~Controller()

```
nsbaci::Controller::~~Controller () [default]
```

Default destructor.

The QTimer is automatically cleaned up through Qt's parent-child system.

8.11.3 Member Function Documentation**8.11.3.1 compileFailed**

```
void nsbaci::Controller::compileFailed (
    std::vector< UIError > errors) [signal]
```

Emitted when compilation fails.

Parameters

<i>errors</i>	List of compilation errors with line/column information.
---------------	--

Here is the caller graph for this function:



8.11.3.2 compileSucceeded

```
void nsbaci::Controller::compileSucceeded () [signal]
```

Emitted when compilation succeeds.

After this signal, the compiled program is ready to be loaded into the runtime via [onRunRequested\(\)](#). Here is the caller graph for this function:



8.11.3.3 inputRequested

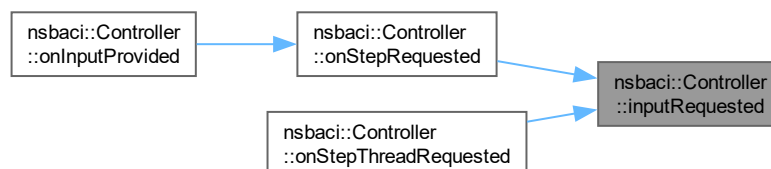
```
void nsbaci::Controller::inputRequested (
    const QString & prompt) [signal]
```

Emitted when the runtime needs user input (cin, read, etc.).

Parameters

<i>prompt</i>	The prompt message to display to the user.
---------------	--

Here is the caller graph for this function:



8.11.3.4 loadFailed

```
void nsbaci::Controller::loadFailed (
    std::vector< UIError > errors) [signal]
```

Emitted when a file load operation fails.

Parameters

<i>errors</i>	List of errors describing what went wrong.
---------------	--

Here is the caller graph for this function:

**8.11.3.5 loadSucceeded**

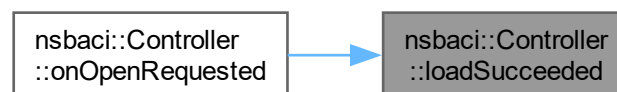
```
void nsbaci::Controller::loadSucceeded (
    const QString & contents) [signal]
```

Emitted when a file load operation succeeds.

Parameters

<i>contents</i>	The loaded file contents as a QString.
-----------------	--

Here is the caller graph for this function:

**8.11.3.6 onCompileRequested**

```
void nsbaci::Controller::onCompileRequested (
    nsbaci::types::Text contents) [slot]
```

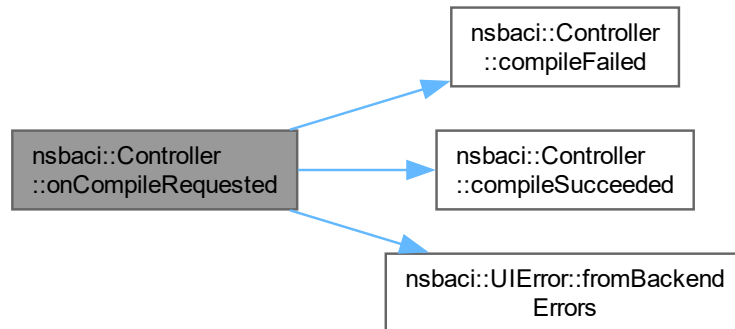
Handles a request to compile source code.

Compiles the provided source code into p-code instructions. On success, the instructions are stored in the `CompilerService` ready to be loaded into the runtime.

Parameters

<i>contents</i>	The BACI source code to compile.
-----------------	----------------------------------

Here is the call graph for this function:

**8.11.3.7 onInputProvided**

```
void nsbaci::Controller::onInputProvided (
    const QString & input) [slot]
```

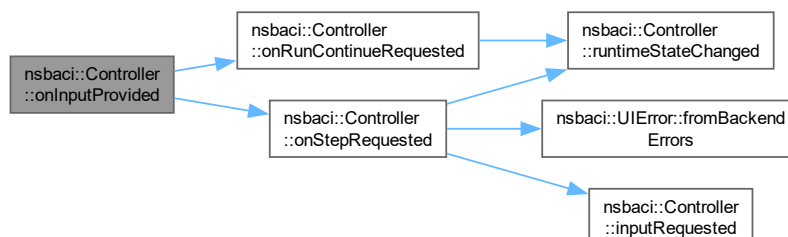
Provides user input to the runtime.

Called when the user enters input in response to an `inputRequested` signal. If the program was running continuously before the input request, execution is automatically resumed.

Parameters

<i>input</i>	The user-provided input string.
--------------	---------------------------------

Here is the call graph for this function:



8.11.3.8 onOpenRequested

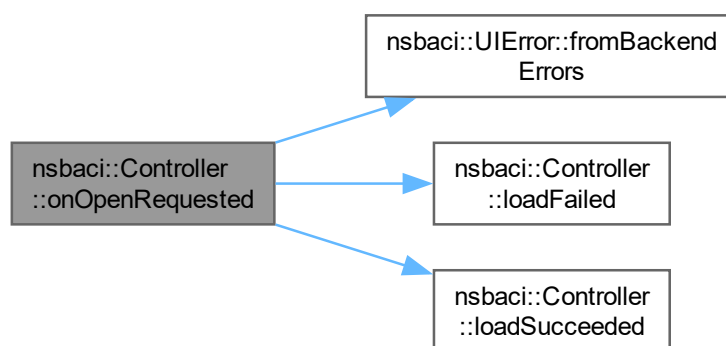
```
void nsbaci::Controller::onOpenRequested (
    nsbaci::types::File file) [slot]
```

Handles a request to open and load a source file.

Parameters

<i>file</i>	The file path to load.
-------------	------------------------

Here is the call graph for this function:

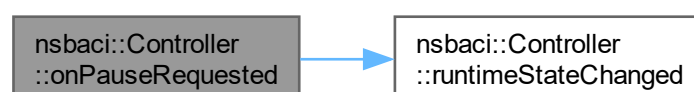


8.11.3.9 onPauseRequested

```
void nsbaci::Controller::onPauseRequested () [slot]
```

Pauses continuous execution.

Stops the execution timer but preserves program state for later resumption. Here is the call graph for this function:

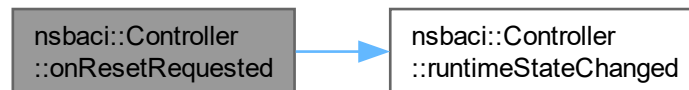


8.11.3.10 onResetRequested

```
void nsbaci::Controller::onResetRequested () [slot]
```

Resets the runtime to initial state.

Clears all thread state and memory but keeps the loaded program. The program can be re-run from the beginning. Here is the call graph for this function:



8.11.3.11 onRunContinueRequested

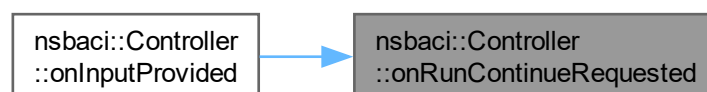
```
void nsbaci::Controller::onRunContinueRequested () [slot]
```

Starts or resumes continuous execution mode.

Begins timer-based execution where batches of instructions are executed periodically, allowing the UI to remain responsive. Here is the call graph for this function:



Here is the caller graph for this function:

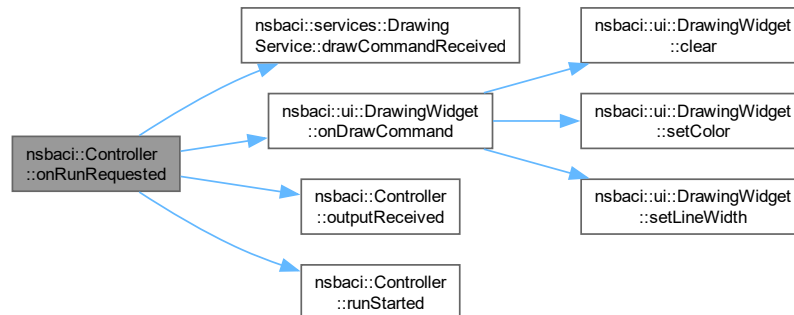


8.11.3.12 onRunRequested

```
void nsbaci::Controller::onRunRequested () [slot]
```

Handles a request to load and prepare a compiled program for execution.

Takes the compiled instructions and symbol table from the CompilerService and loads them into the RuntimeService. Sets up the output callback for forwarding runtime output to the UI. Here is the call graph for this function:



8.11.3.13 onSaveRequested

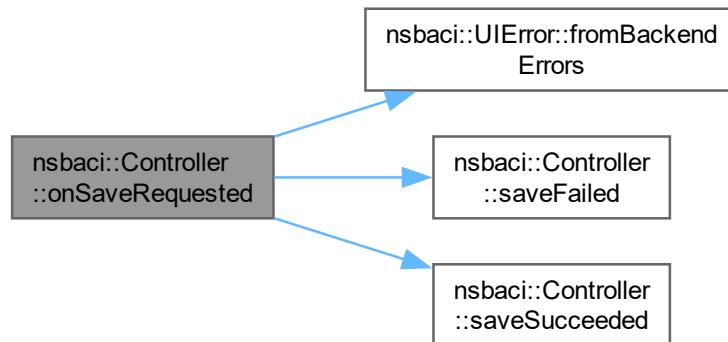
```
void nsbaci::Controller::onSaveRequested (
    nsbaci::types::File file,
    nsbaci::types::Text contents) [slot]
```

Handles a request to save source code to a file.

Parameters

<i>file</i>	The target file path.
<i>contents</i>	The source code content to save.

Here is the call graph for this function:

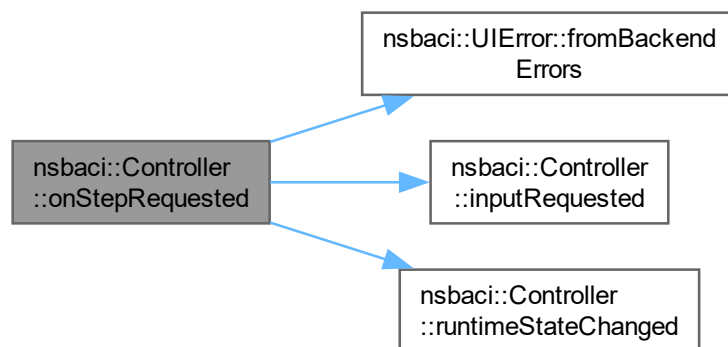


8.11.3.14 onStepRequested

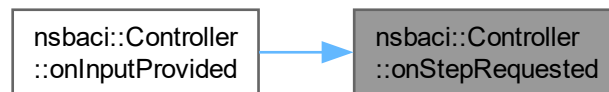
```
void nsbaci::Controller::onStepRequested () [slot]
```

Executes a single instruction across any ready thread.

The scheduler picks the next thread to run and executes one instruction. Updates the UI with new thread and variable states. Here is the call graph for this function:



Here is the caller graph for this function:



8.11.3.15 onStepThreadRequested

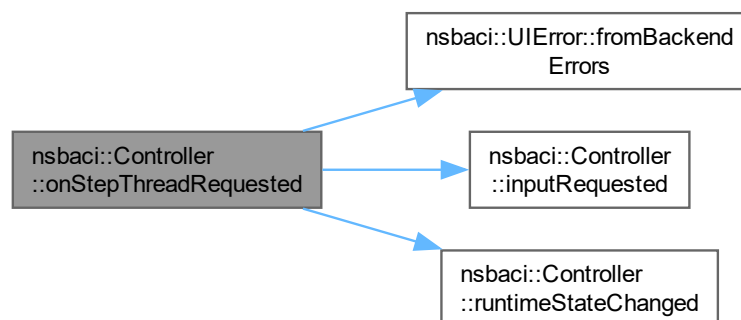
```
void nsbaci::Controller::onStepThreadRequested (
    nsbaci::types::ThreadID threadId) [slot]
```

Executes a single instruction on a specific thread.

Parameters

<i>threadId</i>	The ID of the thread to step.
-----------------	-------------------------------

Here is the call graph for this function:



8.11.3.16 onStopRequested

```
void nsbaci::Controller::onStopRequested () [slot]
```

Stops execution and unloads the program.

Completely stops the runtime and marks no program as loaded.

8.11.3.17 outputReceived

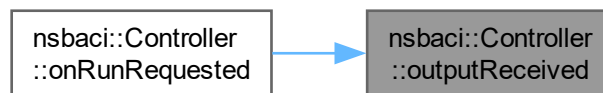
```
void nsbaci::Controller::outputReceived (
    const QString & output) [signal]
```

Emitted when the runtime produces output (cout, writeln, etc.).

Parameters

<i>output</i>	The output string to display in the console.
---------------	--

Here is the caller graph for this function:



8.11.3.18 runStarted

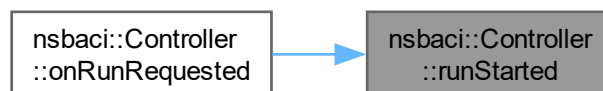
```
void nsbaci::Controller::runStarted (
    const QString & programName) [signal]
```

Emitted when a program is loaded and ready for execution.

Parameters

<i>programName</i>	The name of the loaded program.
--------------------	---------------------------------

Here is the caller graph for this function:



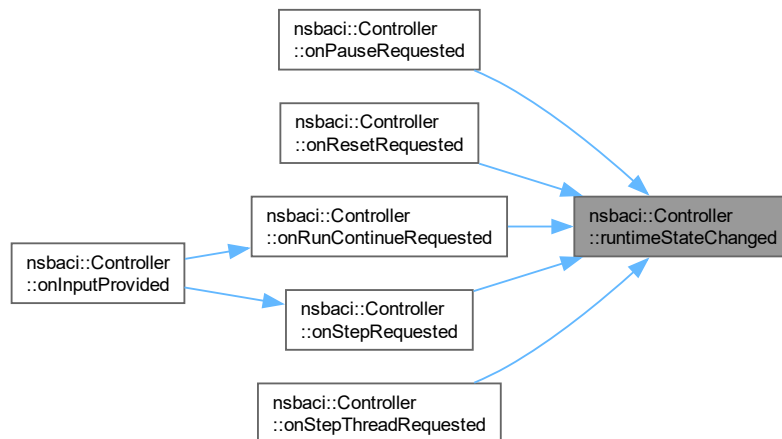
8.11.3.19 runtimeStateChanged

```
void nsbaci::Controller::runtimeStateChanged (
    bool running,
    bool halted) [signal]
```

Parameters

<i>running</i>	True if the program is currently executing continuously.
<i>halted</i>	True if the program has terminated (reached Halt instruction).

Here is the caller graph for this function:

**8.11.3.20 saveFailed**

```
void nsbaci::Controller::saveFailed (
    std::vector< UIError > errors) [signal]
```

Emitted when a file save operation fails.

Parameters

<i>errors</i>	List of errors describing what went wrong.
---------------	--

Here is the caller graph for this function:



8.11.3.21 threadsUpdated

```
void nsbaci::Controller::threadsUpdated (
    const std::vector< nsbaci::ui::ThreadInfo > & threads) [signal]
```

Emitted when thread information needs to be updated in the UI.

Parameters

<i>threads</i>	Current state of all threads including PC, state, and current instruction.
----------------	--

8.11.3.22 variablesUpdated

```
void nsbaci::Controller::variablesUpdated (
    const std::vector< nsbaci::ui::VariableInfo > & variables) [signal]
```

Emitted when variable information needs to be updated in the UI.

Parameters

<i>variables</i>	Current values of all program variables.
------------------	--

The documentation for this class was generated from the following files:

- [controller.h](#)
- [controller.cpp](#)

8.12 nsbaci::factories::DefaultDrawingBackend Struct Reference

The documentation for this struct was generated from the following file:

- [drawingServiceFactory.h](#)

8.13 nsbaci::factories::DefaultFileSystem Struct Reference

The documentation for this struct was generated from the following file:

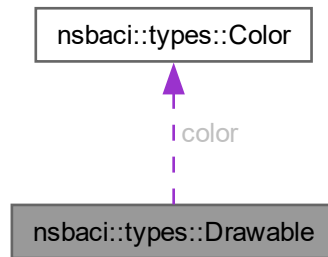
- [fileServiceFactory.h](#)

8.14 nsbaci::types::Drawable Struct Reference

Complete drawable object with shape, color, and visibility.

```
#include <drawingTypes.h>
```

Collaboration diagram for nsbaci::types::Drawable:



Public Member Functions

- **Drawable** ([Shape](#) s, [Color](#) c, bool vis=true, int32_t z=0)

Public Attributes

- [Shape](#) shape
- [Color](#) color
- bool **visible** = true
- int32_t **zIndex** = 0

8.14.1 Detailed Description

Complete drawable object with shape, color, and visibility.

The documentation for this struct was generated from the following file:

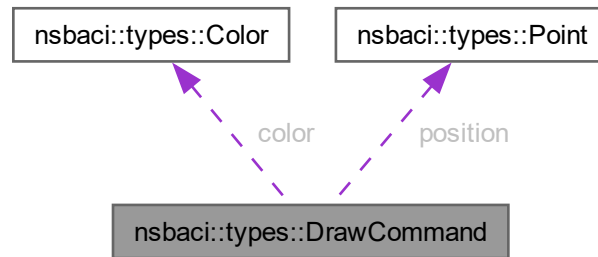
- [drawingTypes.h](#)

8.15 nsbaci::types::DrawCommand Struct Reference

Represents a single drawing command to be executed.

```
#include <drawingTypes.h>
```

Collaboration diagram for nsbaci::types::DrawCommand:



Static Public Member Functions

- static [DrawCommand](#) **clear** ()
- static [DrawCommand](#) **clearWithColor** ([Color](#) c)
- static [DrawCommand](#) **setColor** ([Color](#) c)
- static [DrawCommand](#) **setPosition** ([Point](#) p)
- static [DrawCommand](#) **drawShape** ([Shape](#) s, [Color](#) c)
- static [DrawCommand](#) **fill** ([Color](#) c)
- static [DrawCommand](#) **setLineWidth** (int32_t width)
- static [DrawCommand](#) **refresh** ()

Public Attributes

- [DrawCommandType](#) **type**
- [Color](#) **color**
- [Point](#) **position**
- [Shape](#) **shape**
- int32_t **lineWidth** = 1

8.15.1 Detailed Description

Represents a single drawing command to be executed.

The documentation for this struct was generated from the following file:

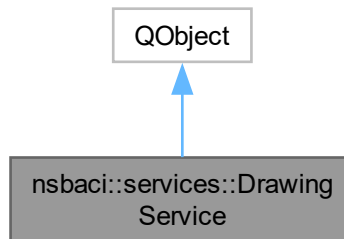
- [drawingTypes.h](#)

8.16 nsbaci::services::DrawingService Class Reference

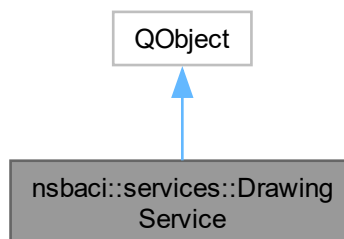
Adapter service for graphical output backends.

```
#include <drawingService.h>
```

Inheritance diagram for nsbaci::services::DrawingService:



Collaboration diagram for nsbaci::services::DrawingService:



Signals

- void `drawCommandReceived` (const `nsbaci::types::DrawCommand` &command)
Emitted when a drawing command should be executed.
- void `clearRequested` (const `nsbaci::types::Color` &backgroundColor)
Emitted when the canvas should be cleared.
- void `drawRequested` (const `nsbaci::types::Drawable` &drawable)
Emitted when a shape should be drawn.
- void `refreshRequested` ()
Emitted when the canvas should refresh.
- void `canvasSizeChanged` (const `nsbaci::types::Size` &size)
Emitted when canvas size changes.

Public Member Functions

- **DrawingService** (QObject *parent=nullptr)
- **DrawingService** (const DrawingService &)=delete
- DrawingService & **operator=** (const DrawingService &)=delete
- **DrawingService** (DrawingService &&)=delete
- DrawingService & **operator=** (DrawingService &&)=delete
- void **setColor** (uint8_t r, uint8_t g, uint8_t b)
Set the current drawing color using RGB values.
- void **setColor** (uint8_t r, uint8_t g, uint8_t b, uint8_t a)
Set the current drawing color using RGBA values.
- void **setColor** (const nsbaci::types::Color &color)
Set the current drawing color using a Color struct.
- nsbaci::types::Color **getCurrentColor** () const
Get the current drawing color.
- void **setPosition** (int32_t x, int32_t y)
Set the current drawing position.
- void **setPosition** (const nsbaci::types::Point &point)
Set the current drawing position using a Point.
- nsbaci::types::Point **getCurrentPosition** () const
Get the current drawing position.
- void **clear** ()
Clear the canvas with optional background color.
- void **clear** (const nsbaci::types::Color &color)
Clear the canvas with a specific color.
- void **fill** ()
Fill the entire canvas with the current color.
- void **refresh** ()
Request a canvas refresh/redraw.
- void **setLineWidth** (int32_t width)
Set the line thickness for subsequent drawings.
- int32_t **getLineWidth** () const
Get the current line width.
- void **drawCircle** (int32_t centerX, int32_t centerY, int32_t radius, bool filled=false)
Draw a circle at the specified position.
- void **drawRectangle** (int32_t x, int32_t y, int32_t width, int32_t height, bool filled=false)
Draw a rectangle at the specified position.
- void **drawTriangle** (int32_t x1, int32_t y1, int32_t x2, int32_t y2, int32_t x3, int32_t y3, bool filled=false)
Draw a triangle with three vertices.
- void **drawLine** (int32_t x1, int32_t y1, int32_t x2, int32_t y2)
Draw a line between two points.
- void **drawEllipse** (int32_t centerX, int32_t centerY, int32_t radiusX, int32_t radiusY, bool filled=false)
Draw an ellipse at the specified position.
- void **drawPixel** (int32_t x, int32_t y)
Draw a single pixel at the specified position.
- void **drawText** (int32_t x, int32_t y, const std::string &text, int32_t fontSize=12)
Draw text at the specified position.
- void **drawShape** (const nsbaci::types::Shape &shape)
Draw a generic shape.
- nsbaci::types::Size **getCanvasSize** () const
Get the canvas size.

- void [setCanvasSize](#) (int32_t width, int32_t height)
Set the canvas size.
- void **reset** ()
Reset the drawing service to initial state.
- void [processCommand](#) (const [nsbaci::types::DrawCommand](#) &command)
Process a drawing command from the runtime.

8.16.1 Detailed Description

Adapter service for graphical output backends.

The [DrawingService](#) manages drawing state and emits signals when drawing operations are requested by the runtime. It follows an SDL-like approach where you set the render color and then draw shapes.

Usage pattern:

1. [setColor](#)(r, g, b) - Set the current drawing color
2. [drawCircle](#)/[drawRect](#)/etc - Draw shapes with current color
3. [clear](#)() - Clear the canvas

The service emits signals that can be connected to any drawing backend (Qt widget, OpenGL, etc.)

8.16.2 Member Function Documentation

8.16.2.1 canvasSizeChanged

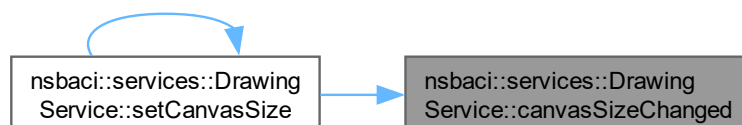
```
void nsbaci::services::DrawingService::canvasSizeChanged (
    const nsbaci::types::Size & size) [signal]
```

Emitted when canvas size changes.

Parameters

<i>size</i>	The new canvas size.
-------------	----------------------

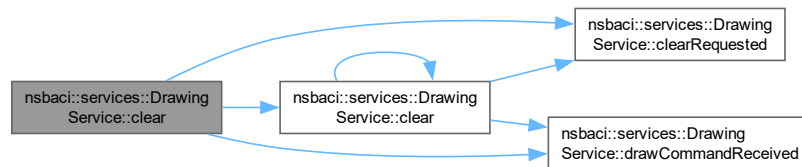
Here is the caller graph for this function:



Parameters

<i>color</i>	The background color.
--------------	-----------------------

Here is the call graph for this function:

**8.16.2.3 clearRequested**

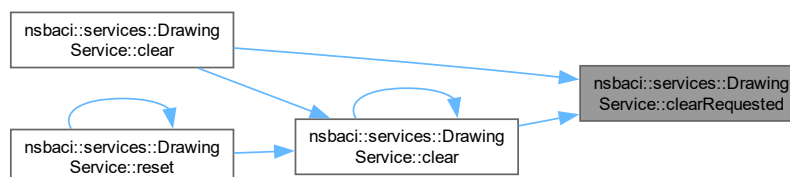
```
void nsbaci::services::DrawingService::clearRequested (
    const nsbaci::types::Color & backgroundColor) [signal]
```

Emitted when the canvas should be cleared.

Parameters

<i>backgroundColor</i>	The color to clear with.
------------------------	--------------------------

Here is the caller graph for this function:

**8.16.2.4 drawCircle()**

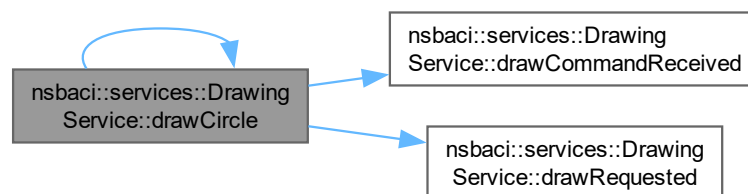
```
void nsbaci::services::DrawingService::drawCircle (
    int32_t centerX,
    int32_t centerY,
    int32_t radius,
    bool filled = false)
```

Draw a circle at the specified position.

Parameters

<i>centerX</i>	Center X coordinate
<i>centerY</i>	Center Y coordinate
<i>radius</i>	Circle radius
<i>filled</i>	Whether to fill the circle

Here is the call graph for this function:



Here is the caller graph for this function:

**8.16.2.5 drawCommandReceived**

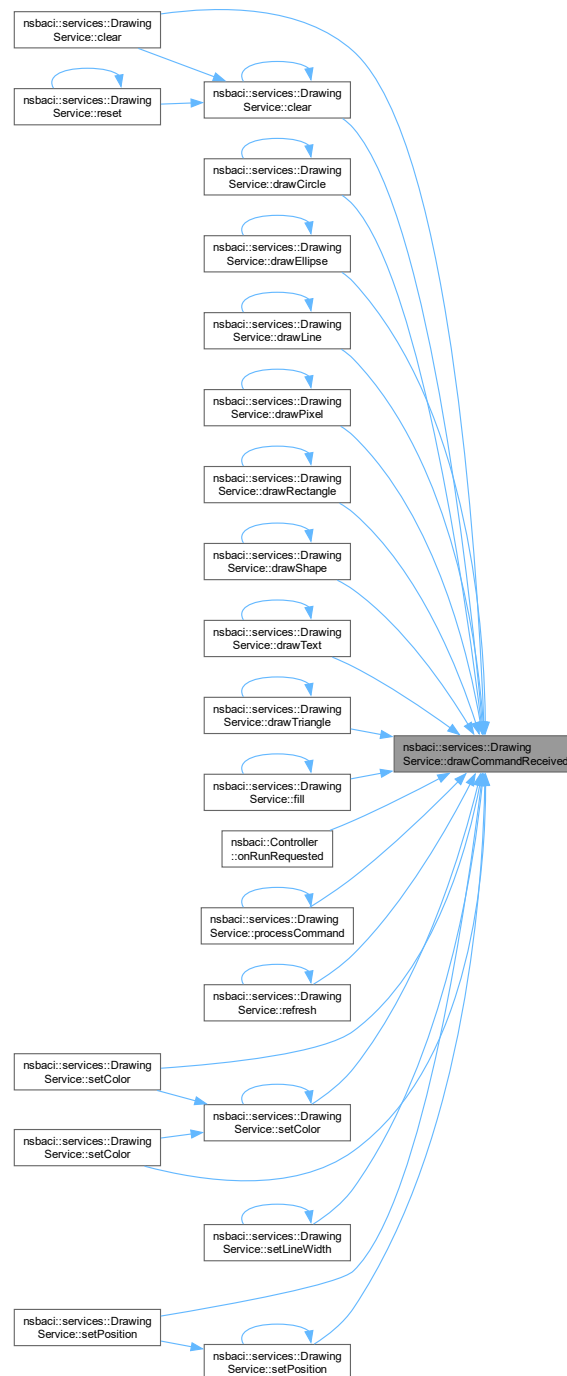
```
void nsbaci::services::DrawingService::drawCommandReceived (
    const nsbaci::types::DrawCommand & command) [signal]
```

Emitted when a drawing command should be executed.

Parameters

<i>command</i>	The drawing command to execute.
----------------	---------------------------------

Here is the caller graph for this function:



8.16.2.6 drawEllipse()

```

void nsbaci::services::DrawingService::drawEllipse (
    int32_t centerX,
    int32_t centerY,
    int32_t radiusX,

```



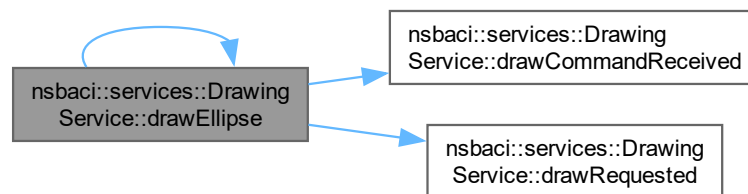
```
int32_t radiusY,
bool filled = false)
```

Draw an ellipse at the specified position.

Parameters

<i>centerX</i>	Center X coordinate
<i>centerY</i>	Center Y coordinate
<i>radiusX</i>	Horizontal radius
<i>radiusY</i>	Vertical radius
<i>filled</i>	Whether to fill the ellipse

Here is the call graph for this function:



Here is the caller graph for this function:



8.16.2.7 drawLine()

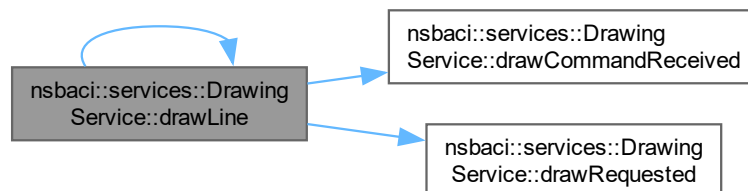
```
void nsbaci::services::DrawingService::drawLine (
    int32_t x1,
    int32_t y1,
    int32_t x2,
    int32_t y2)
```

Draw a line between two points.

Parameters

<i>x1,y1</i>	Start point
<i>x2,y2</i>	End point

Here is the call graph for this function:



Here is the caller graph for this function:

**8.16.2.8 drawPixel()**

```

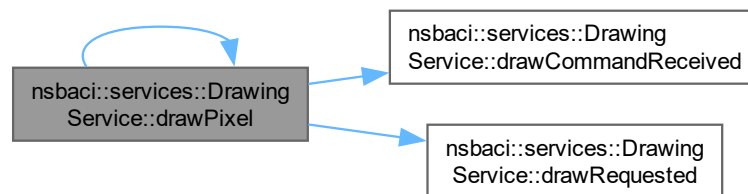
void nsbaci::services::DrawingService::drawPixel (
    int32_t x,
    int32_t y)
  
```

Draw a single pixel at the specified position.

Parameters

<i>x</i>	X coordinate
<i>y</i>	Y coordinate

Here is the call graph for this function:



Here is the caller graph for this function:



8.16.2.9 drawRectangle()

```

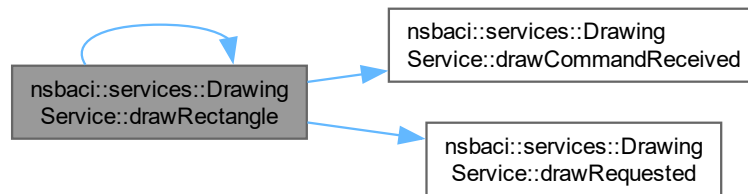
void nsbaci::services::DrawingService::drawRectangle (
    int32_t x,
    int32_t y,
    int32_t width,
    int32_t height,
    bool filled = false)
  
```

Draw a rectangle at the specified position.

Parameters

<i>x</i>	Top-left X coordinate
<i>y</i>	Top-left Y coordinate
<i>width</i>	Rectangle width
<i>height</i>	Rectangle height
<i>filled</i>	Whether to fill the rectangle

Here is the call graph for this function:



Here is the caller graph for this function:



8.16.2.10 drawRequested

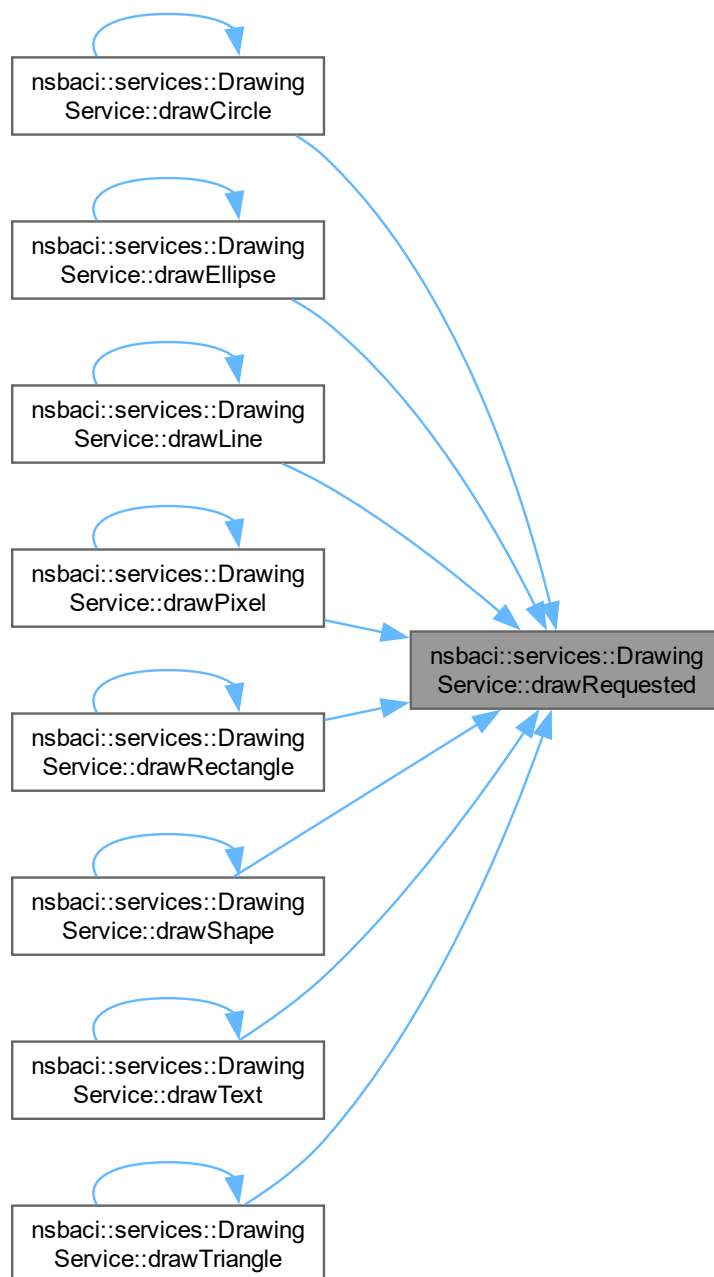
```
void nsbaci::services::DrawingService::drawRequested (
    const nsbaci::types::Drawable & drawable) [signal]
```

Emitted when a shape should be drawn.

Parameters

<i>drawable</i>	The drawable object containing shape and color.
-----------------	---

Here is the caller graph for this function:



8.16.2.11 drawShape()

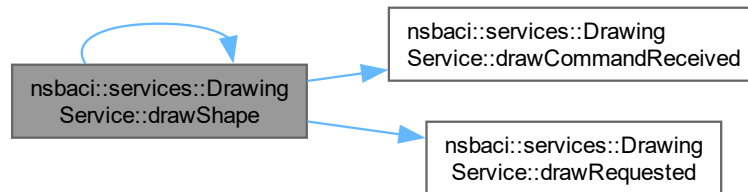
```
void nsbaci::services::DrawingService::drawShape (
    const nsbaci::types::Shape & shape)
```

Draw a generic shape.

Parameters

<i>shape</i>	The shape to draw.
--------------	--------------------

Here is the call graph for this function:



Here is the caller graph for this function:

**8.16.2.12 drawText()**

```

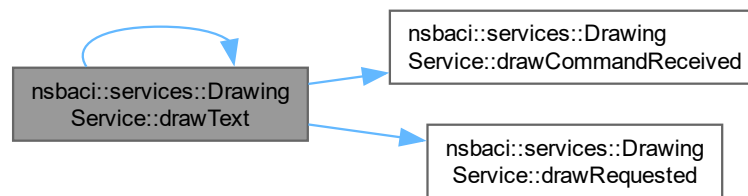
void nsbaci::services::DrawingService::drawText (
    int32_t x,
    int32_t y,
    const std::string & text,
    int32_t fontSize = 12)
  
```

Draw text at the specified position.

Parameters

<i>x</i>	X coordinate
<i>y</i>	Y coordinate
<i>text</i>	The text to draw
<i>fontSize</i>	Font size in points

Here is the call graph for this function:



Here is the caller graph for this function:



8.16.2.13 drawTriangle()

```

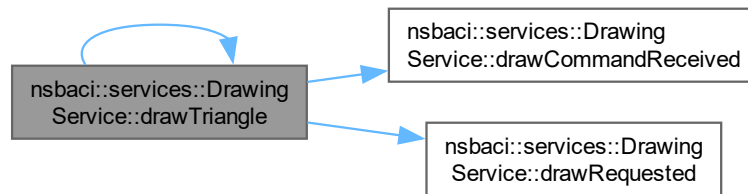
void nsbaci::services::DrawingService::drawTriangle (
    int32_t x1,
    int32_t y1,
    int32_t x2,
    int32_t y2,
    int32_t x3,
    int32_t y3,
    bool filled = false)
  
```

Draw a triangle with three vertices.

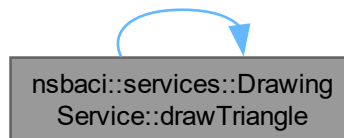
Parameters

<i>x1,y1</i>	First vertex
<i>x2,y2</i>	Second vertex
<i>x3,y3</i>	Third vertex
<i>filled</i>	Whether to fill the triangle

Here is the call graph for this function:



Here is the caller graph for this function:



8.16.2.14 `getCanvasSize()`

```
nsbaci::types::Size nsbaci::services::DrawingService::getCanvasSize () const [inline]
```

Get the canvas size.

Returns

The canvas size.

8.16.2.15 `getCurrentColor()`

```
nsbaci::types::Color nsbaci::services::DrawingService::getCurrentColor () const [inline]
```

Get the current drawing color.

Returns

The current color.

8.16.2.16 getCurrentPosition()

```
nsbaci::types::Point nsbaci::services::DrawingService::getCurrentPosition () const [inline]
```

Get the current drawing position.

Returns

The current position.

8.16.2.17 getLineWidth()

```
int32_t nsbaci::services::DrawingService::getLineWidth () const [inline]
```

Get the current line width.

Returns

The current line width.

8.16.2.18 processCommand()

```
void nsbaci::services::DrawingService::processCommand (
    const nsbaci::types::DrawCommand & command)
```

Process a drawing command from the runtime.

This method is intended to be called from the runtime's drawing callback. It processes the command and emits the appropriate signal.

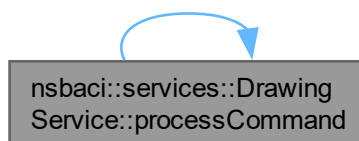
Parameters

<i>command</i>	The drawing command to process.
----------------	---------------------------------

Here is the call graph for this function:



Here is the caller graph for this function:



8.16.2.19 setCanvasSize()

```
void nsbaci::services::DrawingService::setCanvasSize (
    int32_t width,
    int32_t height)
```

Set the canvas size.

Parameters

<i>width</i>	Canvas width
<i>height</i>	Canvas height

Here is the call graph for this function:



Here is the caller graph for this function:



8.16.2.20 setColor() [1/3]

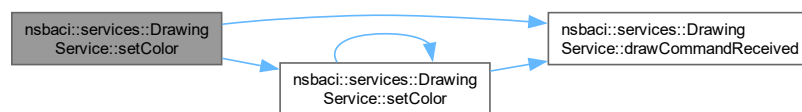
```
void nsbaci::services::DrawingService::setColor (
    const nsbaci::types::Color & color)
```

Set the current drawing color using a [Color](#) struct.

Parameters

<i>color</i>	The color to set.
--------------	-------------------

Here is the call graph for this function:



8.16.2.21 setColor() [2/3]

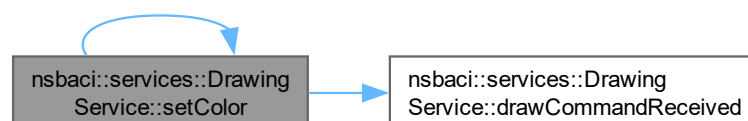
```
void nsbaci::services::DrawingService::setColor (
    uint8_t r,
    uint8_t g,
    uint8_t b)
```

Set the current drawing color using RGB values.

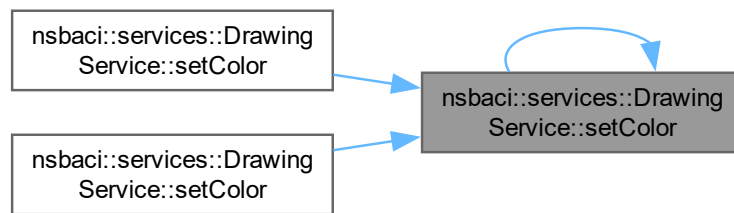
Parameters

<i>r</i>	Red component (0-255)
<i>g</i>	Green component (0-255)
<i>b</i>	Blue component (0-255)

Here is the call graph for this function:



Here is the caller graph for this function:



8.16.2.22 setColor() [3/3]

```

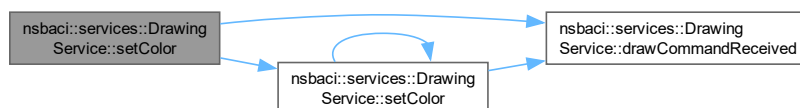
void nsbaci::services::DrawingService::setColor (
    uint8_t r,
    uint8_t g,
    uint8_t b,
    uint8_t a)
  
```

Set the current drawing color using RGBA values.

Parameters

<i>r</i>	Red component (0-255)
<i>g</i>	Green component (0-255)
<i>b</i>	Blue component (0-255)
<i>a</i>	Alpha component (0-255)

Here is the call graph for this function:



8.16.2.23 setLineWidth()

```

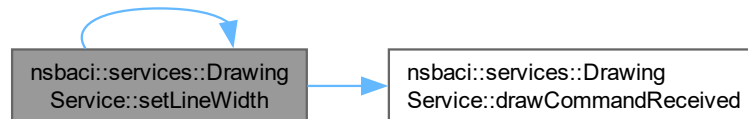
void nsbaci::services::DrawingService::setLineWidth (
    int32_t width)
  
```

Set the line thickness for subsequent drawings.

Parameters

<i>width</i>	Line width in pixels.
--------------	-----------------------

Here is the call graph for this function:



Here is the caller graph for this function:

**8.16.2.24 setPosition() [1/2]**

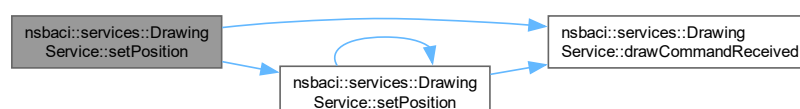
```
void nsbaci::services::DrawingService::setPosition (
    const nsbaci::types::Point & point)
```

Set the current drawing position using a [Point](#).

Parameters

<i>point</i>	The position to set.
--------------	----------------------

Here is the call graph for this function:



8.16.2.25 setPosition() [2/2]

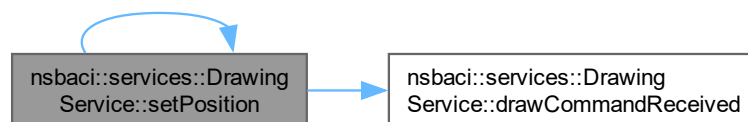
```
void nsbaci::services::DrawingService::setPosition (
    int32_t x,
    int32_t y)
```

Set the current drawing position.

Parameters

<i>x</i>	X coordinate
<i>y</i>	Y coordinate

Here is the call graph for this function:



Here is the caller graph for this function:



The documentation for this class was generated from the following files:

- [drawingService.h](#)
- [drawingService.cpp](#)

8.17 nsbaci::factories::DrawingServiceFactory Class Reference

Factory for creating DrawingService instances.

```
#include <drawingServiceFactory.h>
```

Static Public Member Functions

- static std::unique_ptr< [nsbaci::services::DrawingService](#) > **createService** ([DefaultDrawingBackend](#))

8.17.1 Detailed Description

Factory for creating DrawingService instances.

The documentation for this class was generated from the following files:

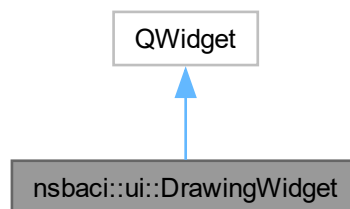
- [drawingServiceFactory.h](#)
- [drawingServiceFactory.cpp](#)

8.18 nsbaci::ui::DrawingWidget Class Reference

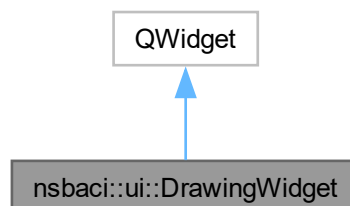
Qt widget that provides a drawing canvas.

```
#include <drawingWidget.h>
```

Inheritance diagram for nsbaci::ui::DrawingWidget:



Collaboration diagram for nsbaci::ui::DrawingWidget:



Public Slots

- void **onDrawCommand** (const [nsbaci::types::DrawCommand](#) &command)
Handle a drawing command.
- void **onDrawRequested** (const [nsbaci::types::Drawable](#) &drawable)
Handle a drawable object.
- void **onClearRequested** (const [nsbaci::types::Color](#) &color)
Clear the canvas with a specific color.
- void **onRefreshRequested** ()
Refresh the canvas display.
- void **onCanvasSizeChanged** (const [nsbaci::types::Size](#) &size)
Handle canvas size change.
- void **setColor** (uint8_t r, uint8_t g, uint8_t b, uint8_t a=255)
Set the current drawing color.
- void **clear** ()
Clear the canvas with the current background color.
- void **drawCircle** (int32_t centerX, int32_t centerY, int32_t radius, bool filled=false)
Draw a circle.
- void **drawRectangle** (int32_t x, int32_t y, int32_t width, int32_t height, bool filled=false)
Draw a rectangle.
- void **drawTriangle** (int32_t x1, int32_t y1, int32_t x2, int32_t y2, int32_t x3, int32_t y3, bool filled=false)
Draw a triangle.
- void **drawLine** (int32_t x1, int32_t y1, int32_t x2, int32_t y2)
Draw a line.
- void **drawEllipse** (int32_t centerX, int32_t centerY, int32_t radiusX, int32_t radiusY, bool filled=false)
Draw an ellipse.
- void **drawPixel** (int32_t x, int32_t y)
Draw a pixel.
- void **drawText** (int32_t x, int32_t y, const QString &text, int32_t fontSize=12)
Draw text.
- void **setLineWidth** (int32_t width)
Set the line width.
- void **reset** ()
Reset the widget to initial state.

Public Member Functions

- [DrawingWidget](#) (QWidget *parent=nullptr)
Constructs the drawing widget.
- [~DrawingWidget](#) () override=default
Destructor.
- [nsbaci::types::Size](#) **getCanvasSize** () const
Get the current canvas size.
- QSize **minimumSizeHint** () const override
Get the minimum size hint for the widget.
- QSize **sizeHint** () const override
Get the size hint for the widget.

Protected Member Functions

- void [paintEvent](#) (QPaintEvent *event) override
Paint event handler.
- void [resizeEvent](#) (QResizeEvent *event) override
Resize event handler.

8.18.1 Detailed Description

Qt widget that provides a drawing canvas.

The [DrawingWidget](#) maintains a pixel buffer (QPixmap) where all drawing operations are performed. It supports various shapes, colors, and provides a persistent canvas that preserves drawings between repaints.

8.18.2 Constructor & Destructor Documentation

8.18.2.1 DrawingWidget()

```
nsbaci::ui::DrawingWidget::DrawingWidget (
    QWidget * parent = nullptr) [explicit]
```

Constructs the drawing widget.

Parameters

<i>parent</i>	Optional parent widget.
---------------	-------------------------

Here is the call graph for this function:



8.18.3 Member Function Documentation

8.18.3.1 getCanvasSize()

```
nsbaci::types::Size nsbaci::ui::DrawingWidget::getCanvasSize () const
```

Get the current canvas size.

Returns

The canvas size.

8.18.3.2 `minimumSizeHint()`

```
QSize nsbaci::ui::DrawingWidget::minimumSizeHint () const [override]
```

Get the minimum size hint for the widget.

Returns

The minimum size.

8.18.3.3 `onCanvasSizeChanged`

```
void nsbaci::ui::DrawingWidget::onCanvasSizeChanged (  
    const nsbaci::types::Size & size) [slot]
```

Handle canvas size change.

Parameters

<i>size</i>	The new canvas size.
-------------	----------------------

Here is the call graph for this function:



8.18.3.4 `onClearRequested`

```
void nsbaci::ui::DrawingWidget::onClearRequested (  
    const nsbaci::types::Color & color) [slot]
```

Clear the canvas with a specific color.

Parameters

<i>color</i>	The background color.
--------------	-----------------------

Here is the call graph for this function:



8.18.3.5 onDrawCommand

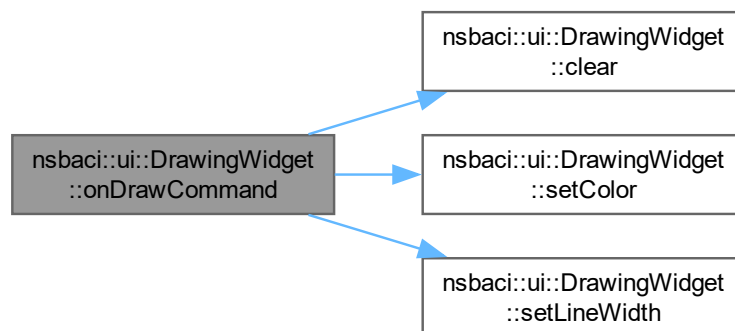
```
void nsbaci::ui::DrawingWidget::onDrawCommand (
    const nsbaci::types::DrawCommand & command) [slot]
```

Handle a drawing command.

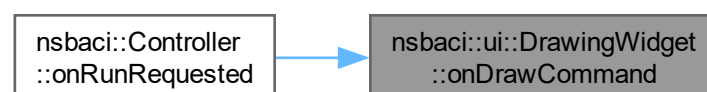
Parameters

<i>command</i>	The command to execute.
----------------	-------------------------

Here is the call graph for this function:



Here is the caller graph for this function:



8.18.3.6 onDrawRequested

```
void nsbaci::ui::DrawingWidget::onDrawRequested (
    const nsbaci::types::Drawable & drawable) [slot]
```

Handle a drawable object.

Parameters

<i>drawable</i>	The drawable to render.
-----------------	-------------------------

8.18.3.7 paintEvent()

```
void nsbaci::ui::DrawingWidget::paintEvent (  
    QPaintEvent * event) [override], [protected]
```

Paint event handler.

Parameters

<i>event</i>	The paint event.
--------------	------------------

8.18.3.8 resizeEvent()

```
void nsbaci::ui::DrawingWidget::resizeEvent (  
    QResizeEvent * event) [override], [protected]
```

Resize event handler.

Parameters

<i>event</i>	The resize event.
--------------	-------------------

8.18.3.9 sizeHint()

```
QSize nsbaci::ui::DrawingWidget::sizeHint () const [override]
```

Get the size hint for the widget.

Returns

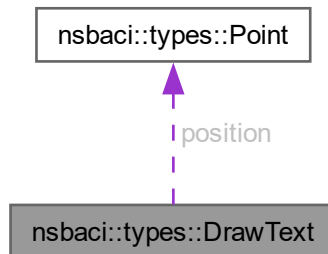
The preferred size.

The documentation for this class was generated from the following files:

- [drawingWidget.h](#)
- [drawingWidget.cpp](#)

8.19 nsbaci::types::DrawText Struct Reference

Collaboration diagram for nsbaci::types::DrawText:



Public Member Functions

- **DrawText** ([Point](#) p, std::string text, int32_t size=12)

Public Attributes

- [Point](#) **position**
- std::string **content**
- int32_t **fontSize** = 12

The documentation for this struct was generated from the following file:

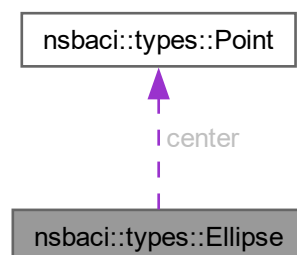
- [drawingTypes.h](#)

8.20 nsbaci::types::Ellipse Struct Reference

[Ellipse](#) shape with center and radii.

```
#include <drawingTypes.h>
```

Collaboration diagram for nsbaci::types::Ellipse:



Public Member Functions

- **Ellipse** ([Point](#) c, int32_t rx, int32_t ry, bool fill=false)

Public Attributes

- [Point](#) **center**
- int32_t **radiusX** = 0
- int32_t **radiusY** = 0
- bool **filled** = false

8.20.1 Detailed Description

[Ellipse](#) shape with center and radii.

The documentation for this struct was generated from the following file:

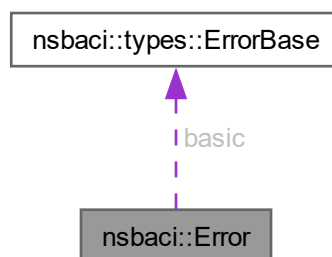
- [drawingTypes.h](#)

8.21 nsbaci::Error Class Reference

Represents an error with a message and optional code.

```
#include <error.h>
```

Collaboration diagram for nsbaci::Error:



Public Attributes

- [nsbaci::types::ErrorBase](#) **basic**
- [nsbaci::types::ErrorPayload](#) **payload**

8.21.1 Detailed Description

Represents an error with a message and optional code.

The documentation for this class was generated from the following file:

- [error.h](#)

8.22 nsbaci::types::ErrorBase Struct Reference

Base structure containing common error properties.

```
#include <errorTypes.h>
```

Public Attributes

- [ErrSeverity](#) **severity**
- ErrMessage **message**
- [ErrType](#) **type**

8.22.1 Detailed Description

Base structure containing common error properties.

The documentation for this struct was generated from the following file:

- [errorTypes.h](#)

8.23 nsbaci::ui::ErrorDialogFactory Class Reference

Factory for creating error dialogs from [UIError](#) objects.

```
#include <errorDialogFactory.h>
```

Public Types

- using [DialogInvoker](#) = std::function<QMessageBox::StandardButton()>
Callable type that shows a dialog when invoked.

Static Public Member Functions

- static [DialogInvoker](#) [getDialogFromUIError](#) (const [UIError](#) &error, QWidget *parent=nullptr)
Creates a dialog invoker from a [UIError](#).
- static std::vector< [DialogInvoker](#) > [getDialogsFromUIErrors](#) (const std::vector< [UIError](#) > &errors, QWidget *parent=nullptr)
Creates dialog invokers for multiple [UIErrors](#).
- static [DialogInvoker](#) [getSuccessDialog](#) (const QString &title, const QString &message, QWidget *parent=nullptr)
Creates a success message dialog invoker.
- static void [showErrors](#) (const std::vector< [UIError](#) > &errors, QWidget *parent=nullptr)
Shows all error dialogs sequentially.
- static QMessageBox::StandardButton [showError](#) (const [UIError](#) &error, QWidget *parent=nullptr)
Shows a single error dialog immediately.
- static void [showSuccess](#) (const QString &title, const QString &message, QWidget *parent=nullptr)
Shows a success message dialog immediately.

8.23.1 Detailed Description

Factory for creating error dialogs from [UIError](#) objects.

Provides static methods to convert [UIError](#) objects into QMessageBox dialogs. Supports two modes:

- **Deferred:** Returns a [DialogInvoker](#) callable for later invocation
- **Immediate:** Shows the dialog right away via convenience methods

See also

[DialogInvoker](#)

8.23.2 Member Typedef Documentation

8.23.2.1 DialogInvoker

```
using nsbaci::ui::ErrorDialogFactory::DialogInvoker = std::function<QMessageBox::StandardButton()>
```

Callable type that shows a dialog when invoked.

A [DialogInvoker](#) is a packaged dialog that can be invoked at any time. When called, it displays the dialog (blocking) and returns which button the user clicked. Think of it as a "suspended dialog" or "dialog builder" that you can trigger when ready.

Returns

The button that was clicked (QMessageBox::StandardButton).

8.23.3 Member Function Documentation

Generated by Doxygen

8.23.3.1 getDialogFromUIError()

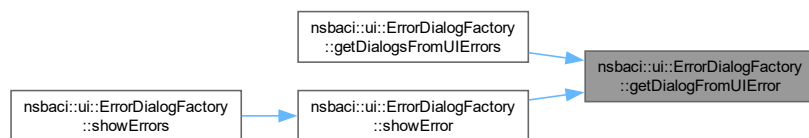
Parameters

<i>error</i>	The UIError to display.
<i>parent</i>	Parent widget for the dialog.

Returns

A callable that shows the dialog when invoked.

Here is the caller graph for this function:

**8.23.3.2 getDialogsFromUIErrors()**

```

std::vector< ErrorDialogFactory::DialogInvoker > nsbaci::ui::ErrorDialogFactory::getDialogsFromUIErrors (
    const std::vector< UIError > & errors,
    QWidget * parent = nullptr) [static]
  
```

Creates dialog invokers for multiple UIErrors.

Parameters

<i>errors</i>	Vector of UIErrors to convert.
<i>parent</i>	Parent widget for all dialogs.

Returns

Vector of callables, one per error.

Here is the call graph for this function:



8.23.3.3 getSuccessDialog()

```

ErrorDialogFactory::DialogInvoker nsbaci::ui::ErrorDialogFactory::getSuccessDialog (
    const QString & title,
    const QString & message,
    QWidget * parent = nullptr) [static]

```

Creates a success message dialog invoker.

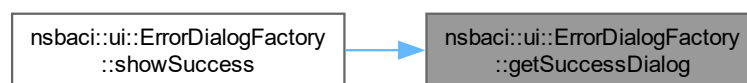
Parameters

<i>title</i>	Dialog title.
<i>message</i>	Success message body.
<i>parent</i>	Parent widget for the dialog.

Returns

A callable that shows the success dialog when invoked.

Here is the caller graph for this function:



8.23.3.4 showError()

```

QMessageBox::StandardButton nsbaci::ui::ErrorDialogFactory::showError (
    const UIError & error,
    QWidget * parent = nullptr) [static]

```

Shows a single error dialog immediately.

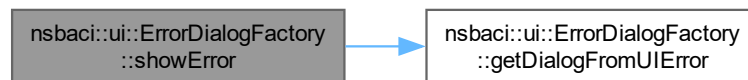
Parameters

<i>error</i>	The UIError to display.
<i>parent</i>	Parent widget for the dialog.

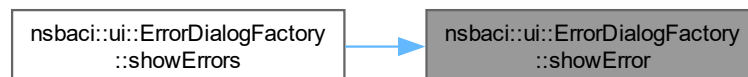
Returns

The button that was clicked.

Here is the call graph for this function:



Here is the caller graph for this function:

**8.23.3.5 showErrors()**

```
void nsbaci::ui::ErrorDialogFactory::showErrors (
    const std::vector< UIError > & errors,
    QWidget * parent = nullptr) [static]
```

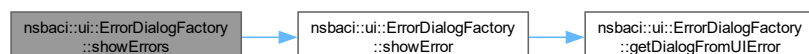
Shows all error dialogs sequentially.

Convenience method that creates and immediately shows dialogs for all provided UIErrors.

Parameters

<i>errors</i>	Vector of UIErrors to display.
<i>parent</i>	Parent widget for all dialogs.

Here is the call graph for this function:



8.23.3.6 showSuccess()

```
void nsbaci::ui::ErrorDialogFactory::showSuccess (
    const QString & title,
    const QString & message,
    QWidget * parent = nullptr) [static]
```

Shows a success message dialog immediately.

Parameters

<i>title</i>	Dialog title.
<i>message</i>	Success message body.
<i>parent</i>	Parent widget for the dialog.

Here is the call graph for this function:



The documentation for this class was generated from the following files:

- [errorDialogFactory.h](#)
- [errorDialogFactory.cpp](#)

8.24 nsbaci::ui::ExampleEntry Struct Reference

Data for a single example entry in the User Guide.

```
#include <userGuideDialog.h>
```

Public Attributes

- QString **title**
Display title (e.g. "Variables & Operators").
- QString **category**
Category grouping (e.g. "Basics").
- QString **resourcePath**
Qt resource path (e.g. ":/examples/01_basics.nsb").
- QString **description**
Short description shown above the code.

8.24.1 Detailed Description

Data for a single example entry in the User Guide.

The documentation for this struct was generated from the following file:

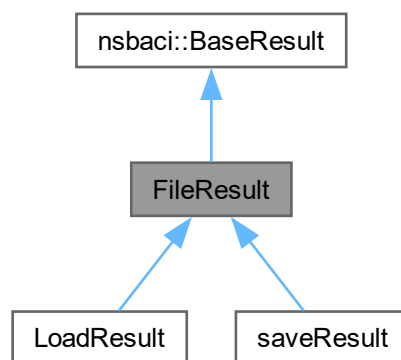
- [userGuideDialog.h](#)

8.25 FileResult Struct Reference

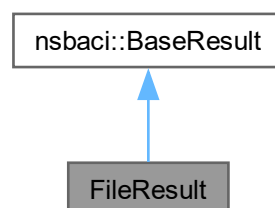
Base result type for file operations.

```
#include <fileService.h>
```

Inheritance diagram for FileResult:



Collaboration diagram for FileResult:



Public Member Functions

- **FileResult** ()
Default constructor creates a successful result.
- **FileResult** (std::vector< [nsbaci::Error](#) > errs)
Constructs a result from a vector of errors.
- **FileResult** ([nsbaci::Error](#) error)
Constructs a failed result from a single error.
- **FileResult** ([FileResult](#) &&) noexcept=default
- **FileResult** & **operator=** ([FileResult](#) &&) noexcept=default
- **FileResult** (const [FileResult](#) &)=default
- **FileResult** & **operator=** (const [FileResult](#) &)=default

Public Member Functions inherited from [nsbaci::BaseResult](#)

- **BaseResult** ()
Default constructor creates a successful result.
- **BaseResult** (std::vector< [nsbaci::Error](#) > errs)
Constructs a result from a vector of errors.
- **BaseResult** ([nsbaci::Error](#) error)
Constructs a failed result from a single error.
- **BaseResult** ([BaseResult](#) &&) noexcept=default
- **BaseResult** & **operator=** ([BaseResult](#) &&) noexcept=default
- **BaseResult** (const [BaseResult](#) &)=default
- **BaseResult** & **operator=** (const [BaseResult](#) &)=default

Additional Inherited Members

Public Attributes inherited from [nsbaci::BaseResult](#)

- bool **ok**
True if the operation succeeded.
- std::vector< [nsbaci::Error](#) > **errors**
Errors encountered (empty if ok is true).

8.25.1 Detailed Description

Base result type for file operations.

Extends [BaseResult](#) with file-specific semantics. All file operation results inherit from this type.

Invariant

If ok is false, errors vector contains at least one error.

8.25.2 Constructor & Destructor Documentation

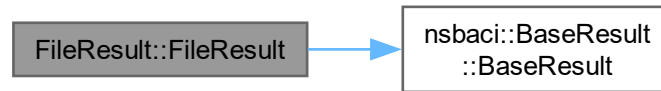
8.25.2.1 [FileResult](#)() [1/2]

```
FileResult::FileResult (
    std::vector< nsbaci::Error > errs) [inline], [explicit]
```

Parameters

<i>errs</i>	Vector of errors encountered during the operation.
-------------	--

Here is the call graph for this function:

**8.25.2.2 FileResult() [2/2]**

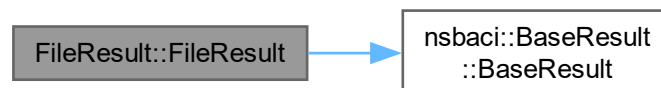
```
FileResult::FileResult (
    nsbaci::Error error) [inline], [explicit]
```

Constructs a failed result from a single error.

Parameters

<i>error</i>	The error that caused the operation to fail.
--------------	--

Here is the call graph for this function:



The documentation for this struct was generated from the following file:

- [fileService.h](#)

8.26 nsbaci::services::FileService Class Reference

Service for handling file system operations on BACI source files.

```
#include <fileService.h>
```

Public Member Functions

- [saveResult save](#) ([nsbaci::types::Text](#) contents, [nsbaci::types::File](#) file)
Saves source code content to a file.
- [LoadResult load](#) ([nsbaci::types::File](#) file)
Loads source code content from a file.
- [FileService](#) ()=default
Default constructor.
- [FileService](#) (const [FileService](#) &)=delete
- [FileService](#) & [operator=](#) (const [FileService](#) &)=delete
- [FileService](#) ([FileService](#) &&)=default
- [FileService](#) & [operator=](#) ([FileService](#) &&)=default

8.26.1 Detailed Description

Service for handling file system operations on BACI source files.

[FileService](#) provides methods for saving and loading BACI source code files. It enforces the .nsb file extension and provides detailed error reporting for various failure scenarios including:

- Empty or invalid file paths
- Invalid file extensions (must be .nsb)
- Non-existent directories or files
- Permission and I/O errors

The service is designed to be move-only (non-copyable) to ensure single ownership and prevent accidental resource duplication.

Usage example:

```
FileService fs;

// Save a file
auto saveRes = fs.save(sourceCode, "program.nsb");
if (!saveRes.ok) {
    // Handle save error
}

// Load a file
auto loadRes = fs.load("program.nsb");
if (loadRes.ok) {
    std::string code = loadRes.contents;
}
```

8.26.2 Member Function Documentation

8.26.2.1 load()

```
LoadResult nsbaci::services::FileService::load (
    nsbaci::types::File file)
```

Loads source code content from a file.

Validates the file path, extension, and existence before reading. Returns the complete file contents on success.

Parameters

<i>file</i>	The source file path to load (must have .nsb extension).
-------------	--

Returns

[LoadResult](#) containing file contents on success, or error details on failure.

8.26.2.2 save()

```
saveResult nsbaci::services::FileService::save (
    nsbaci::types::Text contents,
    nsbaci::types::File file)
```

Saves source code content to a file.

Validates the file path and extension before attempting to write. Creates or overwrites the target file with the provided contents.

Parameters

<i>contents</i>	The source code text to save.
<i>file</i>	The target file path (must have .nsb extension).

Returns

[saveResult](#) indicating success or containing error details.

Note

The parent directory must exist; this method does not create directories.

The documentation for this class was generated from the following files:

- [fileService.h](#)
- [fileService.cpp](#)

8.27 nsbaci::factories::FileServiceFactory Class Reference

Factory for creating FileService instances.

```
#include <fileServiceFactory.h>
```

Static Public Member Functions

- static [nsbaci::services::FileService](#) **createService** ([DefaultFileSystem](#))

8.27.1 Detailed Description

Factory for creating FileService instances.

The documentation for this class was generated from the following files:

- [fileServiceFactory.h](#)
- [fileServiceFactory.cpp](#)

8.28 nsbaci::compiler::Instruction Struct Reference

Represents a single instruction in the virtual machine.

```
#include <instruction.h>
```

Public Member Functions

- **Instruction** ([Opcode](#) op)
- **Instruction** ([Opcode](#) op, int32_t op1)
- **Instruction** ([Opcode](#) op, uint32_t op1)
- **Instruction** ([Opcode](#) op, std::string op1)
- **Instruction** ([Opcode](#) op, int32_t op1, int32_t op2)
- **Instruction** ([Opcode](#) op, uint32_t op1, int32_t op2)

Public Attributes

- [Opcode](#) opcode
- [Operand](#) operand1
- [Operand](#) operand2

8.28.1 Detailed Description

Represents a single instruction in the virtual machine.

The documentation for this struct was generated from the following file:

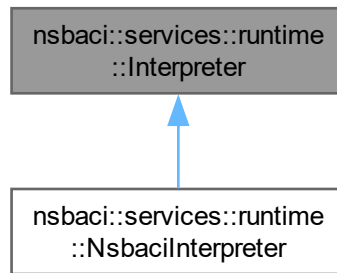
- [instruction.h](#)

8.29 nsbaci::services::runtime::Interpreter Class Reference

Executes instructions for threads within a program context.

```
#include <interpreter.h>
```

Inheritance diagram for nsbaci::services::runtime::Interpreter:



Public Member Functions

- virtual [InterpreterResult](#) [executeInstruction](#) ([Thread](#) &t, [Program](#) &program)=0
Executes the current instruction for the given thread with the program context.
- virtual void [provideInput](#) (const std::string &input)=0
Provide input to a thread waiting for input.
- virtual bool [isWaitingForInput](#) () const =0
Check if interpreter is waiting for input.
- virtual void [setOutputCallback](#) ([OutputCallback](#) callback)=0
Set the output callback for print operations.
- virtual void [setDrawingCallback](#) ([DrawingCallback](#) callback)=0
Set the drawing callback for drawing operations.
- virtual void [reset](#) ()=0
Reset interpreter state for a new run.

8.29.1 Detailed Description

Executes instructions for threads within a program context.

The [Interpreter](#) is responsible for fetching and executing instructions based on the current thread's program counter.

8.29.2 Member Function Documentation

8.29.2.1 executeInstruction()

```
virtual InterpreterResult nsbaci::services::runtime::Interpreter::executeInstruction (
    Thread & t,
    Program & program) [pure virtual]
```

Executes the current instruction for the given thread with the program context

Parameters

<i>t</i>	The thread whose instruction should be executed.
<i>program</i>	The program context in which to execute the instruction

Implemented in [nsbaci::services::runtime::NsbaciInterpreter](#).

8.29.2.2 isWaitingForInput()

```
virtual bool nsbaci::services::runtime::Interpreter::isWaitingForInput () const [pure virtual]
```

Check if interpreter is waiting for input.

Returns

True if waiting for input.

Implemented in [nsbaci::services::runtime::NsbaciInterpreter](#).

8.29.2.3 provideInput()

```
virtual void nsbaci::services::runtime::Interpreter::provideInput (
    const std::string & input) [pure virtual]
```

Provide input to a thread waiting for input.

Parameters

<i>input</i>	The input string.
--------------	-------------------

Implemented in [nsbaci::services::runtime::NsbaciInterpreter](#).

8.29.2.4 reset()

```
virtual void nsbaci::services::runtime::Interpreter::reset () [pure virtual]
```

Reset interpreter state for a new run.

Clears any pending input and waiting state.

Implemented in [nsbaci::services::runtime::NsbaciInterpreter](#).

8.29.2.5 setDrawingCallback()

```
virtual void nsbaci::services::runtime::Interpreter::setDrawingCallback (
    DrawingCallback callback) [pure virtual]
```

Set the drawing callback for drawing operations.

Parameters

<i>callback</i>	Function to call when drawing is requested.
-----------------	---

Implemented in [nsbaci::services::runtime::NsbaciInterpreter](#).

8.29.2.6 setOutputCallback()

```
virtual void nsbaci::services::runtime::Interpreter::setOutputCallback (  
    OutputCallback callback) [pure virtual]
```

Set the output callback for print operations.

Parameters

<i>callback</i>	Function to call when output is produced.
-----------------	---

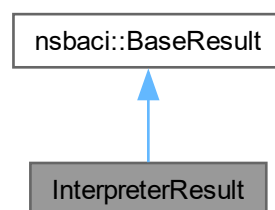
Implemented in [nsbaci::services::runtime::NsbaciInterpreter](#).

The documentation for this class was generated from the following file:

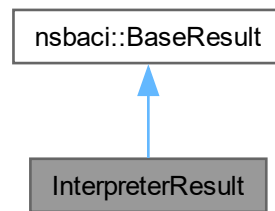
- [interpreter.h](#)

8.30 InterpreterResult Struct Reference

Inheritance diagram for InterpreterResult:



Collaboration diagram for InterpreterResult:



Public Member Functions

- **InterpreterResult** (std::vector< [nsbaci::Error](#) > errs)
- **InterpreterResult** ([nsbaci::Error](#) error)
- **InterpreterResult** ([InterpreterResult](#) &&) noexcept=default
- [InterpreterResult](#) & **operator=** ([InterpreterResult](#) &&) noexcept=default
- **InterpreterResult** (const [InterpreterResult](#) &)=default
- [InterpreterResult](#) & **operator=** (const [InterpreterResult](#) &)=default

Public Member Functions inherited from [nsbaci::BaseResult](#)

- **BaseResult** ()
Default constructor creates a successful result.
- [BaseResult](#) (std::vector< [nsbaci::Error](#) > errs)
Constructs a result from a vector of errors.
- [BaseResult](#) ([nsbaci::Error](#) error)
Constructs a failed result from a single error.
- **BaseResult** ([BaseResult](#) &&) noexcept=default
- [BaseResult](#) & **operator=** ([BaseResult](#) &&) noexcept=default
- **BaseResult** (const [BaseResult](#) &)=default
- [BaseResult](#) & **operator=** (const [BaseResult](#) &)=default

Public Attributes

- bool **needsInput** = false
Thread is waiting for input.
- std::string **inputPrompt**
Prompt to show for input.
- std::string **output**
Output produced by this instruction.
- bool **shouldBlock** = false
Thread should be blocked (semaphore wait).
- uint32_t **blockingSemaphore** = 0
Address of semaphore causing block.
- bool **shouldYield** = false

- *Thread should yield after instruction.*
- bool **cobeginStart** = false
Starting a cobegin block.
- bool **coendWait** = false
Waiting for cobegin threads to finish.
- int32_t **expectedThreadCount** = 0
Number of threads to wait for at coend.
- bool **createThread** = false
Should create a new thread.
- uint32_t **newThreadPC** = 0
PC for new thread (if createThread is true).
- bool **signalSemaphore** = false
Signal was called on a semaphore.
- uint32_t **signaledSemaphore** = 0
Address of semaphore that was signaled.

Public Attributes inherited from nsbaci::BaseResult

- bool **ok**
True if the operation succeeded.
- std::vector< nsbaci::Error > **errors**
Errors encountered (empty if ok is true).

The documentation for this struct was generated from the following file:

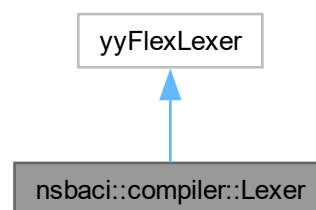
- [interpreter.h](#)

8.31 nsbaci::compiler::Lexer Class Reference

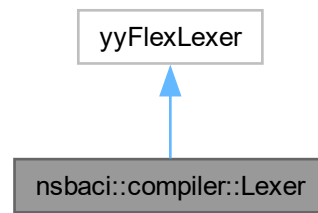
Flex-based lexer for BACI source code.

```
#include <lexer.h>
```

Inheritance diagram for nsbaci::compiler::Lexer:



Collaboration diagram for nsbaci::compiler::Lexer:



Public Member Functions

- **Lexer** (`std::istream *in=nullptr`)
- **int yylex** (`Parser::semantic_type *yylval, Parser::location_type *yylloc`)

8.31.1 Detailed Description

Flex-based lexer for BACI source code.

The documentation for this class was generated from the following file:

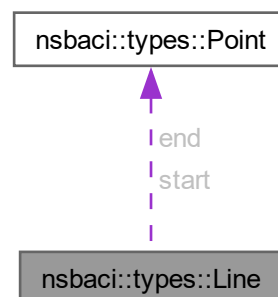
- [lexer.h](#)

8.32 nsbaci::types::Line Struct Reference

[Line](#) segment from start to end point.

```
#include <drawingTypes.h>
```

Collaboration diagram for nsbaci::types::Line:



Public Member Functions

- **Line** ([Point](#) s, [Point](#) e, int32_t t=1)

Public Attributes

- [Point](#) start
- [Point](#) end
- int32_t thickness = 1

8.32.1 Detailed Description

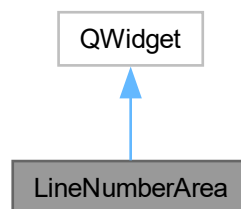
[Line](#) segment from start to end point.

The documentation for this struct was generated from the following file:

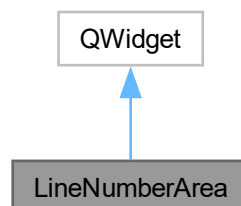
- [drawingTypes.h](#)

8.33 LineNumberArea Class Reference

Inheritance diagram for LineNumberArea:



Collaboration diagram for LineNumberArea:



Public Member Functions

- **LineNumberArea** ([CodeEditor](#) *editor)
- **QSize sizeHint** () const override

Protected Member Functions

- void **paintEvent** (QPaintEvent *event) override

The documentation for this class was generated from the following file:

- [codeeditor.h](#)

8.34 nsbaci::types::LoadError Struct Reference

[Error](#) payload for file load errors.

```
#include <errorTypes.h>
```

Public Attributes

- [File](#) **associatedFile**

8.34.1 Detailed Description

[Error](#) payload for file load errors.

The documentation for this struct was generated from the following file:

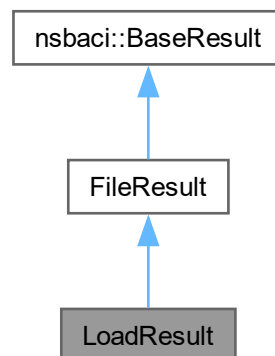
- [errorTypes.h](#)

8.35 LoadResult Struct Reference

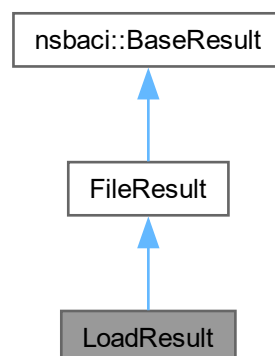
Result type for file load operations.

```
#include <fileService.h>
```

Inheritance diagram for LoadResult:



Collaboration diagram for LoadResult:



Public Member Functions

- **LoadResult ()**
Default constructor creates a successful but empty result.
- **LoadResult** ([nsbaci::types::Text](#) conts, [nsbaci::types::File](#) name)

Constructs a successful result with file contents.

- **LoadResult** (std::vector< [nsbaci::Error](#) > errs)

Constructs a result from a vector of errors.

- **LoadResult** ([nsbaci::Error](#) error)

Constructs a failed result from a single error.

- **LoadResult** ([LoadResult](#) &&) noexcept=default
- **LoadResult** & **operator=** ([LoadResult](#) &&) noexcept=default
- **LoadResult** (const [LoadResult](#) &)=default
- **LoadResult** & **operator=** (const [LoadResult](#) &)=default

Public Member Functions inherited from [FileResult](#)

- **FileResult** ()

Default constructor creates a successful result.

- **FileResult** (std::vector< [nsbaci::Error](#) > errs)

Constructs a result from a vector of errors.

- **FileResult** ([nsbaci::Error](#) error)

Constructs a failed result from a single error.

- **FileResult** ([FileResult](#) &&) noexcept=default
- **FileResult** & **operator=** ([FileResult](#) &&) noexcept=default
- **FileResult** (const [FileResult](#) &)=default
- **FileResult** & **operator=** (const [FileResult](#) &)=default

Public Member Functions inherited from [nsbaci::BaseResult](#)

- **BaseResult** ()

Default constructor creates a successful result.

- **BaseResult** (std::vector< [nsbaci::Error](#) > errs)

Constructs a result from a vector of errors.

- **BaseResult** ([nsbaci::Error](#) error)

Constructs a failed result from a single error.

- **BaseResult** ([BaseResult](#) &&) noexcept=default
- **BaseResult** & **operator=** ([BaseResult](#) &&) noexcept=default
- **BaseResult** (const [BaseResult](#) &)=default
- **BaseResult** & **operator=** (const [BaseResult](#) &)=default

Public Attributes

- [nsbaci::types::Text](#) **contents**

The loaded file contents.

- [nsbaci::types::File](#) **fileName**

The filename for display purposes.

Public Attributes inherited from [nsbaci::BaseResult](#)

- bool **ok**

True if the operation succeeded.

- std::vector< [nsbaci::Error](#) > **errors**

Errors encountered (empty if ok is true).

8.35.1 Detailed Description

Result type for file load operations.

Extends [FileResult](#) with the loaded file contents and filename. On successful load, contains the file contents as a string and the filename for display purposes.

8.35.2 Constructor & Destructor Documentation

8.35.2.1 LoadResult() [1/3]

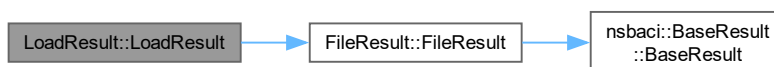
```
LoadResult::LoadResult (
    nsbaci::types::Text conts,
    nsbaci::types::File name) [inline]
```

Constructs a successful result with file contents.

Parameters

<i>conts</i>	The loaded file contents.
<i>name</i>	The filename (without path) for display.

Here is the call graph for this function:



8.35.2.2 LoadResult() [2/3]

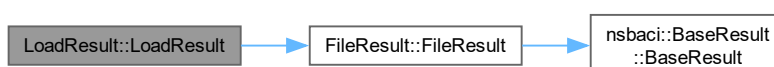
```
LoadResult::LoadResult (
    std::vector< nsbaci::Error > errs) [inline], [explicit]
```

Constructs a result from a vector of errors.

Parameters

<i>errs</i>	Vector of errors encountered during the load.
-------------	---

Here is the call graph for this function:



8.35.2.3 LoadResult() [3/3]

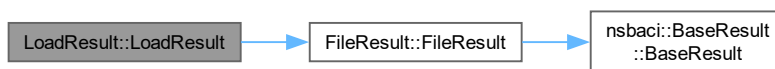
```
LoadResult::LoadResult (
    nsbaci::Error error) [inline], [explicit]
```

Constructs a failed result from a single error.

Parameters

<i>error</i>	The error that caused the load to fail.
--------------	---

Here is the call graph for this function:

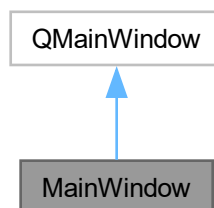


The documentation for this struct was generated from the following file:

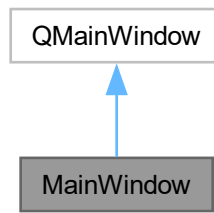
- [fileService.h](#)

8.36 MainWindow Class Reference

Inheritance diagram for MainWindow:



Collaboration diagram for MainWindow:



Public Slots

- void **setEditorContents** (const QString &contents)
- void **setStatusMessage** (const QString &message)
- void **setCurrentFile** (const QString &fileName, bool modified=false)
- void **onSaveSucceeded** ()
- void **onSaveFailed** (std::vector< [nsbaci::UIError](#) > errors)
- void **onLoadSucceeded** (const QString &contents)
- void **onLoadFailed** (std::vector< [nsbaci::UIError](#) > errors)
- void **onCompileSucceeded** ()
- void **onCompileFailed** (std::vector< [nsbaci::UIError](#) > errors)
- void **onRunStarted** (const QString &programName)
- void **onRuntimeStateChanged** (bool running, bool halted)
- void **onThreadsUpdated** (const std::vector< [nsbaci::ui::ThreadInfo](#) > &threads)
- void **onVariablesUpdated** (const std::vector< [nsbaci::ui::VariableInfo](#) > &variables)
- void **onOutputReceived** (const QString &output)
- void **onInputRequested** (const QString &prompt)

Signals

- void **saveRequested** (const QString &filePath, const QString &contents)
- void **openRequested** (const QString &filePath)
- void **compileRequested** (const QString &contents)
- void **runRequested** ()
- void **stepRequested** ()
- void **stepThreadRequested** (nsbaci::types::ThreadID threadId)
- void **runContinueRequested** ()
- void **pauseRequested** ()
- void **resetRequested** ()
- void **stopRequested** ()
- void **inputProvided** (const QString &input)

Public Member Functions

- **MainWindow** (QWidget *parent=nullptr)

The documentation for this class was generated from the following files:

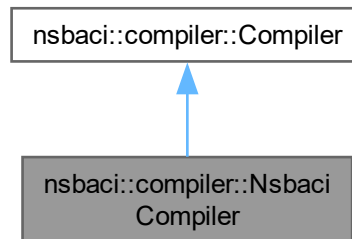
- [mainwindow.h](#)
- [mainwindow.cpp](#)

8.37 nsbaci::compiler::NsbaciCompiler Class Reference

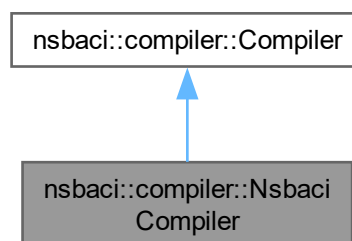
nsbaci compiler implementation using flex and bison.

```
#include <nsbaciCompiler.h>
```

Inheritance diagram for nsbaci::compiler::NsbaciCompiler:



Collaboration diagram for nsbaci::compiler::NsbaciCompiler:



Public Member Functions

- **NsbaciCompiler** ()=default
Default constructor.
- **~NsbaciCompiler** () override=default
Destructor.
- **CompilerResult compile** (const std::string &source) override
Compiles nsbaci source code from a string.
- **CompilerResult compile** (std::istream &input) override
Compiles nsbaci source code from an input stream.

Public Member Functions inherited from nsbaci::compiler::Compiler

- **Compiler** ()=default
Default constructor.
- virtual **~Compiler** ()=default
Virtual destructor.

8.37.1 Detailed Description

nsbaci compiler implementation using flex and bison.

Usage example:

```
NsbaciCompiler compiler;
auto result = compiler.compile("int x = 5; cout << x << endl;");
if (result.ok) {
    // result.instructions contains the p-code
    // result.symbols contains variable information
} else {
    // Handle compilation errors
}
```

8.37.2 Member Function Documentation

8.37.2.1 compile() [1/2]

```
CompilerResult nsbaci::compiler::NsbaciCompiler::compile (
    const std::string & source) [override], [virtual]
```

Compiles nsbaci source code from a string.

Creates a string stream from the source and delegates to the stream compile method.

Parameters

<i>source</i>	The nsbaci source code to compile.
---------------	------------------------------------

Returns

[CompilerResult](#) with instructions and symbols on success, or detailed error information on failure.

Implements [nsbaci::compiler::Compiler](#).

Here is the call graph for this function:



Here is the caller graph for this function:



8.37.2.2 compile() [2/2]

```
CompilerResult nsbaci::compiler::NsbaciCompiler::compile (
    std::istream & input) [override], [virtual]
```

Compiles nsbaci source code from an input stream.

Performs the actual compilation by:

1. Creating a [Lexer](#) to tokenize the input
2. Creating a Parser with the lexer
3. Running the parser to generate instructions
4. Collecting any errors that occurred

Parameters

<i>input</i>	The input stream containing nsbaci source code.
--------------	---

Returns

[CompilerResult](#) with instructions and symbols on success, or detailed error information on failure.

Implements [nsbaci::compiler::Compiler](#).

The documentation for this class was generated from the following files:

- [nsbaciCompiler.h](#)
- [nsbaciCompiler.cpp](#)

8.38 nsbaci::factories::NsbaciCompiler Struct Reference

The documentation for this struct was generated from the following file:

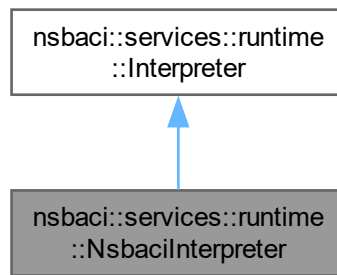
- [compilerServiceFactory.h](#)

8.39 nsbaci::services::runtime::NsbaciInterpreter Class Reference

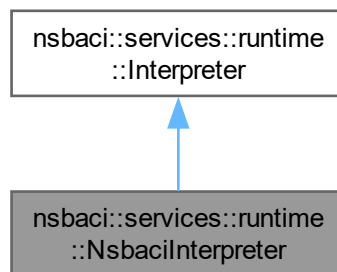
BACI-specific implementation of the [Interpreter](#).

```
#include <nsbaciInterpreter.h>
```

Inheritance diagram for nsbaci::services::runtime::NsbaciInterpreter:



Collaboration diagram for nsbaci::services::runtime::NsbaciInterpreter:



Public Member Functions

- [InterpreterResult](#) [executeInstruction](#) ([Thread](#) &t, [Program](#) &program) override
Executes the current instruction for the given thread with the program context.
- void [provideInput](#) (const std::string &input) override
Provide input to a thread waiting for input.
- bool [isWaitingForInput](#) () const override
Check if interpreter is waiting for input.
- void [setOutputCallback](#) ([OutputCallback](#) callback) override
Set the output callback for print operations.
- void [setDrawingCallback](#) ([DrawingCallback](#) callback) override
Set the drawing callback for drawing operations.
- void [reset](#) () override
Reset interpreter state for a new run.

8.39.1 Detailed Description

BACI-specific implementation of the [Interpreter](#).

[NsbaciInterpreter](#) executes BACI p-code instructions, managing the execution state and interaction with the program context.

8.39.2 Member Function Documentation

8.39.2.1 executeInstruction()

```
InterpreterResult nsbaci::services::runtime::NsbaciInterpreter::executeInstruction (
    Thread & t,
    Program & program) [override], [virtual]
```

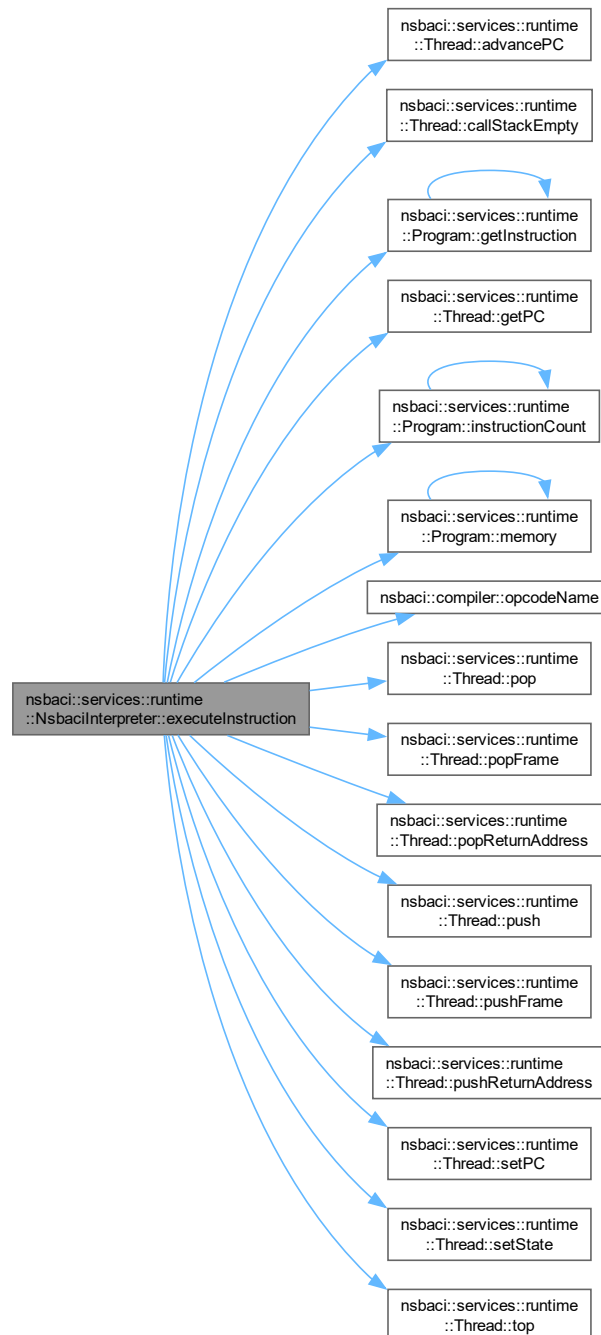
Executes the current instruction for the given thread with the program context.

Parameters

<i>t</i>	The thread whose instruction should be executed.
<i>program</i>	The program context in which to execute the instruction.

InterpreterResult indicating success or any errors encountered.

Here is the call graph for this function:



8.39.2.2 isWaitingForInput()

```
bool nsbaci::services::runtime::NsbaciInterpreter::isWaitingForInput () const [override],
[virtual]
```

Check if interpreter is waiting for input.

Returns

True if waiting for input.

Implements [nsbaci::services::runtime::Interpreter](#).

8.39.2.3 provideInput()

```
void nsbaci::services::runtime::NsbaciInterpreter::provideInput (
    const std::string & input) [override], [virtual]
```

Provide input to a thread waiting for input.

Parameters

<i>input</i>	The input string.
--------------	-------------------

Implements [nsbaci::services::runtime::Interpreter](#).

8.39.2.4 reset()

```
void nsbaci::services::runtime::NsbaciInterpreter::reset () [override], [virtual]
```

Reset interpreter state for a new run.

Clears any pending input and waiting state.

Implements [nsbaci::services::runtime::Interpreter](#).

8.39.2.5 setDrawingCallback()

```
void nsbaci::services::runtime::NsbaciInterpreter::setDrawingCallback (
    DrawingCallback callback) [override], [virtual]
```

Set the drawing callback for drawing operations.

Parameters

<i>callback</i>	Function to call when drawing is requested.
-----------------	---

Implements [nsbaci::services::runtime::Interpreter](#).

8.39.2.6 setOutputCallback()

Parameters

<i>callback</i>	Function to call when output is produced.
-----------------	---

Implements [nsbaci::services::runtime::Interpreter](#).

The documentation for this class was generated from the following files:

- [nsbaciInterpreter.h](#)
- [nsbaciInterpreter.cpp](#)

8.40 nsbaci::factories::NsbaciRuntime Struct Reference

The documentation for this struct was generated from the following file:

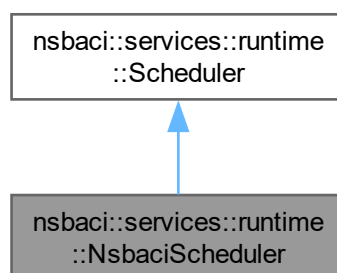
- [runtimeServiceFactory.h](#)

8.41 nsbaci::services::runtime::NsbaciScheduler Class Reference

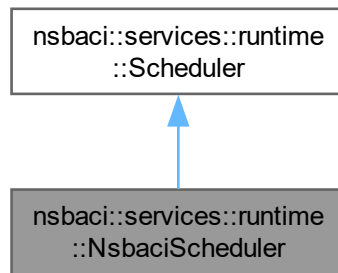
BACI-specific implementation of the [Scheduler](#).

```
#include <nsbaciScheduler.h>
```

Inheritance diagram for nsbaci::services::runtime::NsbaciScheduler:



Collaboration diagram for nsbaci::services::runtime::NsbaciScheduler:



Public Member Functions

- [Thread](#) * [pickNext](#) () override
Pick the next thread to run.
- void [addThread](#) ([Thread](#) thread) override
Add a new thread to the scheduler.
- void [blockCurrent](#) () override
Block the currently running thread.
- void [blockOnSemaphore](#) (uint32_t semaphoreAddr) override
Block current thread on a semaphore.
- size_t [unblockSemaphore](#) (uint32_t semaphoreAddr) override
Unblock threads waiting on a semaphore.
- void [unblock](#) (nsbaci::types::ThreadID threadId) override
Move a thread from blocked to ready state.
- void [yield](#) () override
Yield the current thread (move to back of ready queue).
- void [terminateCurrent](#) () override
Terminate the currently running thread.
- bool [hasThreads](#) () const override
Check if there are any threads left to run.
- [Thread](#) * [current](#) () override
Get the currently running thread.
- void [clear](#) () override
Clear all threads and reset scheduler state.
- void [unblockIO](#) () override
Move all I/O waiting threads back to ready queue.
- const std::vector< [Thread](#) > & [getThreads](#) () const override
Get all threads managed by the scheduler.
- void [blockOnCoend](#) (int32_t expectedThreads) override
Block current thread on coend (waiting for spawned threads).
- void [checkCoendUnblock](#) () override
Check if coend-blocked threads should be unblocked.

Additional Inherited Members

Protected Attributes inherited from [nsbaci::services::runtime::Scheduler](#)

- `std::vector< Thread > threads`
All threads owned by scheduler.
- `std::vector< size_t > readyQueue`
Indices of ready threads.
- `std::vector< size_t > blockedQueue`
Indices of blocked threads.
- `std::vector< size_t > ioQueue`
Indices of I/O waiting threads.
- `std::optional< size_t > runningIndex`
Index of currently running thread.
- `std::unordered_map< uint32_t, std::vector< size_t > > semaphoreQueues`
Map from semaphore address to list of waiting thread indices.
- `std::vector< std::pair< size_t, int32_t > > coendQueue`
Threads waiting at coend, with their expected thread counts.

8.41.1 Detailed Description

BACI-specific implementation of the [Scheduler](#).

[NsbaciScheduler](#) implements a round-robin scheduling algorithm with support for blocked, ready, running, and I/O waiting states. Threads are selected randomly from the ready queue to simulate non-deterministic concurrent execution.

8.41.2 Member Function Documentation

8.41.2.1 addThread()

```
void nsbaci::services::runtime::NsbaciScheduler::addThread (
    Thread thread) [override], [virtual]
```

Add a new thread to the scheduler.

Parameters

<i>thread</i>	The thread to add.
---------------	--------------------

Implements [nsbaci::services::runtime::Scheduler](#).

Here is the call graph for this function:



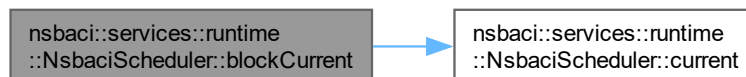
8.41.2.2 blockCurrent()

```
void nsbaci::services::runtime::NsbaciScheduler::blockCurrent () [override], [virtual]
```

Block the currently running thread.

Implements [nsbaci::services::runtime::Scheduler](#).

Here is the call graph for this function:



8.41.2.3 blockOnCoend()

```
void nsbaci::services::runtime::NsbaciScheduler::blockOnCoend (
    int32_t expectedThreads) [override], [virtual]
```

Block current thread on coend (waiting for spawned threads).

Parameters

<i>expectedThreads</i>	Number of threads that should terminate.
------------------------	--

Implements [nsbaci::services::runtime::Scheduler](#).

Here is the call graph for this function:



8.41.2.4 blockOnSemaphore()

```
void nsbaci::services::runtime::NsbaciScheduler::blockOnSemaphore (
    uint32_t semaphoreAddr) [override], [virtual]
```

Block current thread on a semaphore.

Parameters

<i>semaphoreAddr</i>	The address of the semaphore.
----------------------	-------------------------------

Implements [nsbaci::services::runtime::Scheduler](#).

Here is the call graph for this function:



8.41.2.5 checkCoendUnblock()

```
void nsbaci::services::runtime::NsbaciScheduler::checkCoendUnblock () [override], [virtual]
```

Check if coend-blocked threads should be unblocked.

Called after a thread terminates.

Implements [nsbaci::services::runtime::Scheduler](#).

Here is the caller graph for this function:



8.41.2.6 clear()

```
void nsbaci::services::runtime::NsbaciScheduler::clear () [override], [virtual]
```

Clear all threads and reset scheduler state.

Implements [nsbaci::services::runtime::Scheduler](#).

8.41.2.7 current()

```
Thread * nsbaci::services::runtime::NsbaciScheduler::current () [override], [virtual]
```

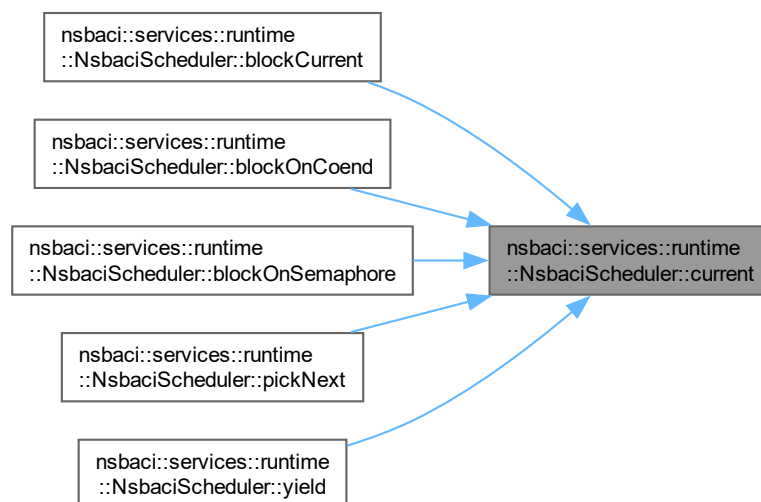
Get the currently running thread.

Returns

Pointer to current thread, or nullptr if none.

Implements [nsbaci::services::runtime::Scheduler](#).

Here is the caller graph for this function:



8.41.2.8 getThreads()

```
const std::vector< Thread > & nsbaci::services::runtime::NsbaciScheduler::getThreads () const [override], [virtual]
```

Get all threads managed by the scheduler.

Returns

Const reference to the threads vector.

Implements [nsbaci::services::runtime::Scheduler](#).

8.41.2.9 hasThreads()

```
bool nsbaci::services::runtime::NsbaciScheduler::hasThreads () const [override], [virtual]
```

Check if there are any threads left to run.

Returns

True if there are threads in any queue.

Implements [nsbaci::services::runtime::Scheduler](#).

8.41.2.10 pickNext()

```
Thread * nsbaci::services::runtime::NsbaciScheduler::pickNext () [override], [virtual]
```

Pick the next thread to run.

Returns

Pointer to the next thread, or nullptr if no threads are ready.

Implements [nsbaci::services::runtime::Scheduler](#).

Here is the call graph for this function:



8.41.2.11 terminateCurrent()

```
void nsbaci::services::runtime::NsbaciScheduler::terminateCurrent () [override], [virtual]
```

Terminate the currently running thread.

Implements [nsbaci::services::runtime::Scheduler](#).

Here is the call graph for this function:



Parameters

<i>threadId</i>	The ID of the thread to unblock.
-----------------	----------------------------------

Implements [nsbaci::services::runtime::Scheduler](#).

8.41.2.13 unblockIO()

```
void nsbaci::services::runtime::NsbaciScheduler::unblockIO () [override], [virtual]
```

Move all I/O waiting threads back to ready queue.

Called when input becomes available.

Implements [nsbaci::services::runtime::Scheduler](#).

8.41.2.14 unblockSemaphore()

```
size_t nsbaci::services::runtime::NsbaciScheduler::unblockSemaphore (
    uint32_t semaphoreAddr) [override], [virtual]
```

Unblock threads waiting on a semaphore.

Parameters

<i>semaphoreAddr</i>	The address of the semaphore.
----------------------	-------------------------------

Returns

Number of threads unblocked.

Implements [nsbaci::services::runtime::Scheduler](#).

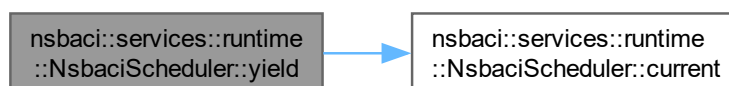
8.41.2.15 yield()

```
void nsbaci::services::runtime::NsbaciScheduler::yield () [override], [virtual]
```

Yield the current thread (move to back of ready queue).

Implements [nsbaci::services::runtime::Scheduler](#).

Here is the call graph for this function:



The documentation for this class was generated from the following files:

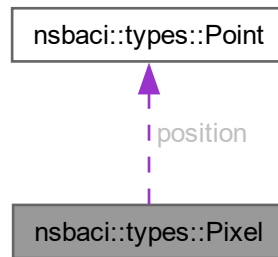
- [nsbaciScheduler.h](#)
- [nsbaciScheduler.cpp](#)

8.42 nsbaci::types::Pixel Struct Reference

Single pixel at a position.

```
#include <drawingTypes.h>
```

Collaboration diagram for nsbaci::types::Pixel:



Public Member Functions

- **Pixel** ([Point](#) p)
- **Pixel** (int32_t x, int32_t y)

Public Attributes

- [Point](#) position

8.42.1 Detailed Description

Single pixel at a position.

The documentation for this struct was generated from the following file:

- [drawingTypes.h](#)

8.43 nsbaci::types::Point Struct Reference

2D point/position representation.

```
#include <drawingTypes.h>
```

Public Member Functions

- **Point** (int32_t px, int32_t py)
- bool **operator==** (const Point &other) const
- Point **operator+** (const Point &other) const
- Point **operator-** (const Point &other) const

Public Attributes

- int32_t **x** = 0
- int32_t **y** = 0

8.43.1 Detailed Description

2D point/position representation.

The documentation for this struct was generated from the following file:

- [drawingTypes.h](#)

8.44 nsbaci::ui::Preferences Struct Reference

Holds all user-configurable preferences.

```
#include <preferencesDialog.h>
```

Public Attributes

- [Theme](#) **theme** = Theme::Dark
- int **editorFontSize** = 11
- int **tabWidth** = 4
- bool **showLineNumbers** = true
- bool **wordWrap** = false
- bool **restoreLastFile** = false
- QString **lastFilePath**

8.44.1 Detailed Description

Holds all user-configurable preferences.

This struct is both the runtime representation and the serialisation target (read from / written to QSettings).

The documentation for this struct was generated from the following file:

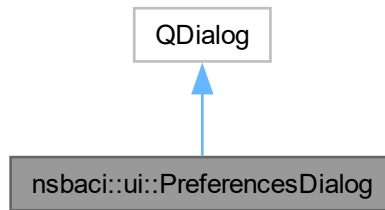
- [preferencesDialog.h](#)

8.45 nsbaci::ui::PreferencesDialog Class Reference

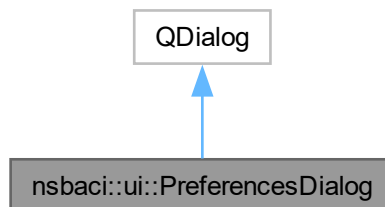
A dialog that allows the user to configure application preferences.

```
#include <preferencesDialog.h>
```

Inheritance diagram for nsbaci::ui::PreferencesDialog:



Collaboration diagram for nsbaci::ui::PreferencesDialog:



Signals

- void `preferencesChanged` (const `nsbaci::ui::Preferences` &prefs)
Emitted when the user accepts the dialog with changed preferences.

Public Member Functions

- **PreferencesDialog** (const `Preferences` ¤t, QWidget *parent=nullptr)
- `Preferences result` () const
Returns the (possibly modified) preferences after the dialog is accepted.

Static Public Member Functions

- static [Preferences](#) **load** ()
Load preferences from QSettings.
- static void **save** (const [Preferences](#) &prefs)
Save preferences to QSettings.

8.45.1 Detailed Description

A dialog that allows the user to configure application preferences.

Features a side menu for browsing preference categories and a stacked panel on the right that shows the appropriate settings widgets for each category. Changes are persisted to QSettings on accept.

8.45.2 Member Function Documentation

8.45.2.1 preferencesChanged

```
void nsbaci::ui::PreferencesDialog::preferencesChanged (  
    const nsbaci::ui::Preferences & prefs) [signal]
```

Emitted when the user accepts the dialog with changed preferences.

Parameters

<i>prefs</i>	The new preference values.
--------------	----------------------------

The documentation for this class was generated from the following files:

- [preferencesDialog.h](#)
- [preferencesDialog.cpp](#)

8.46 nsbaci::services::runtime::Program Class Reference

Represents a compiled program ready for execution.

```
#include <program.h>
```

Public Member Functions

- **Program** (nsbaci::compiler::InstructionStream i)
- **Program** (nsbaci::compiler::InstructionStream i, nsbaci::types::SymbolTable s)
- **Program** (const Program &)=delete
- Program & **operator=** (const Program &)=delete
- **Program** (Program &&)=default
- Program & **operator=** (Program &&)=default
- const nsbaci::compiler::Instruction & **getInstruction** (uint32_t addr) const
Gets instruction at the given address.
- size_t **instructionCount** () const
Gets the total number of instructions.
- nsbaci::types::Memory & **memory** ()
Access to global memory.
- const nsbaci::types::Memory & **memory** () const
- const nsbaci::types::SymbolTable & **symbols** () const
Access to symbol table.
- void **addSymbol** (nsbaci::types::SymbolInfo info)
Add a symbol to the symbol table.
- int32_t **readMemory** (nsbaci::types::MemoryAddr addr) const
Read a value from memory.
- void **writeMemory** (nsbaci::types::MemoryAddr addr, int32_t value)
Write a value to memory.
- void **createSemaphore** (nsbaci::types::MemoryAddr addr, int32_t initialCount)
Create a semaphore at the given address.
- bool **semaphoreWait** (nsbaci::types::MemoryAddr addr, nsbaci::types::ThreadID threadId)
Wait (P operation) on a semaphore.
- nsbaci::types::ThreadID **semaphoreSignal** (nsbaci::types::MemoryAddr addr)
Signal (V operation) on a semaphore.
- bool **hasSemaphore** (nsbaci::types::MemoryAddr addr) const
Check if a semaphore exists at the given address.

8.46.1 Detailed Description

Represents a compiled program ready for execution.

The [Program](#) class contains the instruction vector, memory tables, and other data structures needed for program execution.

8.46.2 Member Function Documentation

8.46.2.1 addSymbol()

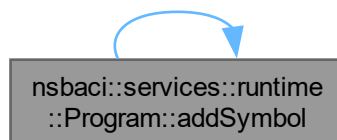
```
void nsbaci::services::runtime::Program::addSymbol (
    nsbaci::types::SymbolInfo info)
```

Add a symbol to the symbol table.

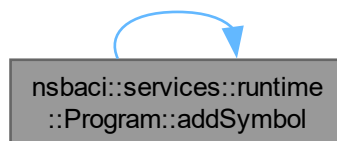
Parameters

<i>info</i>	The symbol information to add.
-------------	--------------------------------

Here is the call graph for this function:



Here is the caller graph for this function:

**8.46.2.2 createSemaphore()**

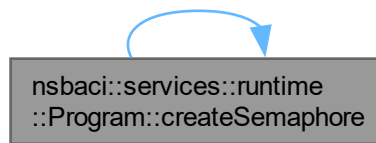
```
void nsbaci::services::runtime::Program::createSemaphore (
    nsbaci::types::MemoryAddr addr,
    int32_t initialCount)
```

Create a semaphore at the given address.

Parameters

<i>addr</i>	Memory address to associate with the semaphore.
<i>initialCount</i>	Initial semaphore count.

Here is the call graph for this function:



Here is the caller graph for this function:



8.46.2.3 getInstruction()

```
const nsbaci::compiler::Instruction & nsbaci::services::runtime::Program::getInstruction (
    uint32_t addr) const
```

Gets instruction at the given address.

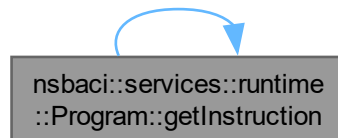
Parameters

<i>addr</i>	The instruction address.
-------------	--------------------------

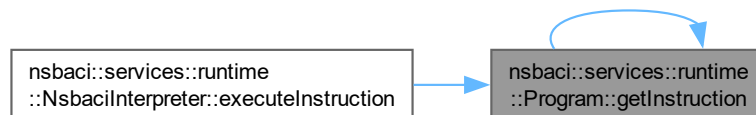
Returns

Reference to the instruction.

Here is the call graph for this function:



Here is the caller graph for this function:

**8.46.2.4 hasSemaphore()**

```
bool nsbaci::services::runtime::Program::hasSemaphore (
    nsbaci::types::MemoryAddr addr) const
```

Check if a semaphore exists at the given address.

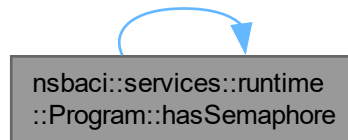
Parameters

<i>addr</i>	Address to check.
-------------	-------------------

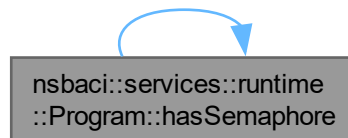
Returns

true if semaphore exists, false otherwise.

Here is the call graph for this function:



Here is the caller graph for this function:

**8.46.2.5 instructionCount()**

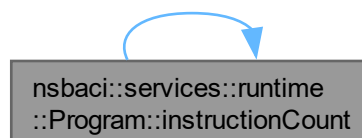
```
size_t nsbaci::services::runtime::Program::instructionCount () const
```

Gets the total number of instructions.

Returns

Number of instructions in the program.

Here is the call graph for this function:



Here is the caller graph for this function:



8.46.2.6 `memory()`

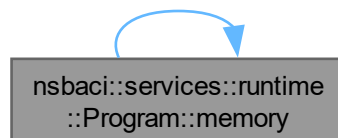
```
nsbaci::types::Memory & nsbaci::services::runtime::Program::memory ()
```

Access to global memory.

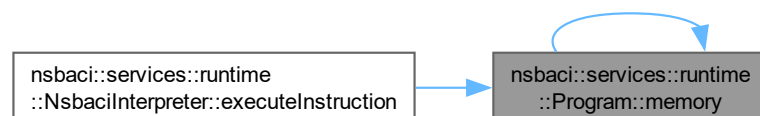
Returns

Reference to memory.

Here is the call graph for this function:



Here is the caller graph for this function:



8.46.2.7 `readMemory()`

```
int32_t nsbaci::services::runtime::Program::readMemory (
    nsbaci::types::MemoryAddr addr) const
```

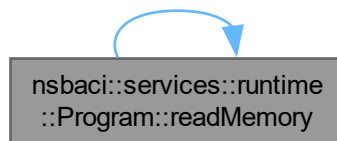

Parameters

<i>addr</i>	Memory address to read from.
-------------	------------------------------

Returns

Value at the address.

Here is the call graph for this function:



Here is the caller graph for this function:

**8.46.2.8 semaphoreSignal()**

```
nsbaci::types::ThreadID nsbaci::services::runtime::Program::semaphoreSignal (  
    nsbaci::types::MemoryAddr addr)
```

Signal (V operation) on a semaphore.

Parameters

<i>addr</i>	Address of the semaphore.
-------------	---------------------------

Returns

ThreadID to wake, or 0 if none waiting.

Here is the call graph for this function:



Here is the caller graph for this function:

**8.46.2.9 semaphoreWait()**

```
bool nsbaci::services::runtime::Program::semaphoreWait (
    nsbaci::types::MemoryAddr addr,
    nsbaci::types::ThreadID threadId)
```

Wait (P operation) on a semaphore.

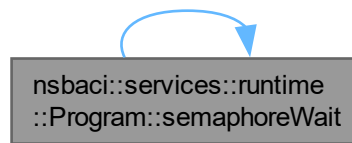
Parameters

<i>addr</i>	Address of the semaphore.
<i>threadId</i>	The thread attempting to wait.

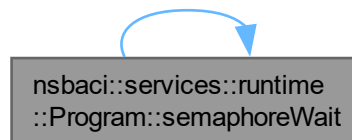
Returns

true if thread can proceed, false if it must block.

Here is the call graph for this function:



Here is the caller graph for this function:

**8.46.2.10 symbols()**

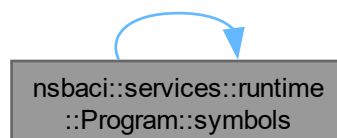
```
const nsbaci::types::SymbolTable & nsbaci::services::runtime::Program::symbols () const
```

Access to symbol table.

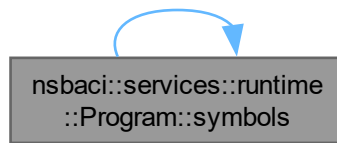
Returns

Const reference to symbol table.

Here is the call graph for this function:



Here is the caller graph for this function:



8.46.2.11 writeMemory()

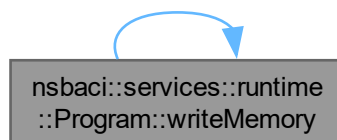
```
void nsbaci::services::runtime::Program::writeMemory (
    nsbaci::types::MemoryAddr addr,
    int32_t value)
```

Write a value to memory.

Parameters

<i>addr</i>	Memory address to write to.
<i>value</i>	Value to write.

Here is the call graph for this function:



Here is the caller graph for this function:



The documentation for this class was generated from the following files:

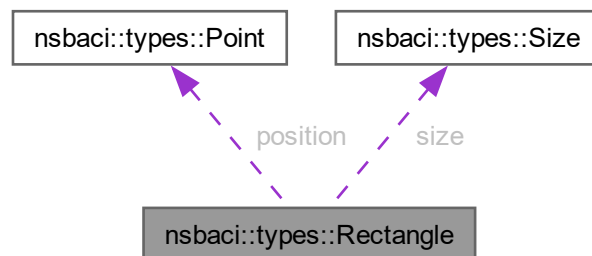
- [program.h](#)
- [program.cpp](#)

8.47 nsbaci::types::Rectangle Struct Reference

[Rectangle](#) shape with position and size.

```
#include <drawingTypes.h>
```

Collaboration diagram for nsbaci::types::Rectangle:



Public Member Functions

- **Rectangle** ([Point](#) pos, [Size](#) s, bool fill=false)
- **Rectangle** (int32_t x, int32_t y, int32_t w, int32_t h, bool fill=false)

Public Attributes

- [Point](#) **position**
- [Size](#) **size**
- bool **filled** = false

8.47.1 Detailed Description

[Rectangle](#) shape with position and size.

The documentation for this struct was generated from the following file:

- [drawingTypes.h](#)

8.48 nsbaci::types::RuntimeError Struct Reference

[Error](#) payload for runtime errors.

```
#include <errorTypes.h>
```

8.48.1 Detailed Description

[Error](#) payload for runtime errors.

The documentation for this struct was generated from the following file:

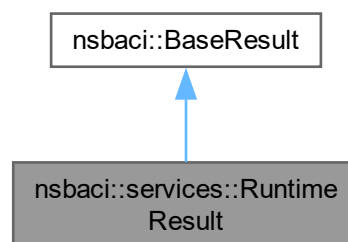
- [errorTypes.h](#)

8.49 nsbaci::services::RuntimeResult Struct Reference

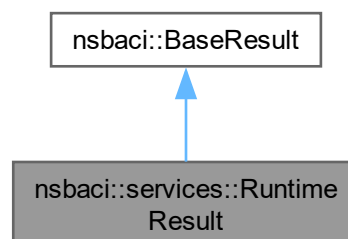
Result of a runtime operation (step, run, etc.).

```
#include <runtimeService.h>
```

Inheritance diagram for nsbaci::services::RuntimeResult:



Collaboration diagram for nsbaci::services::RuntimeResult:



Public Member Functions

- **RuntimeResult** ()
Default constructor creates a successful result.
- **RuntimeResult** (std::vector< nsbaci::Error > errs)
Constructs a result from a vector of errors.
- **RuntimeResult** (nsbaci::Error error)
Constructs a failed result from a single error.
- **RuntimeResult** (RuntimeResult &&) noexcept=default
- **RuntimeResult** & **operator=** (RuntimeResult &&) noexcept=default
- **RuntimeResult** (const RuntimeResult &)=default
- **RuntimeResult** & **operator=** (const RuntimeResult &)=default

Public Member Functions inherited from nsbaci::BaseResult

- **BaseResult** ()
Default constructor creates a successful result.
- **BaseResult** (std::vector< nsbaci::Error > errs)
Constructs a result from a vector of errors.
- **BaseResult** (nsbaci::Error error)
Constructs a failed result from a single error.
- **BaseResult** (BaseResult &&) noexcept=default
- **BaseResult** & **operator=** (BaseResult &&) noexcept=default
- **BaseResult** (const BaseResult &)=default
- **BaseResult** & **operator=** (const BaseResult &)=default

Public Attributes

- bool **halted** = false
True if program has terminated.
- bool **needsInput** = false
True if waiting for user input.
- std::string **inputPrompt**
Prompt to show for input.
- std::string **output**
Output produced by this step.

Public Attributes inherited from nsbaci::BaseResult

- bool **ok**
True if the operation succeeded.
- std::vector< nsbaci::Error > **errors**
Errors encountered (empty if ok is true).

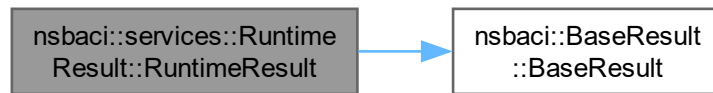
8.49.1 Detailed Description

Result of a runtime operation (step, run, etc.).

Parameters

<i>errs</i>	Vector of runtime errors.
-------------	---------------------------

Here is the call graph for this function:

**8.49.2.2 RuntimeResult() [2/2]**

```

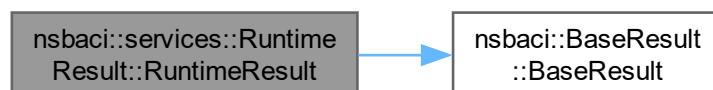
nsbaci::services::RuntimeResult::RuntimeResult (
    nsbaci::Error error) [inline], [explicit]
  
```

Constructs a failed result from a single error.

Parameters

<i>error</i>	The runtime error that occurred.
--------------	----------------------------------

Here is the call graph for this function:



The documentation for this struct was generated from the following file:

- [runtimeService.h](#)

8.50 nsbaci::services::RuntimeService Class Reference

Service that manages program execution.

```
#include <runtimeService.h>
```


Public Member Functions

- [RuntimeService](#) ()=default
Default constructor creates an uninitialized service.
- [RuntimeService](#) (std::unique_ptr< [runtime::Interpreter](#) > i, std::unique_ptr< [runtime::Scheduler](#) > s)
Constructs a [RuntimeService](#) with interpreter and scheduler.
- [~RuntimeService](#) ()=default
Default destructor.
- [RuntimeService](#) (const RuntimeService &)=delete
- [RuntimeService](#) & **operator=** (const [RuntimeService](#) &)=delete
- [RuntimeService](#) (RuntimeService &&)=default
- [RuntimeService](#) & **operator=** ([RuntimeService](#) &&)=default
- void [loadProgram](#) ([runtime::Program](#) &&p)
Loads a compiled program for execution.
- void [reset](#) ()
Resets the runtime to initial state.
- [RuntimeResult](#) [step](#) ()
Executes a single instruction for any ready thread.
- [RuntimeResult](#) [stepThread](#) (nsbaci::types::ThreadID threadId)
Executes a single instruction for a specific thread.
- [RuntimeResult](#) [run](#) (size_t maxSteps=0)
Runs the program until halted, error, or step limit.
- void [pause](#) ()
Pauses continuous execution.
- [RuntimeState](#) [getState](#) () const
Gets the current runtime state.
- bool [isHalted](#) () const
Checks if the program has finished execution.
- size_t [threadCount](#) () const
Gets the number of active threads.
- const std::vector< [runtime::Thread](#) > & [getThreads](#) () const
Gets all threads from the scheduler.
- const [runtime::Program](#) & [getProgram](#) () const
Gets the loaded program.
- void [provideInput](#) (const std::string &input)
Provides input to the runtime.
- bool [isWaitingForInput](#) () const
Checks if the runtime is waiting for user input.
- void [setOutputCallback](#) ([runtime::OutputCallback](#) callback)
Sets the callback for output operations.
- void [setDrawingCallback](#) ([runtime::DrawingCallback](#) callback)
Sets the callback for drawing operations.

8.50.1 Detailed Description

Service that manages program execution.

The service is move-only to ensure single ownership of the interpreter and scheduler components.

Usage example:

```
RuntimeService rs(std::make_unique<NsbaciInterpreter>(),
                  std::make_unique<NsbaciScheduler>());
rs.loadProgram(std::move(compiledProgram));
rs.setOutputCallback([](const std::string& out) { std::cout << out; });

while (!rs.isHalted()) {
    auto result = rs.step();
    if (result.needsInput) {
        rs.provideInput(getUserInput());
    }
}
```

8.50.2 Constructor & Destructor Documentation

8.50.2.1 RuntimeService() [1/2]

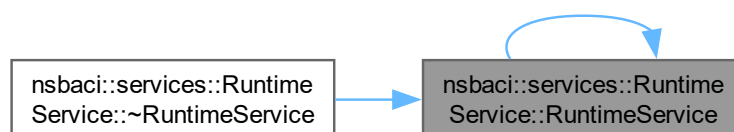
```
nsbaci::services::RuntimeService::RuntimeService () [default]
```

Default constructor creates an uninitialized service.

A service created this way must have its interpreter and scheduler set before use, typically via move assignment from a factory-created instance. Here is the call graph for this function:



Here is the caller graph for this function:



8.50.2.2 RuntimeService() [2/2]

```
nsbaci::services::RuntimeService::RuntimeService (
    std::unique_ptr< runtime::Interpreter > i,
    std::unique_ptr< runtime::Scheduler > s)
```

Constructs a [RuntimeService](#) with interpreter and scheduler.

Parameters

<i>i</i>	Unique pointer to the interpreter implementation.
<i>s</i>	Unique pointer to the scheduler implementation.

8.50.3 Member Function Documentation

8.50.3.1 getProgram()

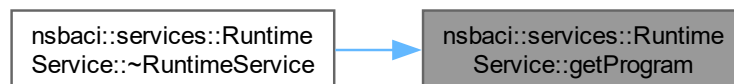
```
const runtime::Program & nsbaci::services::RuntimeService::getProgram () const
```

Gets the loaded program.

Returns

Const reference to the program for accessing instructions and memory.

Here is the caller graph for this function:



8.50.3.2 getState()

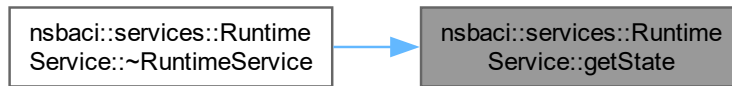
```
RuntimeState nsbaci::services::RuntimeService::getState () const
```

Gets the current runtime state.

Returns

The current [RuntimeState](#) value.

Here is the caller graph for this function:

**8.50.3.3 getThreads()**

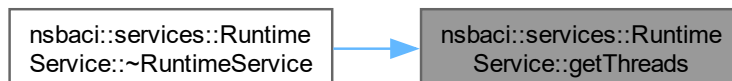
```
const std::vector< runtime::Thread > & nsbaci::services::RuntimeService::getThreads () const
```

Gets all threads from the scheduler.

Returns

Const reference to the threads vector for UI display.

Here is the caller graph for this function:

**8.50.3.4 isHalted()**

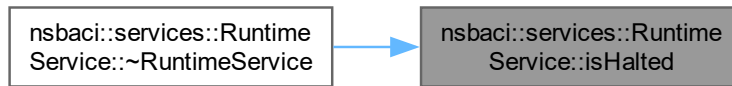
```
bool nsbaci::services::RuntimeService::isHalted () const
```

Checks if the program has finished execution.

Returns

True if state is Halted.

Here is the caller graph for this function:

**8.50.3.5 isWaitingForInput()**

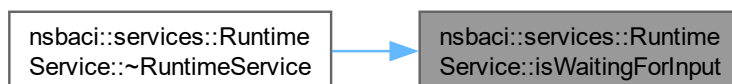
```
bool nsbaci::services::RuntimeService::isWaitingForInput () const
```

Checks if the runtime is waiting for user input.

Returns

True if a Read instruction is blocking execution.

Here is the caller graph for this function:

**8.50.3.6 loadProgram()**

```
void nsbaci::services::RuntimeService::loadProgram (  
    runtime::Program && p)
```

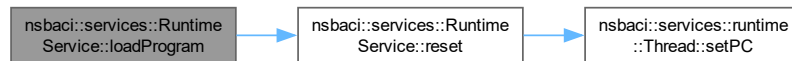
Loads a compiled program for execution.

Initializes the runtime with the program's instructions, symbol table, and memory. Creates the initial main thread and sets state to Paused ready for execution.

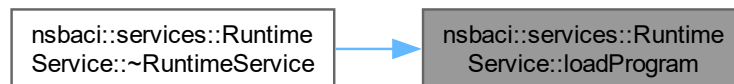
Parameters

<i>p</i>	The compiled program to load (which must be moved into the service).
----------	--

Here is the call graph for this function:



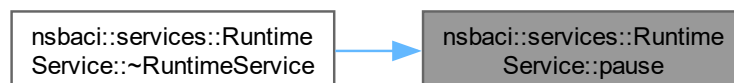
Here is the caller graph for this function:

**8.50.3.7 pause()**

```
void nsbaci::services::RuntimeService::pause ()
```

Pauses continuous execution.

Only affects state if currently Running; changes state to Paused. Here is the caller graph for this function:

**8.50.3.8 provideInput()**

```
void nsbaci::services::RuntimeService::provideInput (
    const std::string & input)
```

Provides input to the runtime.

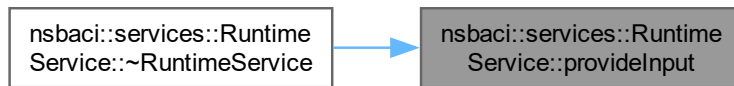
Generated by Doxygen

Called when the user provides input in response to a Read instruction. The input is stored and will be consumed on the next execution step.

Parameters

<i>input</i>	The user-provided input string.
--------------	---------------------------------

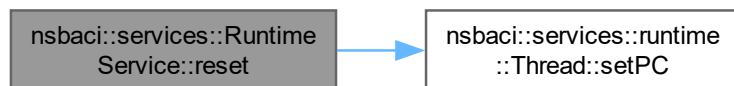
Here is the caller graph for this function:

**8.50.3.9 reset()**

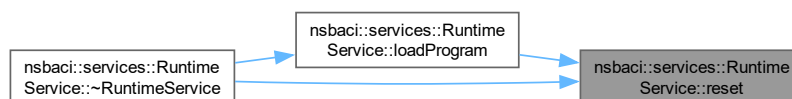
```
void nsbaci::services::RuntimeService::reset ()
```

Resets the runtime to initial state.

Clears all threads, resets memory, and sets state to Idle. The program must be reloaded before execution can continue. Here is the call graph for this function:



Here is the caller graph for this function:



8.50.3.10 run()

```
RuntimeResult nsbaci::services::RuntimeService::run (
    size_t maxSteps = 0)
```

Runs the program until halted, error, or step limit.

Executes instructions continuously until:

- The program halts (reaches Halt instruction)
- An error occurs
- The maximum step count is reached
- Input is required

Parameters

<i>maxSteps</i>	Maximum instructions to execute (0 = unlimited).
-----------------	--

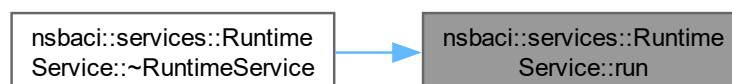
Returns

[RuntimeResult](#) with final execution state.

Here is the call graph for this function:



Here is the caller graph for this function:



8.50.3.11 setDrawingCallback()

```
void nsbaci::services::RuntimeService::setDrawingCallback (
    runtime::DrawingCallback callback)
```

Sets the callback for drawing operations

Parameters

<i>callback</i>	Function to call with DrawCommand objects.
-----------------	--

Here is the caller graph for this function:

**8.50.3.12 setOutputCallback()**

```
void nsbaci::services::RuntimeService::setOutputCallback (
    runtime::OutputCallback callback)
```

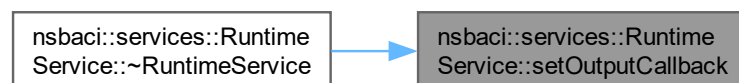
Sets the callback for output operations.

The callback is invoked whenever the program produces output (`Write`, `WriteIn`, `WriteRawString` instructions).

Parameters

<i>callback</i>	Function to call with output strings.
-----------------	---------------------------------------

Here is the caller graph for this function:

**8.50.3.13 step()**

```
RuntimeResult nsbaci::services::RuntimeService::step ()
```

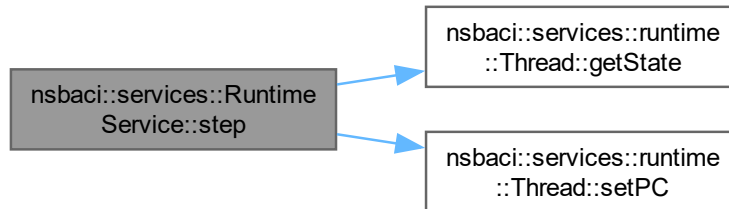
Executes a single instruction for any ready thread.

The scheduler picks the next thread to run and the interpreter executes one instruction from that thread's current position.

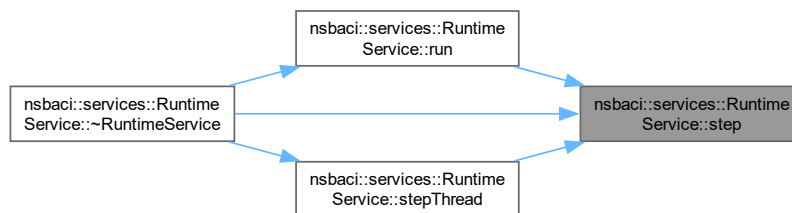
Returns

[RuntimeResult](#) with execution outcome, output, and I/O state.

Here is the call graph for this function:



Here is the caller graph for this function:

**8.50.3.14 stepThread()**

```
RuntimeResult nsbaci::services::RuntimeService::stepThread (
    nsbaci::types::ThreadID threadId)
```

Executes a single instruction for a specific thread.

Allows targeted debugging by stepping only the specified thread.

Parameters

<i>threadId</i>	The ID of the thread to step.
-----------------	-------------------------------

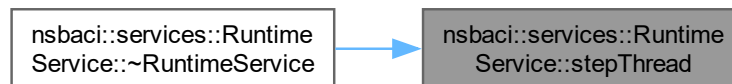
Returns

[RuntimeResult](#) with execution outcome.

Here is the call graph for this function:



Here is the caller graph for this function:

**8.50.3.15 threadCount()**

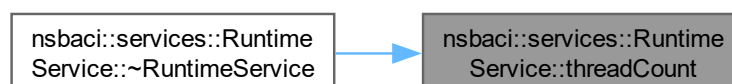
```
size_t nsbaci::services::RuntimeService::threadCount () const
```

Gets the number of active threads.

Returns

Count of threads in the scheduler.

Here is the caller graph for this function:



The documentation for this class was generated from the following files:

- [runtimeService.h](#)
- [runtimeService.cpp](#)

8.51 nsbaci::factories::RuntimeServiceFactory Class Reference

Factory for creating RuntimeService instances.

```
#include <runtimeServiceFactory.h>
```

Static Public Member Functions

- static [nsbaci::services::RuntimeService](#) **createService** ([NsbaciRuntime](#) t)

8.51.1 Detailed Description

Factory for creating RuntimeService instances.

The documentation for this class was generated from the following files:

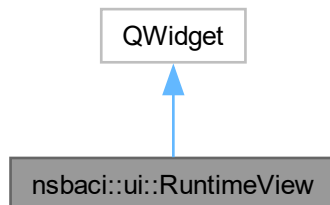
- [runtimeServiceFactory.h](#)
- [runtimeServiceFactory.cpp](#)

8.52 nsbaci::ui::RuntimeView Class Reference

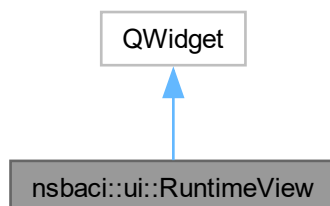
Widget displaying runtime execution state.

```
#include <runtimeView.h>
```

Inheritance diagram for nsbaci::ui::RuntimeView:



Collaboration diagram for nsbaci::ui::RuntimeView:



Public Slots

- void **updateThreads** (const std::vector< [ThreadInfo](#) > &threads)
- void **updateVariables** (const std::vector< [VariableInfo](#) > &variables)
- void **updateCurrentInstruction** (const QString &instruction)
- void **updateExecutionState** (bool running, bool halted)
- void **appendOutput** (const QString &text)
- void **requestInput** (const QString &prompt)
- void **clearConsole** ()
- void **onProgramLoaded** (const QString &programName)
- void **onProgramHalted** ()

Signals

- void **stepRequested** ()
- void **stepThreadRequested** (nsbaci::types::ThreadID threadId)
- void **runRequested** ()
- void **pauseRequested** ()
- void **resetRequested** ()
- void **stopRequested** ()
- void **inputProvided** (const QString &input)

Public Member Functions

- **RuntimeView** (QWidget *parent=nullptr)

8.52.1 Detailed Description

Widget displaying runtime execution state.

Shows:

- Thread list with states and current instruction
- Variables/memory watch panel
- I/O console for program input/output
- Execution controls (step, run, pause, reset)

The documentation for this class was generated from the following files:

- [runtimeView.h](#)
- [runtimeView.cpp](#)

8.53 nsbaci::types::SaveError Struct Reference

[Error](#) payload for file save errors.

```
#include <errorTypes.h>
```

Public Attributes

- [File](#) `associatedFile`

8.53.1 Detailed Description

[Error](#) payload for file save errors.

The documentation for this struct was generated from the following file:

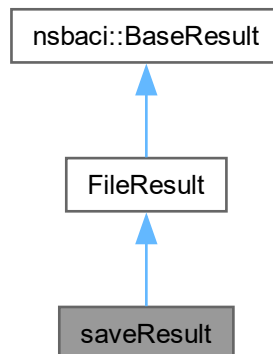
- [errorTypes.h](#)

8.54 saveResult Struct Reference

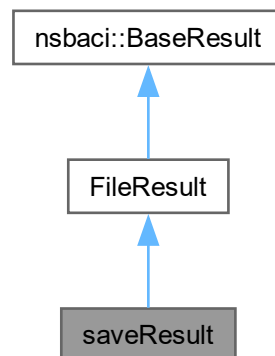
Result type for file save operations.

```
#include <fileService.h>
```

Inheritance diagram for `saveResult`:



Collaboration diagram for saveResult:



Public Member Functions

- **saveResult** ()
Default constructor creates a successful result.
- **saveResult** (std::vector< nsbaci::Error > errs)
Constructs a result from a vector of errors.
- **saveResult** (nsbaci::Error error)
Constructs a failed result from a single error.
- **saveResult** (saveResult &&) noexcept=default
- **saveResult & operator=** (saveResult &&) noexcept=default
- **saveResult** (const saveResult &)=default
- **saveResult & operator=** (const saveResult &)=default

Public Member Functions inherited from FileResult

- **FileResult** ()
Default constructor creates a successful result.
- **FileResult** (std::vector< nsbaci::Error > errs)
Constructs a result from a vector of errors.
- **FileResult** (nsbaci::Error error)
Constructs a failed result from a single error.
- **FileResult** (FileResult &&) noexcept=default
- **FileResult & operator=** (FileResult &&) noexcept=default
- **FileResult** (const FileResult &)=default
- **FileResult & operator=** (const FileResult &)=default

Public Member Functions inherited from `nsbaci::BaseResult`

- **BaseResult** ()
Default constructor creates a successful result.
- **BaseResult** (std::vector< `nsbaci::Error` > errs)
Constructs a result from a vector of errors.
- **BaseResult** (`nsbaci::Error` error)
Constructs a failed result from a single error.
- **BaseResult** (BaseResult &&) noexcept=default
- **BaseResult** & **operator=** (BaseResult &&) noexcept=default
- **BaseResult** (const BaseResult &)=default
- **BaseResult** & **operator=** (const BaseResult &)=default

Additional Inherited Members

Public Attributes inherited from `nsbaci::BaseResult`

- bool **ok**
True if the operation succeeded.
- std::vector< `nsbaci::Error` > **errors**
Errors encountered (empty if ok is true).

8.54.1 Detailed Description

Result type for file save operations.

Contains only success/failure status and error information since save operations do not return additional data on success.

8.54.2 Constructor & Destructor Documentation

8.54.2.1 `saveResult()` [1/2]

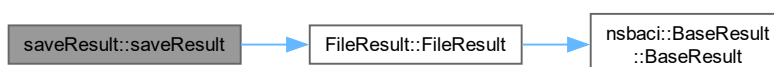
```
saveResult::saveResult (
    std::vector< nsbaci::Error > errs) [inline], [explicit]
```

Constructs a result from a vector of errors.

Parameters

<code>errs</code>	Vector of errors encountered during the save.
-------------------	---

Here is the call graph for this function:



8.54.2.2 saveResult() [2/2]

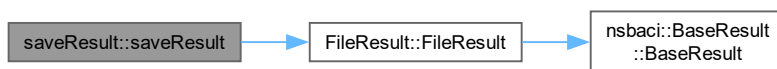
```
saveResult::saveResult (
    nsbaci::Error error) [inline], [explicit]
```

Constructs a failed result from a single error.

Parameters

<i>error</i>	The error that caused the save to fail.
--------------	---

Here is the call graph for this function:



The documentation for this struct was generated from the following file:

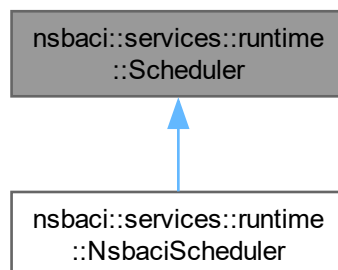
- [fileService.h](#)

8.55 nsbaci::services::runtime::Scheduler Class Reference

Manages thread scheduling and state transitions.

```
#include <scheduler.h>
```

Inheritance diagram for nsbaci::services::runtime::Scheduler:



Public Member Functions

- virtual [Thread](#) * [pickNext](#) ()=0
Pick the next thread to run.
- virtual void [addThread](#) ([Thread](#) thread)=0
Add a new thread to the scheduler.
- virtual void [blockCurrent](#) ()=0
Block the currently running thread.
- virtual void [blockOnSemaphore](#) (uint32_t semaphoreAddr)=0
Block current thread on a semaphore.
- virtual size_t [unblockSemaphore](#) (uint32_t semaphoreAddr)=0
Unblock threads waiting on a semaphore.
- virtual void [unblock](#) (nsbaci::types::ThreadID threadId)=0
Move a thread from blocked to ready state.
- virtual void [yield](#) ()=0
Yield the current thread (move to back of ready queue).
- virtual void [terminateCurrent](#) ()=0
Terminate the currently running thread.
- virtual bool [hasThreads](#) () const =0
Check if there are any threads left to run.
- virtual [Thread](#) * [current](#) ()=0
Get the currently running thread.
- virtual void [clear](#) ()=0
Clear all threads and reset scheduler state.
- virtual void [unblockIO](#) ()=0
Move all I/O waiting threads back to ready queue.
- virtual const std::vector< [Thread](#) > & [getThreads](#) () const =0
Get all threads managed by the scheduler.
- virtual void [blockOnCoend](#) (int32_t expectedThreads)=0
Block current thread on coend (waiting for spawned threads).
- virtual void [checkCoendUnblock](#) ()=0
Check if coend-blocked threads should be unblocked.

Protected Attributes

- std::vector< [Thread](#) > **threads**
All threads owned by scheduler.
- std::vector< size_t > **readyQueue**
Indices of ready threads.
- std::vector< size_t > **blockedQueue**
Indices of blocked threads.
- std::vector< size_t > **ioQueue**
Indices of I/O waiting threads.
- std::optional< size_t > **runningIndex**
Index of currently running thread.
- std::unordered_map< uint32_t, std::vector< size_t > > **semaphoreQueues**
Map from semaphore address to list of waiting thread indices.
- std::vector< std::pair< size_t, int32_t > > **coendQueue**
Threads waiting at coend, with their expected thread counts.

8.55.1 Detailed Description

Manages thread scheduling and state transitions.

The [Scheduler](#) is responsible for determining which thread runs next, managing thread queues for different states, and handling thread state transitions.

8.55.2 Member Function Documentation

8.55.2.1 addThread()

```
virtual void nsbaci::services::runtime::Scheduler::addThread (  
    Thread thread) [pure virtual]
```

Add a new thread to the scheduler.

Parameters

<i>thread</i>	The thread to add.
---------------	--------------------

Implemented in [nsbaci::services::runtime::NsbaciScheduler](#).

8.55.2.2 blockCurrent()

```
virtual void nsbaci::services::runtime::Scheduler::blockCurrent () [pure virtual]
```

Block the currently running thread.

Implemented in [nsbaci::services::runtime::NsbaciScheduler](#).

8.55.2.3 blockOnCoend()

```
virtual void nsbaci::services::runtime::Scheduler::blockOnCoend (  
    int32_t expectedThreads) [pure virtual]
```

Block current thread on coend (waiting for spawned threads).

Parameters

<i>expectedThreads</i>	Number of threads that should terminate.
------------------------	--

Implemented in [nsbaci::services::runtime::NsbaciScheduler](#).

8.55.2.4 blockOnSemaphore()

```
virtual void nsbaci::services::runtime::Scheduler::blockOnSemaphore (  
    uint32_t semaphoreAddr) [pure virtual]
```

Generated by Doxygen

Block current thread on a semaphore.

Parameters

<i>semaphoreAddr</i>	The address of the semaphore.
----------------------	-------------------------------

Implemented in [nsbaci::services::runtime::NsbaciScheduler](#).

8.55.2.5 checkCoendUnblock()

```
virtual void nsbaci::services::runtime::Scheduler::checkCoendUnblock () [pure virtual]
```

Check if coend-blocked threads should be unblocked.

Called after a thread terminates.

Implemented in [nsbaci::services::runtime::NsbaciScheduler](#).

8.55.2.6 clear()

```
virtual void nsbaci::services::runtime::Scheduler::clear () [pure virtual]
```

Clear all threads and reset scheduler state.

Implemented in [nsbaci::services::runtime::NsbaciScheduler](#).

8.55.2.7 current()

```
virtual Thread * nsbaci::services::runtime::Scheduler::current () [pure virtual]
```

Get the currently running thread.

Returns

Pointer to current thread, or nullptr if none.

Implemented in [nsbaci::services::runtime::NsbaciScheduler](#).

8.55.2.8 getThreads()

```
virtual const std::vector< Thread > & nsbaci::services::runtime::Scheduler::getThreads ()  
const [pure virtual]
```

Get all threads managed by the scheduler.

Returns

Const reference to the threads vector.

Implemented in [nsbaci::services::runtime::NsbaciScheduler](#).

8.55.2.9 hasThreads()

```
virtual bool nsbaci::services::runtime::Scheduler::hasThreads () const [pure virtual]
```

Check if there are any threads left to run.

Returns

True if there are threads in any queue.

Implemented in [nsbaci::services::runtime::NsbaciScheduler](#).

8.55.2.10 pickNext()

```
virtual Thread * nsbaci::services::runtime::Scheduler::pickNext () [pure virtual]
```

Pick the next thread to run.

Returns

Pointer to the next thread, or nullptr if no threads are ready.

Implemented in [nsbaci::services::runtime::NsbaciScheduler](#).

8.55.2.11 terminateCurrent()

```
virtual void nsbaci::services::runtime::Scheduler::terminateCurrent () [pure virtual]
```

Terminate the currently running thread.

Implemented in [nsbaci::services::runtime::NsbaciScheduler](#).

8.55.2.12 unblock()

```
virtual void nsbaci::services::runtime::Scheduler::unblock (
    nsbaci::types::ThreadID threadId) [pure virtual]
```

Move a thread from blocked to ready state.

Parameters

<i>threadId</i>	The ID of the thread to unblock.
-----------------	----------------------------------

Implemented in [nsbaci::services::runtime::NsbaciScheduler](#).

8.55.2.13 unblockIO()

Generated by Doxygen

```
virtual void nsbaci::services::runtime::Scheduler::unblockIO () [pure virtual]
```

Move all I/O waiting threads back to ready queue.

Parameters

<i>semaphoreAddr</i>	The address of the semaphore.
----------------------	-------------------------------

Returns

Number of threads unblocked.

Implemented in [nsbaci::services::runtime::NsbaciScheduler](#).

8.55.2.15 yield()

```
virtual void nsbaci::services::runtime::Scheduler::yield () [pure virtual]
```

Yield the current thread (move to back of ready queue).

Implemented in [nsbaci::services::runtime::NsbaciScheduler](#).

The documentation for this class was generated from the following file:

- [scheduler.h](#)

8.56 nsbaci::services::runtime::Semaphore Class Reference

Counting semaphore for process synchronization.

```
#include <semaphore.h>
```

Public Member Functions

- **Semaphore** (int32_t initialCount=0)
- bool [wait](#) (nsbaci::types::ThreadID currentThread)
P operation (wait/decrement).
- nsbaci::types::ThreadID [signal](#) ()
V operation (signal/increment).
- int32_t **getCount** () const
Get current count (for debugging).
- bool **hasWaiting** () const
Check if any threads are blocked.

8.56.1 Detailed Description

Counting semaphore for process synchronization.

Created at runtime when StoreSemaphore instruction executes.

8.56.2 Member Function Documentation

8.56.2.1 signal()

```
nsbaci::types::ThreadID nsbaci::services::runtime::Semaphore::signal ()
```

V operation (signal/increment).

Returns

ThreadID to wake, or 0 if none waiting

8.56.2.2 wait()

```
bool nsbaci::services::runtime::Semaphore::wait (
    nsbaci::types::ThreadID currentThread)
```

P operation (wait/decrement).

Parameters

<i>currentThread</i>	The thread attempting to wait
----------------------	-------------------------------

Returns

true if thread can proceed, false if it must block

The documentation for this class was generated from the following files:

- [semaphore.h](#)
- semaphore.cpp

8.57 nsbaci::types::Size Struct Reference

2D size representation.

```
#include <drawingTypes.h>
```

Public Member Functions

- **Size** (int32_t w, int32_t h)

Public Attributes

- int32_t **width** = 0
- int32_t **height** = 0

8.57.1 Detailed Description

2D size representation.

The documentation for this struct was generated from the following file:

- [drawingTypes.h](#)

8.58 nsbaci::types::SymbolInfo Struct Reference

Information about a variable/symbol.

```
#include <compilerTypes.h>
```

Public Attributes

- [VarName](#) **name**
- [MemoryAddr](#) **address**
- `std::string` **type**
"int", "bool", "char", "void", etc.
- `bool` **isGlobal**

8.58.1 Detailed Description

Information about a variable/symbol.

The documentation for this struct was generated from the following file:

- [compilerTypes.h](#)

8.59 Text Struct Reference

[Text](#) to be drawn at a position.

```
#include <drawingTypes.h>
```

8.59.1 Detailed Description

[Text](#) to be drawn at a position.

The documentation for this struct was generated from the following file:

- [drawingTypes.h](#)

8.60 nsbaci::services::runtime::Thread Class Reference

Represents a thread in the runtime service.

```
#include <thread.h>
```

Public Member Functions

- nsbaci::types::ThreadID [getId](#) () const
Gets the thread ID.
- nsbaci::types::ThreadState [getState](#) () const
Gets the current state of the thread.
- void [setState](#) (nsbaci::types::ThreadState newState)
Sets the state of the thread.
- nsbaci::types::Priority [getPriority](#) () const
Gets the priority of the thread.
- void [setPriority](#) (nsbaci::types::Priority newPriority)
Sets the priority of the thread.
- void **push** (int32_t value)
Push a value onto the thread's stack.
- int32_t **pop** ()
Pop a value from the thread's stack.
- int32_t **top** () const
Peek at the top of the stack without removing.
- uint32_t **getPC** () const
Get the program counter.
- void **setPC** (uint32_t addr)
Set the program counter.
- void **advancePC** ()
Increment the program counter.
- uint32_t **getBP** () const
- void **setBP** (uint32_t addr)
- uint32_t **getSP** () const
- void **setSP** (uint32_t addr)
- void **pushReturnAddress** (uint32_t addr)
Push return address onto call stack.
- uint32_t [popReturnAddress](#) ()
Pop return address from call stack.
- bool **callStackEmpty** () const
Check if call stack is empty.
- void **pushFrame** (const std::vector< int32_t > &savedLocals)
Push a frame with saved local variable values.
- std::vector< int32_t > **popFrame** ()
Pop a frame and get saved local variable values.
- bool **frameStackEmpty** () const
Check if frame stack is empty.

8.60.1 Detailed Description

Represents a thread in the runtime service.

Each thread has its own stack, program counter, and execution state.

8.60.2 Member Function Documentation

8.60.2.1 getId()

```
ThreadID nsbaci::services::runtime::Thread::getId () const
```

Gets the thread ID.

Returns

The unique identifier of this thread.

8.60.2.2 getPriority()

```
Priority nsbaci::services::runtime::Thread::getPriority () const
```

Gets the priority of the thread.

Returns

The current Priority level.

8.60.2.3 getState()

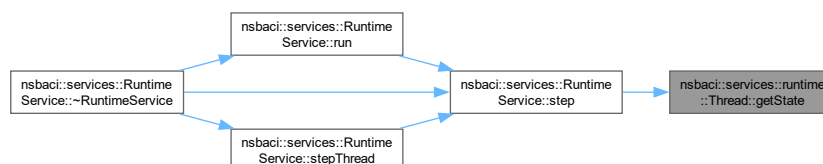
```
ThreadState nsbaci::services::runtime::Thread::getState () const
```

Gets the current state of the thread.

Returns

The current [ThreadState](#).

Here is the caller graph for this function:



8.60.2.4 popReturnAddress()

```
uint32_t nsbaci::services::runtime::Thread::popReturnAddress () [inline]
```

Pop return address from call stack.

Returns

The return address, or 0 if stack is empty.

Here is the caller graph for this function:



8.60.2.5 setPriority()

```
void nsbaci::services::runtime::Thread::setPriority (
    nsbaci::types::Priority newPriority)
```

Sets the priority of the thread.

Parameters

<i>newPriority</i>	The new priority level to set.
--------------------	--------------------------------

8.60.2.6 setState()

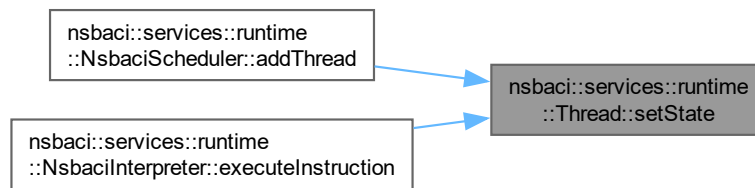
```
void nsbaci::services::runtime::Thread::setState (
    nsbaci::types::ThreadState newState)
```

Sets the state of the thread.

Parameters

<i>newState</i>	The new state to set.
-----------------	-----------------------

Here is the caller graph for this function:



The documentation for this class was generated from the following files:

- [thread.h](#)
- [thread.cpp](#)

8.61 nsbaci::ui::ThreadInfo Struct Reference

Information about a thread for display.

```
#include <runtimeView.h>
```

Public Attributes

- `nsbaci::types::ThreadID` **id**
- [nsbaci::types::ThreadState](#) **state**
- `size_t` **pc**
- `QString` **currentInstruction**

8.61.1 Detailed Description

Information about a thread for display.

The documentation for this struct was generated from the following file:

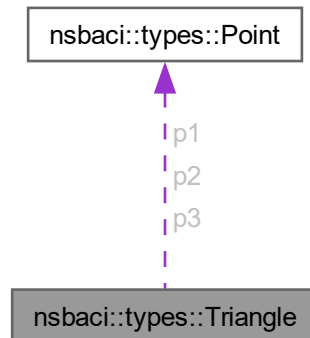
- [runtimeView.h](#)

8.62 nsbaci::types::Triangle Struct Reference

[Triangle](#) shape defined by three vertices.

```
#include <drawingTypes.h>
```

Collaboration diagram for nsbaci::types::Triangle:



Public Member Functions

- **Triangle** ([Point](#) a, [Point](#) b, [Point](#) c, bool fill=false)

Public Attributes

- [Point](#) p1
- [Point](#) p2
- [Point](#) p3
- bool **filled** = false

8.62.1 Detailed Description

[Triangle](#) shape defined by three vertices.

The documentation for this struct was generated from the following file:

- [drawingTypes.h](#)

8.63 nsbaci::UIError Struct Reference

UI-ready error representation for display in dialogs.

```
#include <uiError.h>
```

Static Public Member Functions

- static `std::vector< UIError > fromBackendErrors` (const `std::vector< Error > &errors`)
Converts backend errors to UI-ready errors.

Public Attributes

- `QString title`
- `QString body`
- `nsbaci::types::ErrSeverity severity`

8.63.1 Detailed Description

UI-ready error representation for display in dialogs.

Contains all the information needed to render an error dialog: title, body text, and severity (which maps to an icon).

8.63.2 Member Function Documentation

8.63.2.1 fromBackendErrors()

```
std::vector< UIError > nsbaci::UIError::fromBackendErrors (
    const std::vector< Error > & errors) [static]
```

Converts backend errors to UI-ready errors.

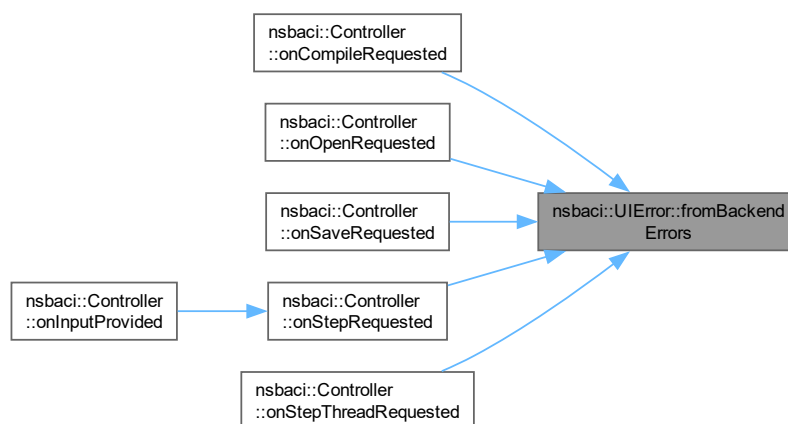
Parameters

<i>errors</i>	Vector of backend Error objects.
---------------	--

Returns

Vector of [UIError](#) objects ready for display.

Here is the caller graph for this function:



The documentation for this struct was generated from the following files:

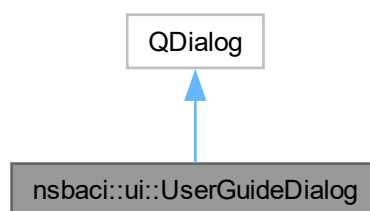
- [uiError.h](#)
- [uiError.cpp](#)

8.64 nsbaci::ui::UserGuideDialog Class Reference

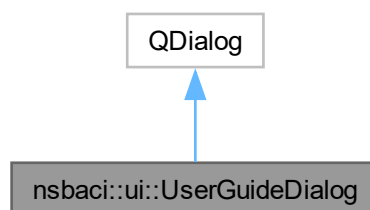
A dialog that provides an integrated user guide with code examples.

```
#include <userGuideDialog.h>
```

Inheritance diagram for nsbaci::ui::UserGuideDialog:



Collaboration diagram for nsbaci::ui::UserGuideDialog:



Signals

- void [loadExampleRequested](#) (const QString &code)
Emitted when the user clicks "Load in Editor".

Public Member Functions

- **UserGuideDialog** (QWidget *parent=nullptr)

8.64.1 Detailed Description

A dialog that provides an integrated user guide with code examples.

Features a side menu for browsing examples by category and a code viewer panel. Users can load examples directly into the editor or open the online documentation.

8.64.2 Member Function Documentation

8.64.2.1 loadExampleRequested

```
void nsbaci::ui::UserGuideDialog::loadExampleRequested (
    const QString & code) [signal]
```

Emitted when the user clicks "Load in Editor".

Parameters

<i>code</i>	The full source code of the selected example.
-------------	---

The documentation for this class was generated from the following files:

- [userGuideDialog.h](#)
- [userGuideDialog.cpp](#)

8.65 nsbaci::ui::VariableInfo Struct Reference

Information about a variable for display.

```
#include <runtimeView.h>
```

Public Attributes

- QString **name**
- QString **type**
- QString **value**
- size_t **address**

8.65.1 Detailed Description

Information about a variable for display.

The documentation for this struct was generated from the following file:

- [runtimeView.h](#)

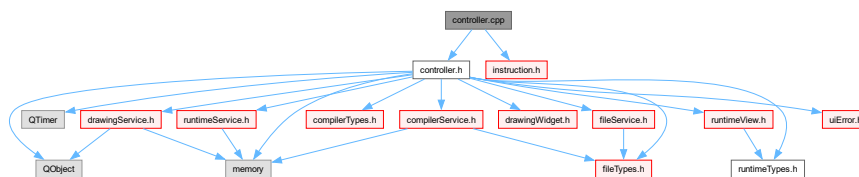
Chapter 9

File Documentation

9.1 controller.cpp File Reference

Implementation of the Controller class for nsbaci.

Include dependency graph for controller.cpp:



Namespaces

- namespace [nsbaci](#)

Root namespace for the nsbaci application.

9.1.1 Detailed Description

Implementation of the Controller class for nsbaci.

This file contains the implementation of all Controller methods that coordinate between the user interface and backend services. It handles the complete workflow from file operations through compilation to runtime execution and monitoring.

The implementation uses Qt's signal-slot mechanism for asynchronous communication with the UI, and a QTimer-based approach for continuous program execution that maintains UI responsiveness.

Author

Nicolás Serrano García

Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

- File operations (save/load source files)
- Compilation workflow (source code to p-code instructions)
- Runtime execution control (run, step, pause, reset)
- Thread scheduling and monitoring
- Variable state tracking and display updates
- Input/output handling between runtime and UI

Author

Nicolás Serrano García

Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

9.3 controller.h

[Go to the documentation of this file.](#)

```

00001
00023
00024 #ifndef NSBACI_CONTROLLER_H
00025 #define NSBACI_CONTROLLER_H
00026
00027 #include <QObject>
00028 #include <QTimer>
00029 #include <memory>
00030
00031 #include "compilerService.h"
00032 #include "compilerTypes.h"
00033 #include "drawingService.h"
00034 #include "drawingWidget.h"
00035 #include "fileService.h"
00036 #include "fileTypes.h"
00037 #include "runtimeService.h"
00038 #include "runtimeTypes.h"
00039 #include "runtimeView.h"
00040 #include "uiError.h"
00041
00049 namespace nsbaci {
00050
00075 class Controller : public QObject {
00076     Q_OBJECT
00077
00078     public:
00091     explicit Controller(nsbaci::services::FileService&& f,
00092                        nsbaci::services::CompilerService&& c,
00093                        nsbaci::services::RuntimeService&& r,
00094                        std::unique_ptr<nsbaci::services::DrawingService> d,
00095                        QObject* parent = nullptr);
00096
00102     ~Controller() = default;
00103
00104     signals:
00109     void saveFailed(std::vector<UIError> errors);
00110
00114     void saveSucceeded();
00115
00120     void loadFailed(std::vector<UIError> errors);
00121
00126     void loadSucceeded(const QString& contents);
00127
00132     void compileFailed(std::vector<UIError> errors);
00133
00140     void compileSucceeded();
00141
00146     void runStarted(const QString& programName);
00147

```

```

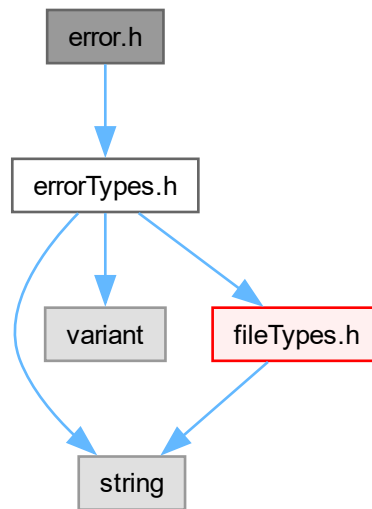
00154 void runtimeStateChanged(bool running, bool halted);
00155
00161 void threadsUpdated(const std::vector<nsbaci::ui::ThreadInfo>& threads);
00162
00167 void variablesUpdated(const std::vector<nsbaci::ui::VariableInfo>& variables);
00168
00173 void outputReceived(const QString& output);
00174
00179 void inputRequested(const QString& prompt);
00180
00181 public slots:
00187 void onSaveRequested(nsbaci::types::File file, nsbaci::types::Text contents);
00188
00193 void onOpenRequested(nsbaci::types::File file);
00194
00204 void onCompileRequested(nsbaci::types::Text contents);
00205
00214 void onRunRequested();
00215
00222 void onStepRequested();
00223
00228 void onStepThreadRequested(nsbaci::types::ThreadID threadId);
00229
00236 void onRunContinueRequested();
00237
00243 void onPauseRequested();
00244
00251 void onResetRequested();
00252
00258 void onStopRequested();
00259
00269 void onInputProvided(const QString& input);
00270
00271 private:
00278 void updateRuntimeDisplay();
00279
00284 std::vector<nsbaci::ui::ThreadInfo> gatherThreadInfo();
00285
00290 std::vector<nsbaci::ui::VariableInfo> gatherVariableInfo();
00291
00299 void runBatch();
00300
00301 nsbaci::services::FileService
00302     fileService;
00303 nsbaci::services::CompilerService
00304     compilerService;
00305 nsbaci::services::RuntimeService
00306     runtimeService;
00307 std::unique_ptr<nsbaci::services::DrawingService>
00308     drawingService;
00309 nsbaci::ui::DrawingWidget* drawingWidget =
00310     nullptr;
00311
00312 QString currentProgramName;
00313 bool programLoaded = false;
00314 bool isRunning = false;
00315 bool wasRunningBeforeInput =
00316     false;
00317 QTimer* runTimer = nullptr;
00318 };
00319
00320 } // namespace nsbaci
00321
00322 #endif // NSBACI_CONTROLLER_H

```

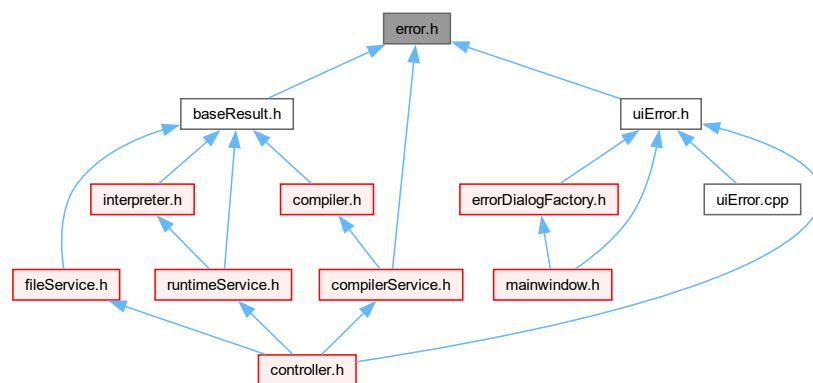
9.4 error.h File Reference

Error class declaration for nsbaci.

Include dependency graph for error.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [nsbaci::Error](#)
Represents an error with a message and optional code.

Namespaces

- namespace [nsbaci](#)
Root namespace for the nsbaci application.

9.4.1 Detailed Description

Error class declaration for nsbaci.

This module defines the error handling structures and classes used throughout the application for consistent error reporting.

Author

Nicolás Serrano García

Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

9.5 error.h

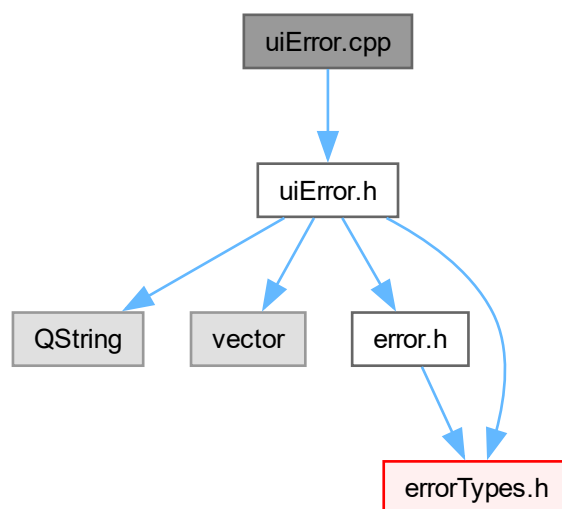
[Go to the documentation of this file.](#)

```
00001
00012
00013 #ifndef NSBACI_ERROR_H
00014 #define NSBACI_ERROR_H
00015
00016 #include "errorTypes.h"
00017
00022 namespace nsbaci {
00023
00028 class Error {
00029 public:
00030     Error() = default;
00031     ~Error() = default;
00032     nsbaci::types::ErrorBase basic;
00033     nsbaci::types::ErrorPayload payload;
00034 };
00035
00036 } // namespace nsbaci
00037
00038 #endif // NSBACI_ERROR_H
```

9.6 uiError.cpp File Reference

Implementation of UIError utilities.

Include dependency graph for uiError.cpp:



Namespaces

- namespace [nsbaci](#)
Root namespace for the nsbaci application.

9.6.1 Detailed Description

Implementation of UIError utilities.

Author

Nicolás Serrano García

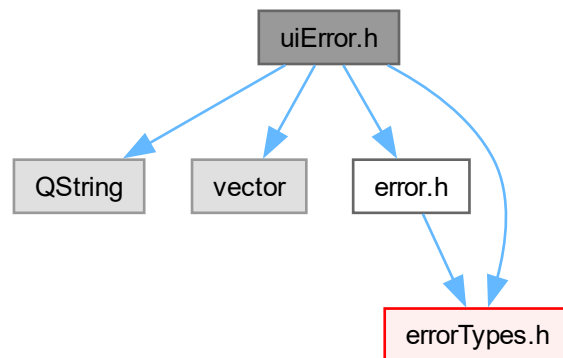
Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

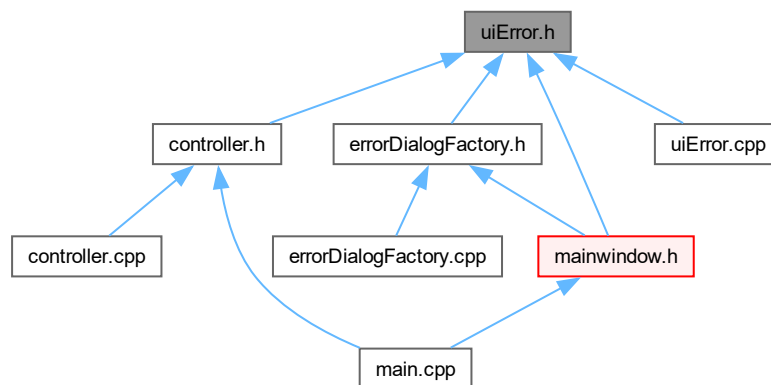
9.7 uiError.h File Reference

UI Error type definitions for nsbaci.

Include dependency graph for `uiError.h`:



This graph shows which files directly or indirectly include this file:



Classes

- struct `nsbaci::UIError`
UI-ready error representation for display in dialogs.

Namespaces

- namespace `nsbaci`
Root namespace for the nsbaci application.

9.7.1 Detailed Description

UI Error type definitions for nsbaci.

This header provides types used to communicate between the controller and the view layer, particularly for error display.

Author

Nicolás Serrano García

Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

9.8 uiError.h

[Go to the documentation of this file.](#)

```
00001
00002
00003 #ifndef NSBACI_UIERROR_H
00004 #define NSBACI_UIERROR_H
00005
00006 #include <QString>
00007 #include <vector>
00008
00009 #include "error.h"
00010 #include "errorTypes.h"
00011
00012 namespace nsbaci {
00013
00014 struct UIError {
00015     QString title; // Dialog title (e.g., "Save Error", "Load Error")
00016     QString body; // Main message body with details
00017     nsbaci::types::ErrSeverity severity; // Maps to icon (Warning, Error, Fatal)
00018
00019     static std::vector<UIError> fromBackendErrors(
00020         const std::vector<Error>& errors);
00021
00022 private:
00023     static QString reasonFromType(nsbaci::types::ErrType type);
00024
00025     static QString titleFromPayload(const nsbaci::types::ErrorPayload& payload);
00026
00027     static QString contextFromPayload(const nsbaci::types::ErrorPayload& payload);
00028 };
00029
00030 } // namespace nsbaci
00031
00032 #endif // NSBACI_UIERROR_H
```

9.9 main.cpp File Reference

Application entry point for nsbaci.

Include dependency graph for main.cpp:



Functions

- void **setupViewController** ([nsbaci::Controller](#) *c, [MainWindow](#) *w)
- int **main** (int argc, char *argv[])

9.9.1 Detailed Description

Application entry point for nsbaci.

This file contains the main function that initializes the Qt application and displays the main window.

Author

Nicolás Serrano García

Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

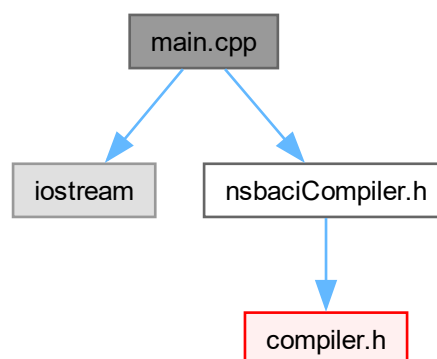
See also

[MainWindow](#)

9.10 services/compilerService/compiler/nsbaci/tools/main.cpp File Reference

CLI tool for testing the nsbaci compiler.

Include dependency graph for services/compilerService/compiler/nsbaci/tools/main.cpp:



Functions

- `int main ()`

9.10.1 Detailed Description

CLI tool for testing the nsbaci compiler.

This is a standalone command-line tool for testing the compiler independently of the main application.

Author

Nicolás Serrano García

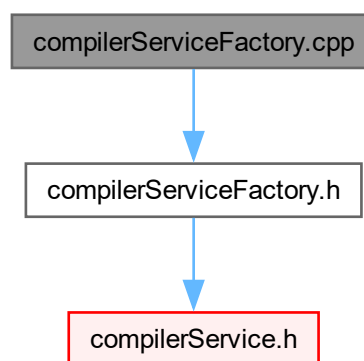
Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

9.11 compilerServiceFactory.cpp File Reference

Implementation unit for the CompilerServiceFactory.

Include dependency graph for compilerServiceFactory.cpp:



Namespaces

- namespace `nsbaci`
Root namespace for the nsbaci application.
- namespace `nsbaci::factories`
Factories namespace for nsbaci.

9.11.1 Detailed Description

Implementation unit for the CompilerServiceFactory.

Author

Nicolás Serrano García

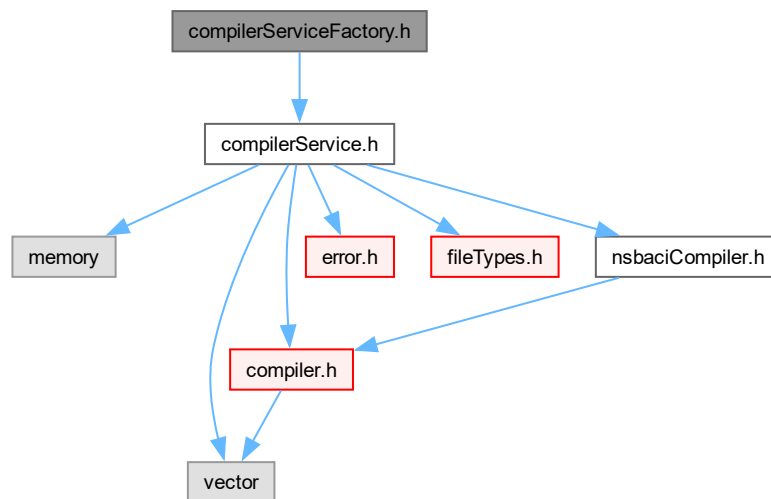
Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

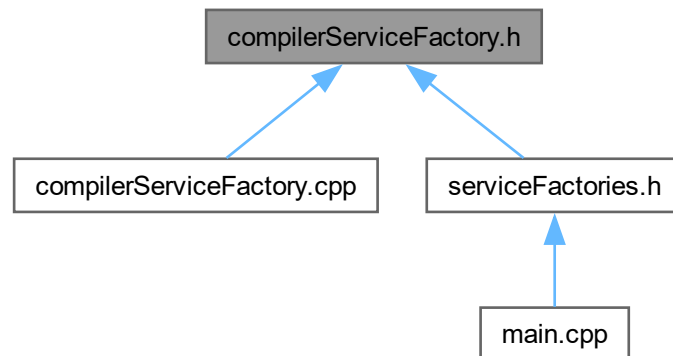
9.12 compilerServiceFactory.h File Reference

CompilerServiceFactory class declaration for nsbaci.

Include dependency graph for compilerServiceFactory.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct [nsbaci::factories::NsbaciCompiler](#)
- class [nsbaci::factories::CompilerServiceFactory](#)
Factory for creating CompilerService instances.

Namespaces

- namespace [nsbaci::factories](#)
Factories namespace for nsbaci.
- namespace [nsbaci](#)
Root namespace for the nsbaci application.

Variables

- constexpr [NsbaciCompiler](#) [nsbaci::factories::nsbaciCompiler](#) {}

9.12.1 Detailed Description

CompilerServiceFactory class declaration for nsbaci.

This factory is responsible for creating CompilerService instances based on configuration or runtime parameters.

Author

Nicolás Serrano García

Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

9.13 compilerServiceFactory.h

[Go to the documentation of this file.](#)

```

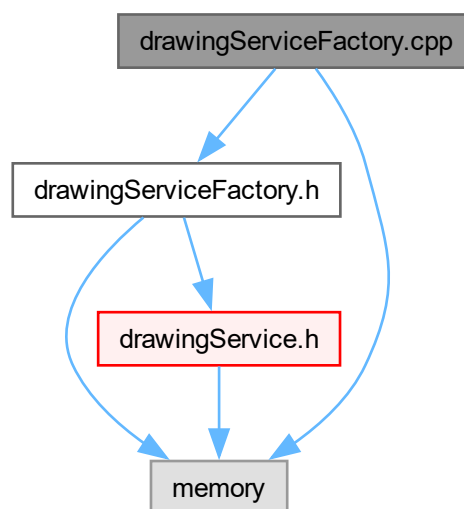
00001
00002
00003 #ifndef NSBACI_COMPILERSERVICEFACTORY_H
00004 #define NSBACI_COMPILERSERVICEFACTORY_H
00005
00006 #include "compilerService.h"
00007
00008 namespace nsbaci::factories {
00009
00010 struct NsbaciCompiler {
00011     explicit NsbaciCompiler() = default;
00012 };
00013
00014 // Tag used to generate a service with a nsbaci compiler
00015 constexpr inline NsbaciCompiler nsbaciCompiler{};
00016
00017 class CompilerServiceFactory {
00018 private:
00019     CompilerServiceFactory() = default;
00020
00021 public:
00022     static nsbaci::services::CompilerService createService(NsbaciCompiler);
00023     // static nsbaci::services::CompilerService createService(OtherCompilerOrTest
00024     // t);
00025     ~CompilerServiceFactory() = default;
00026 };
00027
00028 } // namespace nsbaci::factories
00029 #endif // NSBACI_COMPILERSERVICEFACTORY_H

```

9.14 drawingServiceFactory.cpp File Reference

Implementation unit for the DrawingServiceFactory.

Include dependency graph for drawingServiceFactory.cpp:



Namespaces

- namespace [nsbaci](#)
Root namespace for the nsbaci application.
- namespace [nsbaci::factories](#)
Factories namespace for nsbaci.

9.14.1 Detailed Description

Implementation unit for the DrawingServiceFactory.

Author

Nicolás Serrano García

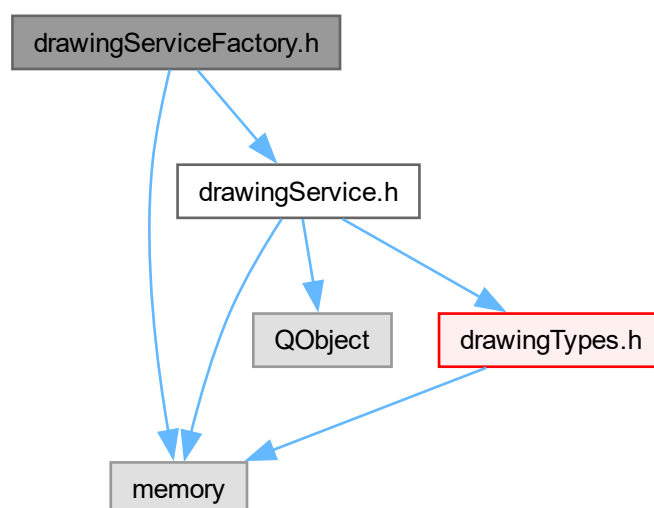
Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

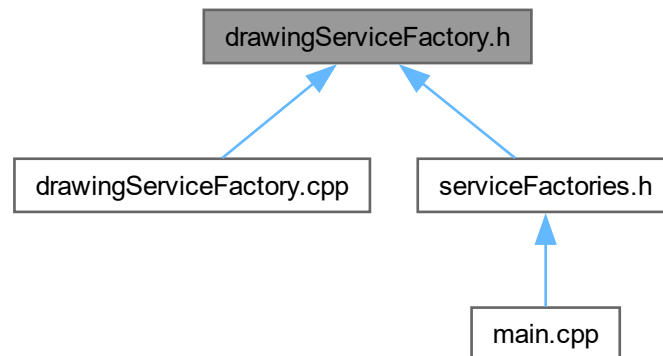
9.15 drawingServiceFactory.h File Reference

DrawingServiceFactory class declaration for nsbaci.

Include dependency graph for drawingServiceFactory.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct [nsbaci::factories::DefaultDrawingBackend](#)
- class [nsbaci::factories::DrawingServiceFactory](#)
Factory for creating DrawingService instances.

Namespaces

- namespace [nsbaci::factories](#)
Factories namespace for nsbaci.
- namespace [nsbaci](#)
Root namespace for the nsbaci application.

Variables

- constexpr [DefaultDrawingBackend](#) [nsbaci::factories::defaultDrawingBackend](#) {}

9.15.1 Detailed Description

DrawingServiceFactory class declaration for nsbaci.

This factory is responsible for creating DrawingService instances based on configuration or runtime parameters.

Author

Nicolás Serrano García

Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

9.16 drawingServiceFactory.h

[Go to the documentation of this file.](#)

```

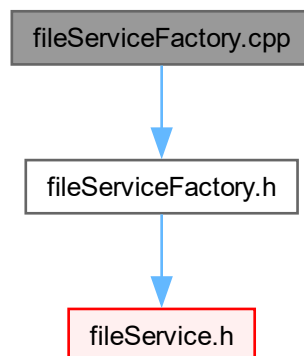
00001
00012
00013 #ifndef NSBACI_DRAWINGSERVICEFACTORY_H
00014 #define NSBACI_DRAWINGSERVICEFACTORY_H
00015
00016 #include <memory>
00017
00018 #include "drawingService.h"
00019
00024 namespace nsbaci::factories {
00025
00026 struct DefaultDrawingBackend {
00027     explicit DefaultDrawingBackend() = default;
00028 };
00029
00030 // tag used to generate a service with the default drawing backend
00031 constexpr inline DefaultDrawingBackend defaultDrawingBackend{};
00032
00037 class DrawingServiceFactory {
00038 private:
00039     DrawingServiceFactory() = default;
00040
00041 public:
00042     static std::unique_ptr<nsbaci::services::DrawingService> createService(
00043         DefaultDrawingBackend);
00044     // static std::unique_ptr<nsbaci::services::DrawingService>
00045     // createService(OtherBackendOrTest t);
00046     ~DrawingServiceFactory() = default;
00047 };
00048
00049 } // namespace nsbaci::factories
00050
00051 #endif // NSBACI_DRAWINGSERVICEFACTORY_H

```

9.17 fileServiceFactory.cpp File Reference

Implementation unit for the FileServiceFactory.

Include dependency graph for fileServiceFactory.cpp:



Namespaces

- namespace [nsbaci](#)
Root namespace for the nsbaci application.
- namespace [nsbaci::factories](#)
Factories namespace for nsbaci.

9.17.1 Detailed Description

Implementation unit for the FileServiceFactory.

Author

Nicolás Serrano García

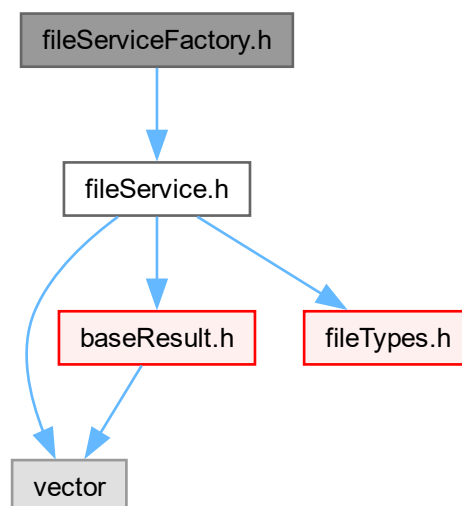
Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

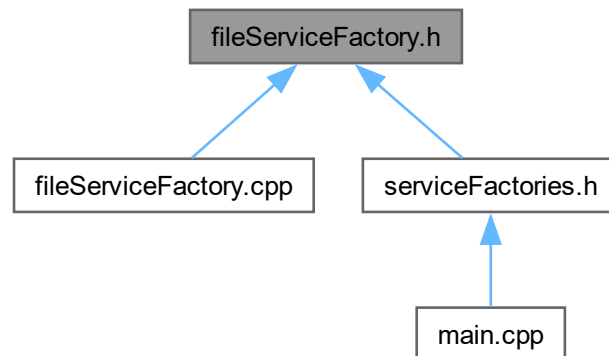
9.18 fileServiceFactory.h File Reference

FileServiceFactory class declaration for nsbaci.

Include dependency graph for fileServiceFactory.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct [nsbaci::factories::DefaultFileSystem](#)
- class [nsbaci::factories::FileServiceFactory](#)
Factory for creating FileService instances.

Namespaces

- namespace [nsbaci::factories](#)
Factories namespace for nsbaci.
- namespace [nsbaci](#)
Root namespace for the nsbaci application.

Variables

- constexpr [DefaultFileSystem](#) [nsbaci::factories::defaultFileSystem](#) {}

9.18.1 Detailed Description

FileServiceFactory class declaration for nsbaci.

This factory is responsible for creating FileService instances based on configuration or runtime parameters.

Author

Nicolás Serrano García

Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

9.19 fileServiceFactory.h

[Go to the documentation of this file.](#)

```

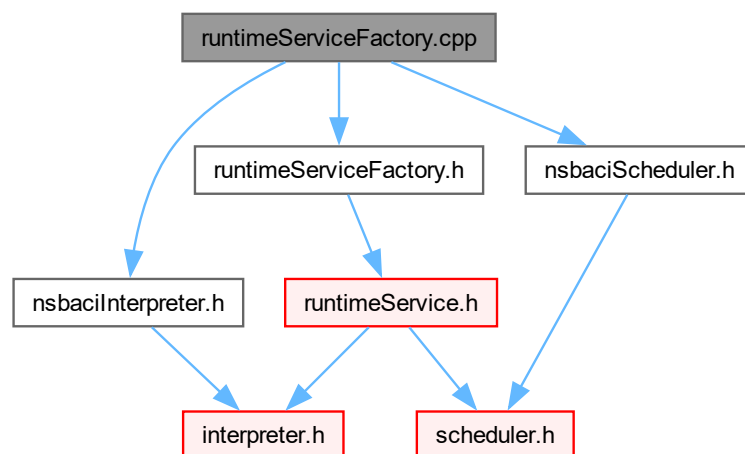
00001
00012
00013 #ifndef NSBACI_FILESERVICEFACTORY_H
00014 #define NSBACI_FILESERVICEFACTORY_H
00015
00016 #include "fileService.h"
00017
00022 namespace nsbaci::factories {
00023
00024 struct DefaultFileSystem {
00025     explicit DefaultFileSystem() = default;
00026 };
00027
00028 // Tag used to generate a service with the default file system
00029 constexpr inline DefaultFileSystem defaultFileSystem{};
00030
00035 class FileServiceFactory {
00036 private:
00037     FileServiceFactory() = default;
00038
00039 public:
00040     static nsbaci::services::FileService createService(DefaultFileSystem);
00041     // static nsbaci::services::FileService createService(OtherFileSystemOrTest
00042     // t);
00043     ~FileServiceFactory() = default;
00044 };
00045
00046 } // namespace nsbaci::factories
00047
00048 #endif // NSBACI_FILESERVICEFACTORY_H

```

9.20 runtimeServiceFactory.cpp File Reference

Implementation unit for the RuntimeServiceFactory.

Include dependency graph for runtimeServiceFactory.cpp:



Namespaces

- namespace [nsbaci](#)
Root namespace for the nsbaci application.
- namespace [nsbaci::factories](#)
Factories namespace for nsbaci.

9.20.1 Detailed Description

Implementation unit for the RuntimeServiceFactory.

Author

Nicolás Serrano García

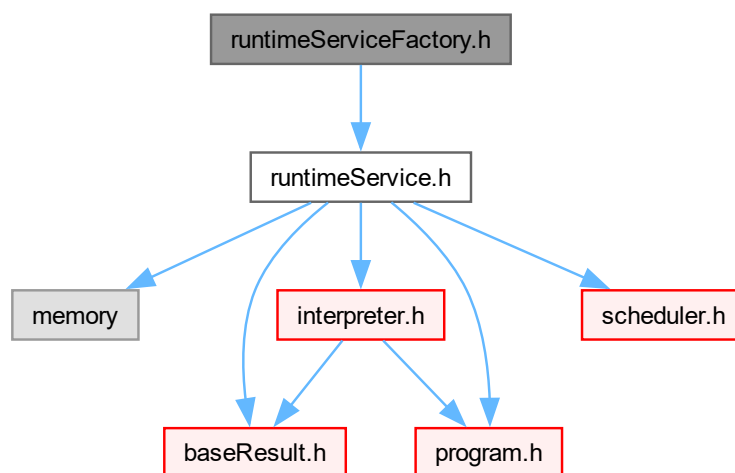
Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

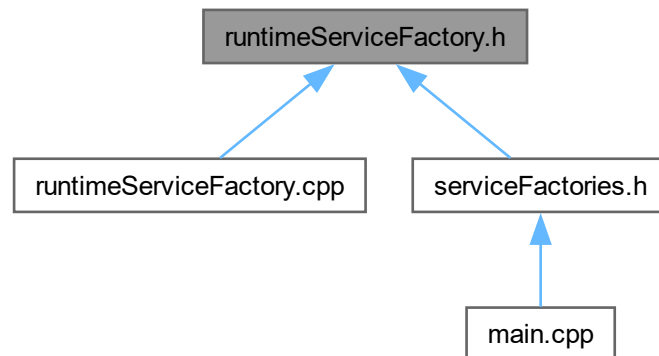
9.21 runtimeServiceFactory.h File Reference

RuntimeServiceFactory class declaration for nsbaci.

Include dependency graph for runtimeServiceFactory.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct `nsbaci::factories::NsbaciRuntime`
- class `nsbaci::factories::RuntimeServiceFactory`
Factory for creating RuntimeService instances.

Namespaces

- namespace `nsbaci::factories`
Factories namespace for nsbaci.
- namespace `nsbaci`
Root namespace for the nsbaci application.

Variables

- constexpr `NsbaciRuntime nsbaci::factories::nsbaciRuntime {}`

9.21.1 Detailed Description

RuntimeServiceFactory class declaration for nsbaci.

This factory is responsible for creating RuntimeService instances based on configuration or runtime parameters.

Author

Nicolás Serrano García

Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

9.22 runtimeServiceFactory.h

[Go to the documentation of this file.](#)

```

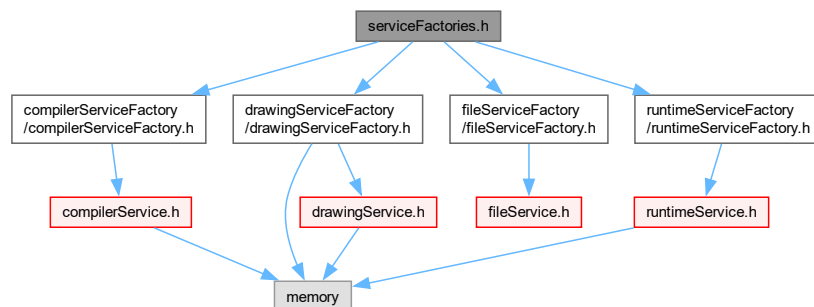
00001
00012
00013 #ifndef NSBACI_RUNTIMESERVICEFACTORY_H
00014 #define NSBACI_RUNTIMESERVICEFACTORY_H
00015
00016 #include "runtimeService.h"
00017
00022 namespace nsbaci::factories {
00023
00024 struct NsbaciRuntime {
00025     explicit NsbaciRuntime() = default;
00026 };
00027
00028 // Tag used to generate a service with a nsbaci runtime
00029 constexpr inline NsbaciRuntime nsbaciRuntime{};
00030
00035 class RuntimeServiceFactory {
00036 private:
00037     RuntimeServiceFactory() = default;
00038
00039 public:
00040     static nsbaci::services::RuntimeService createService(NsbaciRuntime t);
00041     // static nsbaci::services::RuntimeService createService(OtherRuntimeOrTest
00042     // t);
00043     ~RuntimeServiceFactory() = default;
00044 };
00045
00046 } // namespace nsbaci::factories
00047
00048 #endif // NSBACI_RUNTIMESERVICEFACTORY_H

```

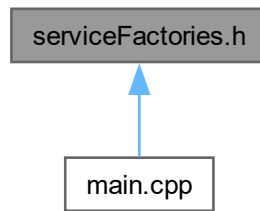
9.23 serviceFactories.h File Reference

Aggregate header for all service factories.

Include dependency graph for serviceFactories.h:



This graph shows which files directly or indirectly include this file:



9.23.1 Detailed Description

Aggregate header for all service factories.

This header includes all service factory headers for convenient access.

Author

Nicolás Serrano García

Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

9.24 serviceFactories.h

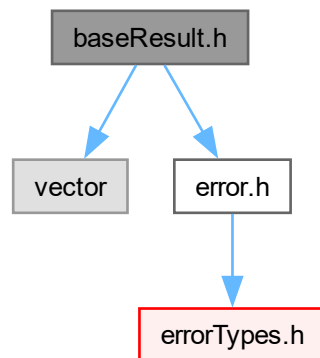
[Go to the documentation of this file.](#)

```
00001
00011
00012 #ifndef NSBACI_SERVICEFACTORIES_H
00013 #define NSBACI_SERVICEFACTORIES_H
00014
00015 #include "compilerServiceFactory/compilerServiceFactory.h"
00016 #include "drawingServiceFactory/drawingServiceFactory.h"
00017 #include "fileServiceFactory/fileServiceFactory.h"
00018 #include "runtimeServiceFactory/runtimeServiceFactory.h"
00019
00020 #endif // NSBACI_SERVICEFACTORIES_H
```

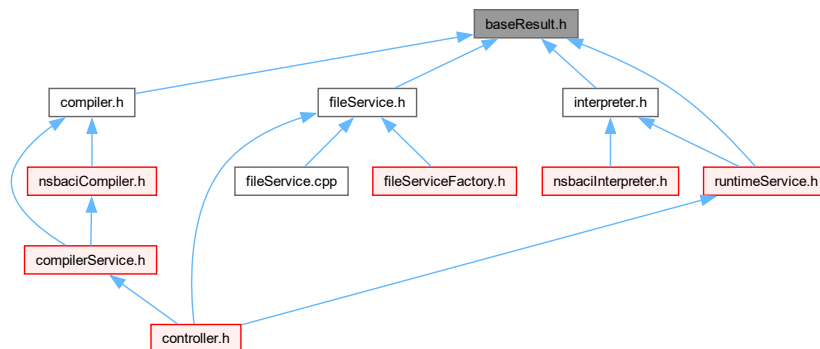
9.25 baseResult.h File Reference

Base result class declaration for nsbaci services.

Include dependency graph for baseResult.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct `nsbaci::BaseResult`
Base result structure for all service operations.

Namespaces

- namespace `nsbaci`
Root namespace for the nsbaci application.

9.25.1 Detailed Description

Base result class declaration for nsbaci services.

This module defines the `BaseResult` struct that serves as the foundational result type for all service operations in the nsbaci application. It implements a simple success/failure pattern with associated error information, providing a consistent interface for error handling across all services.

The `BaseResult` pattern ensures that:

- All service operations return a result that can be checked for success
- Failed operations always provide descriptive error information
- Results can be efficiently moved without copying error data

Author

Nicolás Serrano García

Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

9.26 baseResult.h

[Go to the documentation of this file.](#)

```

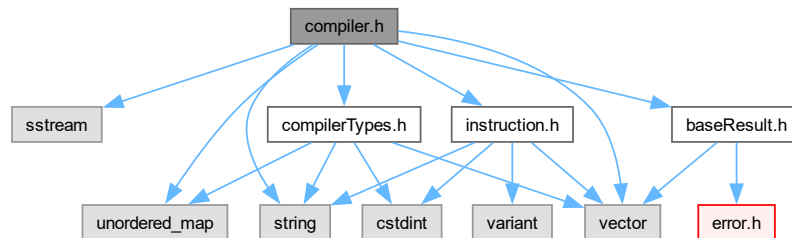
00001
00020
00021 #ifndef NSBACI_SERVICES_BASERESULT_H
00022 #define NSBACI_SERVICES_BASERESULT_H
00023
00024 #include <vector>
00025
00026 #include "error.h"
00027
00032 namespace nsbaci {
00033
00056 struct BaseResult {
00060     BaseResult() : ok(true) {}
00061
00069     explicit BaseResult(std::vector<nsbaci::Error> errs)
00070         : ok(errs.empty()), errors(std::move(errs)) {}
00071
00076     explicit BaseResult(nsbaci::Error error)
00077         : ok(false), errors({std::move(error)}) {}
00078
00079     BaseResult(BaseResult&&) noexcept = default;
00080     BaseResult& operator=(BaseResult&&) noexcept = default;
00081
00082     BaseResult(const BaseResult&) = default;
00083     BaseResult& operator=(const BaseResult&) = default;
00084
00085     bool ok;
00086     std::vector<nsbaci::Error>
00087         errors;
00088 };
00089
00090 } // namespace nsbaci
00091
00092 #endif // NSBACI_SERVICES_BASERESULT_H

```

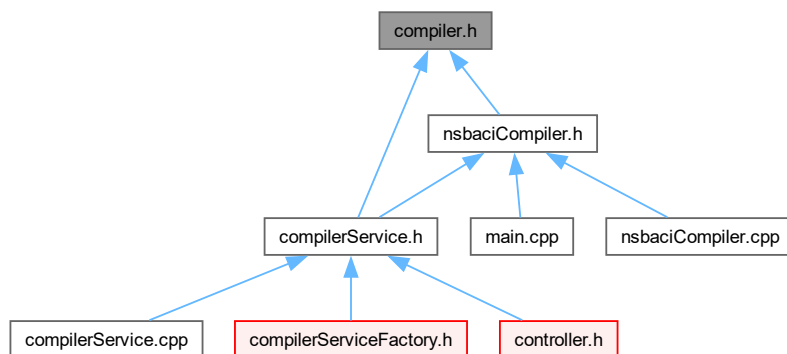
9.27 compiler.h File Reference

Abstract Compiler class declaration for nsbaci.

Include dependency graph for compiler.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct `nsbaci::compiler::CompilerResult`
Result of a compilation operation.
- class `nsbaci::compiler::Compiler`
Abstract base class for all compilers.

Namespaces

- namespace `nsbaci::compiler`
Compiler namespace containing all compilation-related stuff.
- namespace `nsbaci`
Root namespace for the nsbaci application.

9.27.1 Detailed Description

Abstract Compiler class declaration for nsbaci.

This module defines the abstract Compiler interface that all compiler implementations must follow. The Compiler is responsible for transforming nsbaci source code into an executable instruction stream (p-code) that can be run by the virtual machine interpreter.

The design follows the Strategy pattern, allowing different compiler implementations to be plugged in. Currently, NsbaciCompiler is the primary implementation using flex/bison for lexical analysis and parsing.

Author

Nicolás Serrano García

Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

9.28 compiler.h

[Go to the documentation of this file.](#)

```

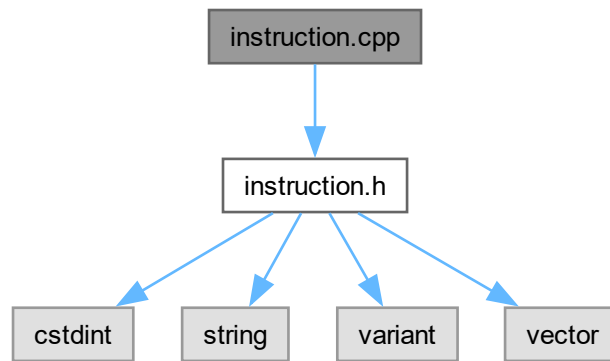
00001
00018
00019 #ifndef NSBACI_COMPILER_COMPILER_H
00020 #define NSBACI_COMPILER_COMPILER_H
00021
00022 #include <sstream>
00023 #include <string>
00024 #include <unordered_map>
00025 #include <vector>
00026
00027 #include "baseResult.h"
00028 #include "compilerTypes.h"
00029 #include "instruction.h"
00030
00035 namespace nsbaci::compiler {
00036
00047 struct CompilerResult : nsbaci::BaseResult {
00051     CompilerResult() : BaseResult() {}
00052
00057     explicit CompilerResult(std::vector<nsbaci::Error> errs)
00058         : BaseResult(std::move(errs)) {}
00059
00064     explicit CompilerResult(nsbaci::Error error) : BaseResult(std::move(error)) {}
00065
00066     CompilerResult(CompilerResult&&) noexcept = default;
00067     CompilerResult& operator=(CompilerResult&&) noexcept = default;
00068
00069     CompilerResult(const CompilerResult&) = default;
00070     CompilerResult& operator=(const CompilerResult&) = default;
00071
00072     InstructionStream instructions;
00073     nsbaci::types::SymbolTable symbols;
00074 };
00075
00092 class Compiler {
00093 public:
00097     Compiler() = default;
00098
00102     virtual ~Compiler() = default;
00103
00114     virtual CompilerResult compile(const std::string& source) = 0;
00115
00125     virtual CompilerResult compile(std::istream& input) = 0;
00126 };
00127
00128 } // namespace nsbaci::compiler
00129
00130 #endif // NSBACI_COMPILER_COMPILER_H

```

9.29 instruction.cpp File Reference

Instruction implementation for nsbaci compiler.

Include dependency graph for instruction.cpp:



Namespaces

- namespace `nsbaci`
Root namespace for the nsbaci application.
- namespace `nsbaci::compiler`
Compiler namespace containing all compilation-related stuff.

Functions

- `const char * nsbaci::compiler::opcodeName (Opcode op)`
Get the string name of an opcode (for debugging/display).

9.29.1 Detailed Description

Instruction implementation for nsbaci compiler.

Author

Nicolás Serrano García

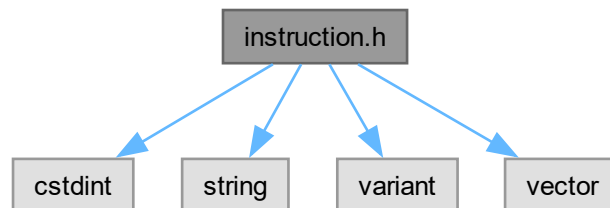
Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

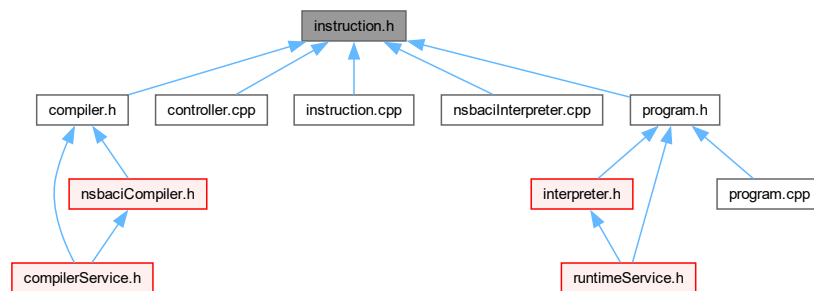
9.30 instruction.h File Reference

Instruction definitions for nsbaci compiler.

Include dependency graph for instruction.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct [nsbaci::compiler::Instruction](#)
Represents a single instruction in the virtual machine.

Namespaces

- namespace [nsbaci::compiler](#)
Compiler namespace containing all compilation-related stuff.
- namespace [nsbaci](#)
Root namespace for the nsbaci application.

Typedefs

- using `nsbaci::compiler::Operand`
Operand types that an instruction can have.
- using `nsbaci::compiler::InstructionStream` = `std::vector<Instruction>`
Vector of instructions representing a compiled program.

Enumerations

- enum class `nsbaci::compiler::Opcode` : `uint8_t` {
`LoadValue` , `LoadAddress` , `LoadIndirect` , `StoreIndirect` ,
`LoadBlock` , `Store` , `StoreKeep` , `PushLiteral` ,
`Swap` , `RotateDown3` , `Index` , `CopyBlock` ,
`ValueAt` , `MarkStack` , `UpdateDisplay` , `Add` ,
`Sub` , `Mult` , `Div` , `Mod` ,
`Negate` , `Complement` , `And` , `Or` ,
`TestEQ` , `TestNE` , `TestLT` , `TestLE` ,
`TestGT` , `TestGE` , `TestEqualKeep` , `Jump` ,
`JumpZero` , `Call` , `ShortCall` , `ShortReturn` ,
`ExitProc` , `ExitFunction` , `EnterFrame` , `LeaveFrame` ,
`Halt` , `BeginFor` , `EndFor` , `Cobegin` ,
`Coend` , `Create` , `ThreadEnd` , `Suspend` ,
`Revive` , `WhichProc` , `Wait` , `Signal` ,
`StoreSemaphore` , `EnterMonitor` , `ExitMonitor` , `CallMonitorInit` ,
`ReturnMonitorInit` , `WaitCondition` , `SignalCondition` , `Empty` ,
`Read` , `Readln` , `Write` , `Writeln` ,
`WriteString` , `WriteRawString` , `EolEof` , `Sprintf` ,
`Sscanf` , `CopyString` , `CopyRawString` , `ConcatString` ,
`ConcatRawString` , `CompareString` , `CompareRawString` , `LengthString` ,
`DrawClear` , `DrawRefresh` , `DrawSetColor` , `DrawSetColorAlpha` ,
`DrawSetLineWidth` , `DrawSetPosition` , `DrawCircle` , `DrawRectangle` ,
`DrawTriangle` , `DrawLine` , `DrawEllipse` , `DrawPixel` ,
`DrawText` , `FillCircle` , `FillRectangle` , `FillTriangle` ,
`FillEllipse` , `MoveTo` , `MoveBy` , `ChangeColor` ,
`MakeVisible` , `Remove` , `Random` , `Test` ,
`_Count` }
- Opcodes for the BACI virtual machine instruction set.*

Functions

- `const char * nsbaci::compiler::opcodeName (Opcode op)`
Get the string name of an opcode (for debugging/display).

9.30.1 Detailed Description

Instruction definitions for nsbaci compiler.

This module defines the instruction set for the BACI virtual machine, including opcodes and instruction representation.

Author

Nicolás Serrano García

Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

9.31 instruction.h

[Go to the documentation of this file.](#)

```

00001
00012
00013 #ifndef NSBACI_COMPILER_INSTRUCTION_H
00014 #define NSBACI_COMPILER_INSTRUCTION_H
00015
00016 #include <stdint>
00017 #include <string>
00018 #include <variant>
00019 #include <vector>
00020
00025 namespace nsbaci::compiler {
00026
00031 enum class Opcode : uint8_t {
00032     // ===== Stack/Memory Operations =====
00033     LoadValue,      // Load value from address onto stack
00034     LoadAddress,    // Load address onto stack
00035     LoadIndirect,    // Load value from address pointed to by top of stack
00036     StoreIndirect,   // Store value to address on stack (pops addr, pops value)
00037     LoadBlock,      // Load block of memory onto stack
00038     Store,           // Store top of stack to address
00039     StoreKeep,       // Store and keep value on stack
00040     PushLiteral,     // Push literal value onto stack
00041     Swap,            // Swap top two stack elements
00042     RotateDown3,     // Rotate top 3: [a,b,c] -> [b,c,a]
00043     Index,           // Array indexing
00044     CopyBlock,       // Copy block of memory
00045     ValueAt,         // Get value at address on stack
00046     MarkStack,       // Mark stack for procedure call
00047     UpdateDisplay,   // Update display register
00048
00049     // ===== Arithmetic Operations =====
00050     Add,             // Addition
00051     Sub,             // Subtraction
00052     Mult,            // Multiplication
00053     Div,             // Integer division
00054     Mod,             // Modulo
00055     Negate,          // Unary negation
00056     Complement,     // Bitwise complement
00057
00058     // ===== Logical Operations =====
00059     And,             // Logical AND
00060     Or,              // Logical OR
00061
00062     // ===== Comparison Operations =====
00063     TestEQ,          // Test equal
00064     TestNE,          // Test not equal
00065     TestLT,          // Test less than
00066     TestLE,          // Test less or equal
00067     TestGT,          // Test greater than
00068     TestGE,          // Test greater or equal
00069     TestEqualKeep,   // Test equal, keep operands
00070
00071     // ===== Control Flow =====
00072     Jump,            // Unconditional jump
00073     JumpZero,        // Jump if top of stack is zero
00074     Call,            // Call procedure
00075     ShortCall,       // Short call (no display update)
00076     ShortReturn,     // Short return
00077     ExitProc,        // Exit procedure
00078     ExitFunction,    // Exit function (with return value)
00079     EnterFrame,      // Enter function frame (save locals) - operand1 = start addr,
00080                     // operand2 = count
00081     LeaveFrame,      // Leave function frame (restore locals) - operand1 = start addr,
00082                     // operand2 = count
00083     Halt,            // Halt execution
00084
00085     // ===== Loop Control =====
00086     BeginFor,        // Begin for loop
00087     EndFor,          // End for loop
00088
00089     // ===== Concurrency - Process =====
00090     Cobegin,         // Begin concurrent block
00091     Coend,           // End concurrent block (operand1 = expected thread count)
00092     Create,          // Create new process (operand1 = start PC)
00093     ThreadEnd,       // Terminate current thread
00094     Suspend,         // Suspend current process
00095     Revive,          // Revive suspended process
00096     WhichProc,       // Get current process ID
00097
00098     // ===== Concurrency - Semaphores =====
00099     Wait,            // Wait on semaphore (P operation)
00100     Signal,          // Signal semaphore (V operation)

```



```

00101 StoreSemaphore, // Initialize semaphore
00102
00103 // ===== Concurrency - Monitors =====
00104 EnterMonitor, // Enter monitor
00105 ExitMonitor, // Exit monitor
00106 CallMonitorInit, // Call monitor initialization
00107 ReturnMonitorInit, // Return from monitor init
00108 WaitCondition, // Wait on condition variable
00109 SignalCondition, // Signal condition variable
00110 Empty, // Check if condition queue is empty
00111
00112 // ===== I/O Operations =====
00113 Read, // Read integer
00114 Readln, // Read line
00115 Write, // Write value
00116 Writeln, // Write newline
00117 WriteString, // Write string
00118 WriteRawString, // Write raw string literal
00119 EolEof, // Check end of line/file
00120 Sprintf, // Format string
00121 Sscanf, // Scan string
00122
00123 // ===== String Operations =====
00124 CopyString, // Copy string
00125 CopyRawString, // Copy raw string
00126 ConcatString, // Concatenate strings
00127 ConcatRawString, // Concatenate raw string
00128 CompareString, // Compare strings
00129 CompareRawString, // Compare raw strings
00130 LengthString, // Get string length
00131
00132 // ===== Graphics/Drawing Operations =====
00133 // Canvas operations
00134 DrawClear, // Clear the canvas (optional: r, g, b on stack)
00135 DrawRefresh, // Force canvas refresh
00136
00137 // Color/state operations
00138 DrawSetColor, // Set drawing color (r, g, b on stack)
00139 DrawSetColorAlpha, // Set drawing color with alpha (r, g, b, a on stack)
00140 DrawSetLineWidth, // Set line width (width on stack)
00141 DrawSetPosition, // Set current position (x, y on stack)
00142
00143 // Shape drawing operations (outline only)
00144 DrawCircle, // Draw circle outline (x, y, radius on stack)
00145 DrawRectangle, // Draw rectangle outline (x, y, width, height on stack)
00146 DrawTriangle, // Draw triangle outline (x1, y1, x2, y2, x3, y3 on stack)
00147 DrawLine, // Draw line (x1, y1, x2, y2 on stack)
00148 DrawEllipse, // Draw ellipse outline (x, y, radiusX, radiusY on stack)
00149 DrawPixel, // Draw pixel (x, y on stack)
00150 DrawText, // Draw text (x, y, fontSize on stack, string in operand)
00151
00152 // Shape fill operations (filled shapes)
00153 FillCircle, // Fill circle (x, y, radius on stack)
00154 FillRectangle, // Fill rectangle (x, y, width, height on stack)
00155 FillTriangle, // Fill triangle (x1, y1, x2, y2, x3, y3 on stack)
00156 FillEllipse, // Fill ellipse (x, y, radiusX, radiusY on stack)
00157
00158 // Legacy graphics (keeping for compatibility)
00159 MoveTo, // Move to absolute position
00160 MoveBy, // Move by relative offset
00161 ChangeColor, // Change drawing color
00162 MakeVisible, // Make object visible
00163 Remove, // Remove object
00164
00165 // ===== Miscellaneous =====
00166 Random, // Generate random number
00167 Test, // Generic test instruction
00168
00169 // ===== Total count =====
00170 _Count // Number of opcodes (keep last)
00171 };
00172
00173 using Operand = std::variant<std::monostate, // No operand
00174 int32_t, // Integer literal or offset
00175 uint32_t, // Unsigned value or address
00176 std::string // String literal
00177 >;
00178
00179 struct Instruction {
00180     Opcode opcode;
00181     Operand operand1;
00182     Operand operand2;
00183
00184     // Convenience constructors
00185     Instruction() : opcode(Opcode::Halt), operand1(), operand2() {}
00186
00187     explicit Instruction(Opcode op) : opcode(op), operand1(), operand2() {}

```

```

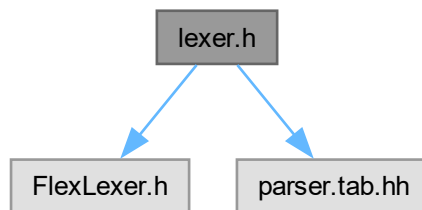
00195
00196 Instruction(Opcode op, int32_t op1) : opcode(op), operand1(op1), operand2() {}
00197
00198 Instruction(Opcode op, uint32_t op1)
00199     : opcode(op), operand1(op1), operand2() {}
00200
00201 Instruction(Opcode op, std::string op1)
00202     : opcode(op), operand1(std::move(op1)), operand2() {}
00203
00204 Instruction(Opcode op, int32_t op1, int32_t op2)
00205     : opcode(op), operand1(op1), operand2(op2) {}
00206
00207 Instruction(Opcode op, uint32_t op1, int32_t op2)
00208     : opcode(op), operand1(op1), operand2(op2) {}
00209 };
00210
00212 using InstructionStream = std::vector<Instruction>;
00213
00219 const char* opcodeName(Opcode op);
00220
00221 } // namespace nsbaci::compiler
00222
00223 #endif // NSBACI_COMPILER_INSTRUCTION_H

```

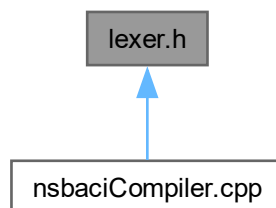
9.32 lexer.h File Reference

Lexer class declaration for nsbaci compiler.

Include dependency graph for lexer.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [nsbaci::compiler::Lexer](#)
Flex-based lexer for BACI source code.

Namespaces

- namespace [nsbaci::compiler](#)
Compiler namespace containing all compilation-related stuff.
- namespace [nsbaci](#)
Root namespace for the nsbaci application.

9.32.1 Detailed Description

Lexer class declaration for nsbaci compiler.

Author

Nicolás Serrano García

Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

9.33 lexer.h

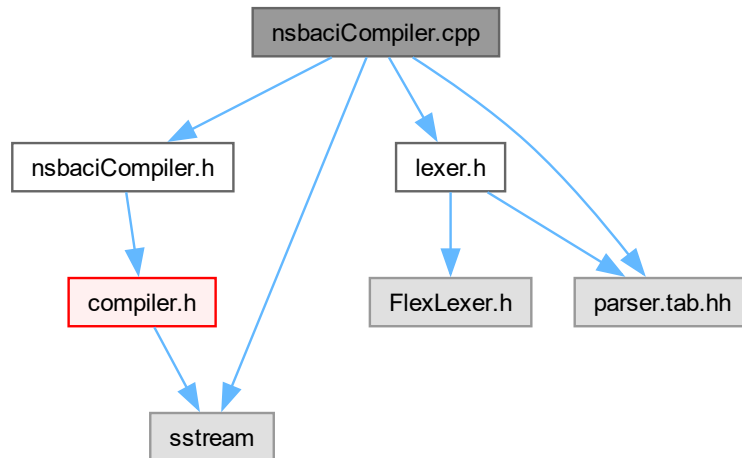
[Go to the documentation of this file.](#)

```
00001 #pragma once
00002
00011
00012 #if !defined(yyFlexLexerOnce)
00013 #include <FlexLexer.h>
00014 #endif
00015
00016 #include "parser.tab.hh"
00017
00022 namespace nsbaci::compiler {
00023
00028 class Lexer : public yyFlexLexer {
00029 public:
00030     Lexer(std::istream* in = nullptr) : yyFlexLexer(in) {}
00031     virtual ~Lexer() = default;
00032
00033     int yylex(Parser::semantic_type* yylval, Parser::location_type* yylloc);
00034 };
00035
00036 } // namespace nsbaci::compiler
```

9.34 nsbaciCompiler.cpp File Reference

NsbaciCompiler class implementation for nsbaci.

Include dependency graph for nsbaciCompiler.cpp:



Namespaces

- namespace `nsbaci`
Root namespace for the nsbaci application.
- namespace `nsbaci::compiler`
Compiler namespace containing all compilation-related stuff.

9.34.1 Detailed Description

NsbaciCompiler class implementation for nsbaci.

This file contains the implementation of the NsbaciCompiler class which uses flex-generated lexer and bison-generated parser to compile nsbaci source code into p-code instructions for the virtual machine.

Author

Nicolás Serrano García

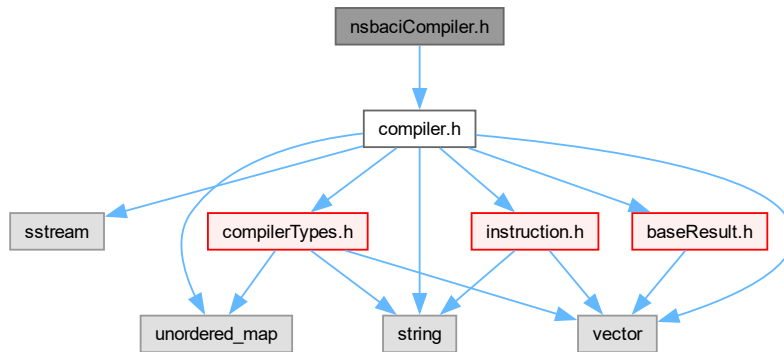
Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

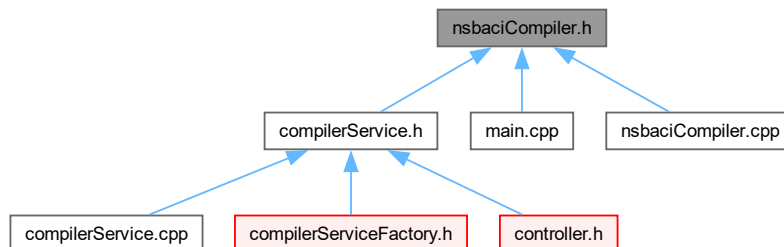
9.35 nsbaciCompiler.h File Reference

NsbaciCompiler class declaration for nsbaci.

Include dependency graph for nsbaciCompiler.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [nsbaci::compiler::NsbaciCompiler](#)
nsbaci compiler implementation using flex and bison.

Namespaces

- namespace [nsbaci::compiler](#)
Compiler namespace containing all compilation-related stuff.
- namespace [nsbaci](#)
Root namespace for the nsbaci application.

9.35.1 Detailed Description

NsbaciCompiler class declaration for nsbaci.

This module provides the concrete nsbaci implementation of the Compiler interface. NsbaciCompiler uses flex for lexical analysis and bison for parsing to compile BACI source code into p-code instructions.

The compiler supports:

- Basic types: int, bool, char
- Arithmetic and comparison operators
- Control flow: if/else, while, for loops
- C++ style I/O: cout << and cin >>
- Variable declarations with optional initialization
- Compound assignment operators (+=, -=, etc.)

Future features (not yet implemented):

- Functions and procedures
- Concurrency primitives (cobegin/coend, semaphores, monitors)
- Arrays and strings

Author

Nicolás Serrano García

Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

9.36 nsbaciCompiler.h

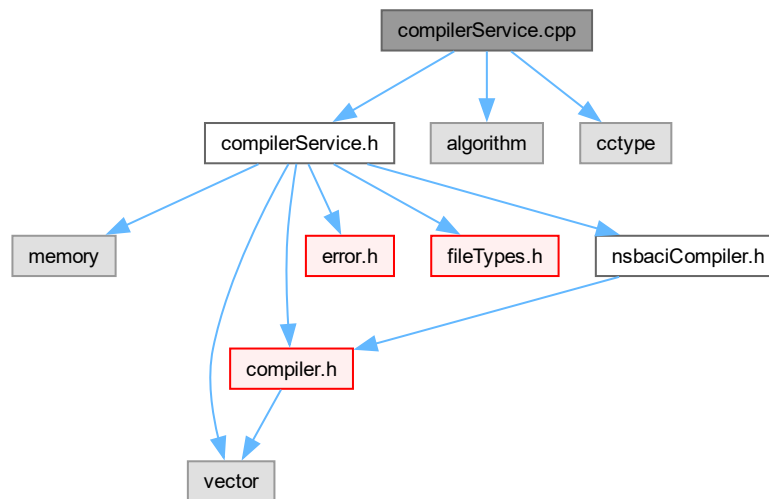
[Go to the documentation of this file.](#)

```
00001
00026
00027 #ifndef NSBACI_COMPILER_NSBACI_COMPILER_H
00028 #define NSBACI_COMPILER_NSBACI_COMPILER_H
00029
00030 #include "compiler.h"
00031
00036 namespace nsbaci::compiler {
00037
00054 class NsbaciCompiler final : public Compiler {
00055 public:
00059     NsbaciCompiler() = default;
00060
00064     ~NsbaciCompiler() override = default;
00065
00076     CompilerResult compile(const std::string& source) override;
00077
00091     CompilerResult compile(std::istream& input) override;
00092 };
00093
00094 } // namespace nsbaci::compiler
00095
00096 #endif // NSBACI_COMPILER_NSBACI_COMPILER_H
```

9.37 compilerService.cpp File Reference

Implementation of the CompilerService class for nsbaci.

Include dependency graph for compilerService.cpp:



Namespaces

- namespace `nsbaci`
Root namespace for the nsbaci application.
- namespace `nsbaci::services`
Services namespace containing all backend service implementations.

9.37.1 Detailed Description

Implementation of the CompilerService class for nsbaci.

Author

Nicolás Serrano García

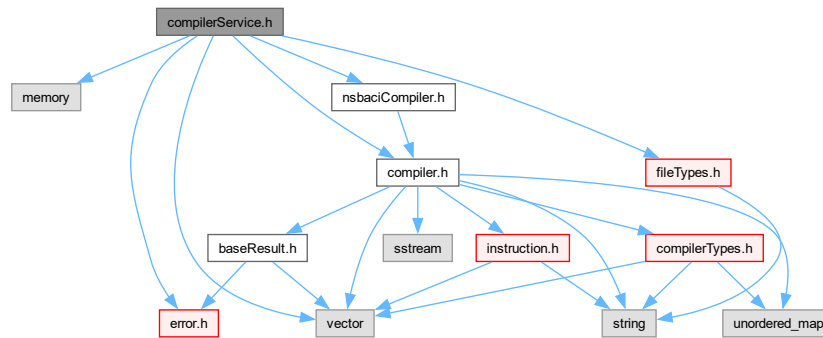
Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

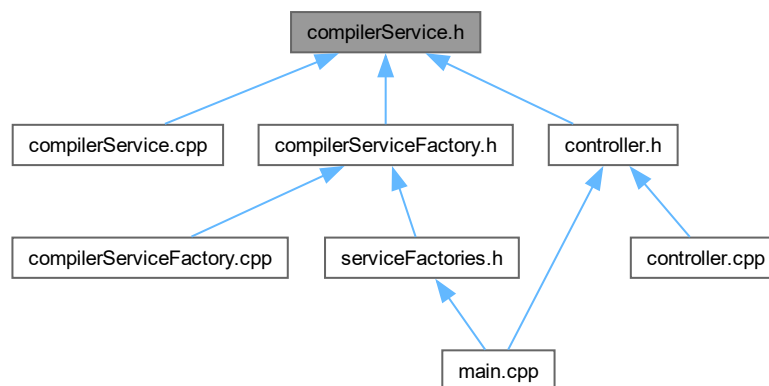
9.38 compilerService.h File Reference

CompilerService class declaration for nsbaci.

Include dependency graph for compilerService.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [nsbaci::services::CompilerService](#)
service for compiling nsbaci (or maybe other stuff in the future) source code.

Namespaces

- namespace [nsbaci::services](#)
Services namespace containing all backend service implementations.
- namespace [nsbaci](#)
Root namespace for the nsbaci application.

9.38.1 Detailed Description

CompilerService class declaration for nsbaci.

This module defines the CompilerService class that provides a high-level interface for compiling nsbaci source code into executable p-code instructions. The service wraps a Compiler implementation and manages the compiled program state, making it available for the runtime system.

The CompilerService acts as a bridge between the Controller and the actual compiler implementation. This means that the compilerService holds a specific compiler, and bubbles the interface of the compiler itself

Author

Nicolás Serrano García

Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

9.39 compilerService.h

[Go to the documentation of this file.](#)

```

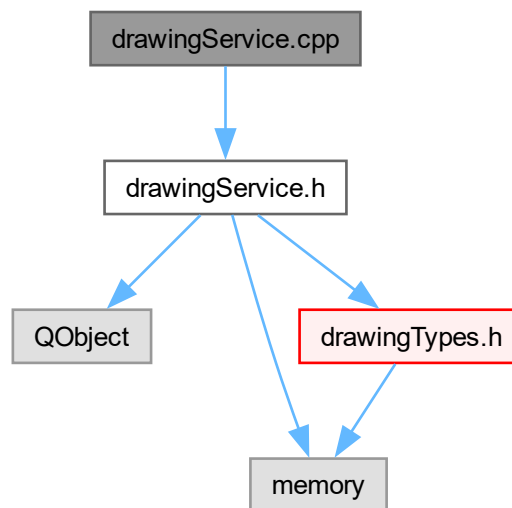
00001
00018
00019 #ifndef NSBACI_COMPILERSERVICE_H
00020 #define NSBACI_COMPILERSERVICE_H
00021
00022 #include <memory>
00023 #include <vector>
00024
00025 #include "compiler.h"
00026 #include "error.h"
00027 #include "fileTypes.h"
00028 #include "nsbaciCompiler.h"
00029
00034 namespace nsbaci::services {
00035
00061 class CompilerService {
00062 public:
00073     CompilerService(std::unique_ptr<nsbaci::compiler::Compiler> c =
00074                     std::make_unique<nsbaci::compiler::NsbaciCompiler>());
00075
00079     ~CompilerService() = default;
00080
00081     CompilerService(const CompilerService&) = delete;
00082     CompilerService& operator=(const CompilerService&) = delete;
00083
00084     CompilerService(CompilerService&&) = default;
00085     CompilerService& operator=(CompilerService&&) = default;
00086
00098     nsbaci::compiler::CompilerResult compile(nsbaci::types::Text raw);
00099
00109     bool hasProgramReady() const;
00110
00120     nsbaci::compiler::InstructionStream takeInstructions();
00121
00131     nsbaci::types::SymbolTable takeSymbols();
00132
00133 private:
00134     std::unique_ptr<nsbaci::compiler::Compiler>
00135         compiler;
00136     nsbaci::compiler::InstructionStream
00137         lastCompiledInstructions;
00138     nsbaci::types::SymbolTable
00139         lastCompiledSymbols;
00140     bool programReady = false;
00141 };
00142
00143 } // namespace nsbaci::services
00144
00145 #endif // NSBACI_COMPILERSERVICE_H

```

9.40 drawingService.cpp File Reference

DrawingService class implementation for nsbaci.

Include dependency graph for drawingService.cpp:



Namespaces

- namespace `nsbaci`
Root namespace for the nsbaci application.
- namespace `nsbaci::services`
Services namespace containing all backend service implementations.

9.40.1 Detailed Description

DrawingService class implementation for nsbaci.

Author

Nicolás Serrano García

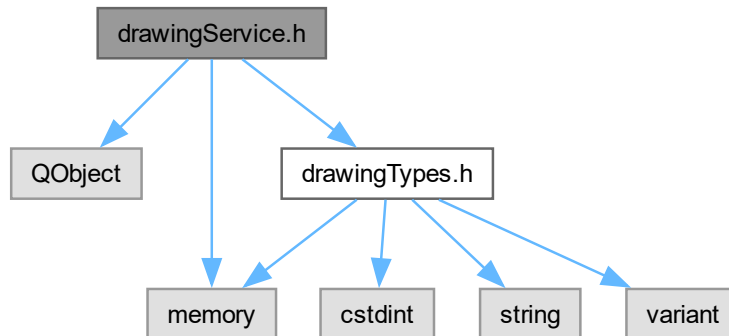
Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

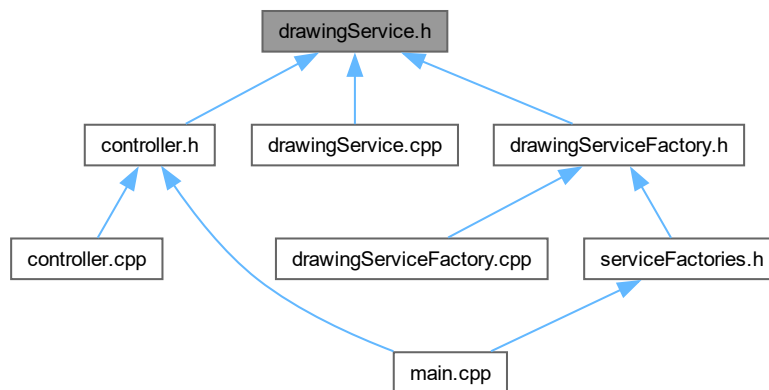
9.41 drawingService.h File Reference

DrawingService class declaration for nsbaci.

Include dependency graph for drawingService.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [nsbaci::services::DrawingService](#)
Adapter service for graphical output backends.

Namespaces

- namespace [nsbaci::services](#)
Services namespace containing all backend service implementations.
- namespace [nsbaci](#)
Root namespace for the nsbaci application.

9.41.1 Detailed Description

DrawingService class declaration for nsbaci.

This service acts as an adapter for graphical output backends. It communicates with the RuntimeService via Qt signals/slots to handle drawing operations triggered by program execution.

Author

Nicolás Serrano García

Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

9.42 drawingService.h

[Go to the documentation of this file.](#)

```

00001
00013
00014 #ifndef NSBACI_SERVICES_DRAWINGSERVICE_H
00015 #define NSBACI_SERVICES_DRAWINGSERVICE_H
00016
00017 #include <QObject>
00018 #include <memory>
00019
00020 #include "drawingTypes.h"
00021
00026 namespace nsbaci::services {
00027
00044 class DrawingService : public QObject {
00045     Q_OBJECT
00046
00047 public:
00048     explicit DrawingService(QObject* parent = nullptr);
00049     ~DrawingService() = default;
00050
00051     // QObject subclasses cannot be copied or moved
00052     DrawingService(const DrawingService&) = delete;
00053     DrawingService& operator=(const DrawingService&) = delete;
00054     DrawingService(DrawingService&&) = delete;
00055     DrawingService& operator=(DrawingService&&) = delete;
00056
00057     // ===== Color Management =====
00058
00065     void setColor(uint8_t r, uint8_t g, uint8_t b);
00066
00074     void setColor(uint8_t r, uint8_t g, uint8_t b, uint8_t a);
00075
00080     void setColor(const nsbaci::types::Color& color);
00081
00086     nsbaci::types::Color getCurrentColor() const { return currentColor_; }
00087
00088     // ===== Position Management =====
00089
00095     void setPosition(int32_t x, int32_t y);
00096
00101     void setPosition(const nsbaci::types::Point& point);
00102
00107     nsbaci::types::Point getCurrentPosition() const { return currentPosition_; }
00108
00109     // ===== Canvas Operations =====
00110
00114     void clear();
00115
00120     void clear(const nsbaci::types::Color& color);
00121
00125     void fill();
00126
00130     void refresh();
00131
00136     void setLineWidth(int32_t width);

```

```

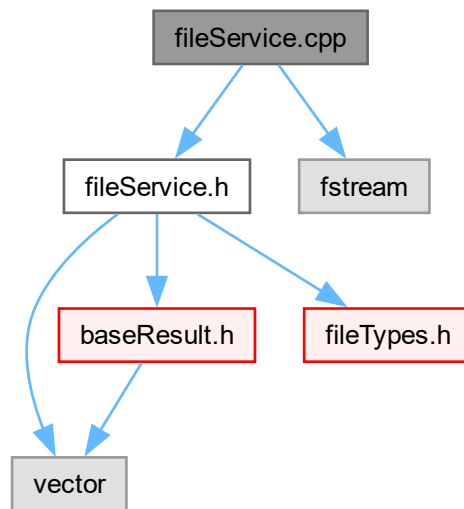
00137
00142 int32_t getLineWidth() const { return lineWidth_; }
00143
00144 // ===== Shape Drawing =====
00145
00153 void drawCircle(int32_t centerX, int32_t centerY, int32_t radius,
00154                bool filled = false);
00155
00164 void drawRectangle(int32_t x, int32_t y, int32_t width, int32_t height,
00165                   bool filled = false);
00166
00174 void drawTriangle(int32_t x1, int32_t y1, int32_t x2, int32_t y2, int32_t x3,
00175                  int32_t y3, bool filled = false);
00176
00182 void drawLine(int32_t x1, int32_t y1, int32_t x2, int32_t y2);
00183
00192 void drawEllipse(int32_t centerX, int32_t centerY, int32_t radiusX,
00193                 int32_t radiusY, bool filled = false);
00194
00200 void drawPixel(int32_t x, int32_t y);
00201
00209 void drawText(int32_t x, int32_t y, const std::string& text,
00210              int32_t fontSize = 12);
00211
00216 void drawShape(const nsbaci::types::Shape& shape);
00217
00218 // ===== Canvas Configuration =====
00219
00224 nsbaci::types::Size getCanvasSize() const { return canvasSize_; }
00225
00231 void setCanvasSize(int32_t width, int32_t height);
00232
00236 void reset();
00237
00246 void processCommand(const nsbaci::types::DrawCommand& command);
00247
00248 signals:
00253 void drawCommandReceived(const nsbaci::types::DrawCommand& command);
00254
00259 void clearRequested(const nsbaci::types::Color& backgroundColor);
00260
00265 void drawRequested(const nsbaci::types::Drawable& drawable);
00266
00270 void refreshRequested();
00271
00276 void canvasSizeChanged(const nsbaci::types::Size& size);
00277
00278 private:
00279 nsbaci::types::Color currentColor_{0, 0, 0}; // Default black
00280 nsbaci::types::Point currentPosition_{0, 0}; // Default origin
00281 int32_t lineWidth_ = 1; // Default line width
00282 nsbaci::types::Size canvasSize_{800, 600}; // Default canvas size
00283 nsbaci::types::Color backgroundColor_{255, 255, 255}; // Default white
00284 };
00285
00286 } // namespace nsbaci::services
00287
00288 #endif // NSBACI_SERVICES_DRAWINGSERVICE_H

```

9.43 fileService.cpp File Reference

Implementation of the FileService class for nsbaci.

Include dependency graph for `fileService.cpp`:



Namespaces

- namespace `nsbaci`
Root namespace for the nsbaci application.
- namespace `nsbaci::services`
Services namespace containing all backend service implementations.

9.43.1 Detailed Description

Implementation of the `FileService` class for `nsbaci`.

This file contains the implementation of file save and load operations for `NsBaci` source files (`.nsb`). It provides comprehensive validation and error handling for all file system operations.

Author

Nicolás Serrano García

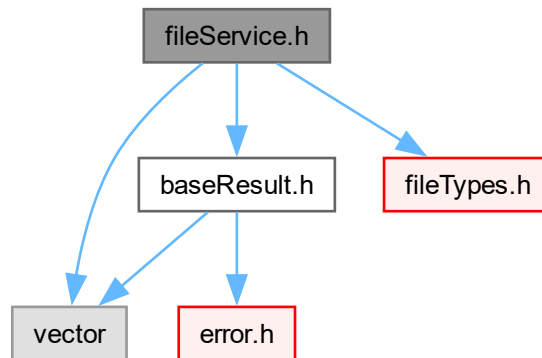
Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

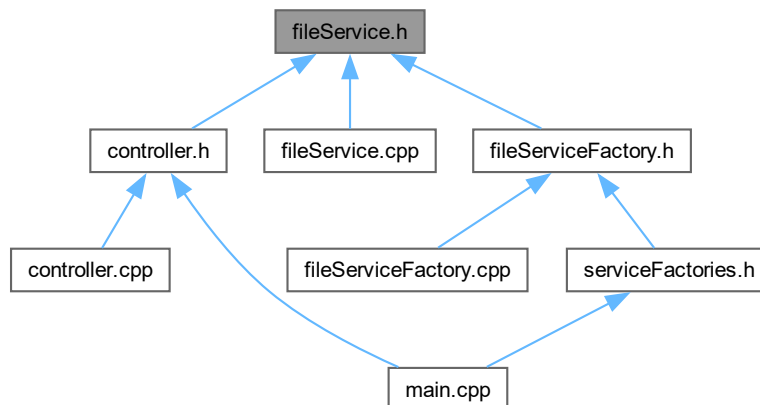
9.44 fileService.h File Reference

FileService class declaration for nsbaci.

Include dependency graph for fileService.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct [FileResult](#)
Base result type for file operations.
- struct [saveResult](#)
Result type for file save operations.
- struct [LoadResult](#)
Result type for file load operations.
- class [nsbaci::services::FileService](#)
Service for handling file system operations on BACI source files.

Namespaces

- namespace [nsbaci::services](#)
Services namespace containing all backend service implementations.
- namespace [nsbaci](#)
Root namespace for the nsbaci application.

9.44.1 Detailed Description

FileService class declaration for nsbaci.

This module defines the FileService class and its associated result types for handling file system operations. The FileService provides a clean abstraction over file I/O operations, specifically tailored for Nsbaci source files (.nsb extension).

The service handles:

- Saving source code to .nsb files with validation
- Loading source code from .nsb files with error checking
- Path validation

Author

Nicolás Serrano García

Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

9.45 fileService.h

[Go to the documentation of this file.](#)

```
00001
00019
00020 #ifndef NSBACI_FILESERVICE_H
00021 #define NSBACI_FILESERVICE_H
00022
00023 #include <vector>
00024
00025 #include "baseResult.h"
00026 #include "fileTypes.h"
00027
00037 struct FileResult : nsbaci::BaseResult {
00041     FileResult() : BaseResult() {}
00042
00047     explicit FileResult(std::vector<nsbaci::Error> errs)
00048         : BaseResult(std::move(errs)) {}
00049
00054     explicit FileResult(nsbaci::Error error) : BaseResult(std::move(error)) {}
00055
00056     FileResult(FileResult&&) noexcept = default;
00057     FileResult& operator=(FileResult&&) noexcept = default;
00058
00059     FileResult(const FileResult&) = default;
00060     FileResult& operator=(const FileResult&) = default;
00061 };
00062
00070 struct saveResult : FileResult {
00074     saveResult() : FileResult() {}
```



```

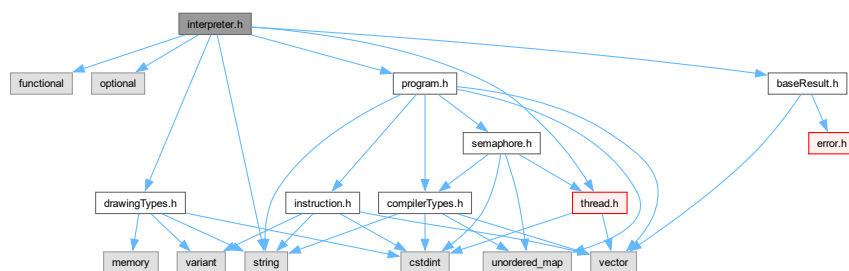
00075
00080 explicit saveResult(std::vector<nsbaci::Error> errs)
00081     : FileResult(std::move(errs)) {}
00082
00087 explicit saveResult(nsbaci::Error error) : FileResult(std::move(error)) {}
00088
00089 saveResult(saveResult&&) noexcept = default;
00090 saveResult& operator=(saveResult&&) noexcept = default;
00091 saveResult(const saveResult&) = default;
00092 saveResult& operator=(const saveResult&) = default;
00093 };
00094
00103 struct LoadResult : FileResult {
00107     LoadResult() : FileResult() {}
00108
00114     LoadResult(nsbaci::types::Text conts, nsbaci::types::File name)
00115         : FileResult(), contents(std::move(conts)), fileName(std::move(name)) {}
00116
00121 explicit LoadResult(std::vector<nsbaci::Error> errs)
00122     : FileResult(std::move(errs)) {}
00123
00128 explicit LoadResult(nsbaci::Error error) : FileResult(std::move(error)) {}
00129
00130 LoadResult(LoadResult&&) noexcept = default;
00131 LoadResult& operator=(LoadResult&&) noexcept = default;
00132 LoadResult(const LoadResult&) = default;
00133 LoadResult& operator=(const LoadResult&) = default;
00134
00135 nsbaci::types::Text contents;
00136 nsbaci::types::File fileName;
00137 };
00138
00146 namespace nsbaci::services {
00147
00181 class FileService {
00182 public:
00196     saveResult save(nsbaci::types::Text contents, nsbaci::types::File file);
00197
00208     LoadResult load(nsbaci::types::File file);
00209
00213     FileService() = default;
00214
00215     FileService(const FileService&) = delete;
00216     FileService& operator=(const FileService&) = delete;
00217
00218     FileService(FileService&&) = default;
00219     FileService& operator=(FileService&&) = default;
00220
00221     ~FileService() = default;
00222 };
00223
00224 } // namespace nsbaci::services
00225
00226 #endif // NSBACI_FILESERVICE_H

```

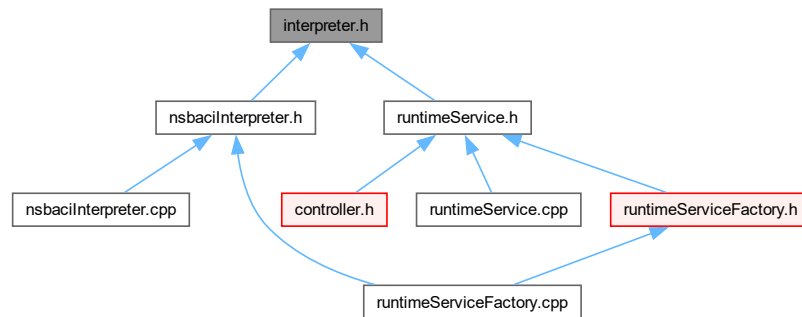
9.46 interpreter.h File Reference

Interpreter class declaration for nsbaci runtime service.

Include dependency graph for interpreter.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct [InterpreterResult](#)
- class [nsbaci::services::runtime::Interpreter](#)
Executes instructions for threads within a program context.

Namespaces

- namespace [nsbaci::services::runtime](#)
Runtime services namespace for nsbaci.
- namespace [nsbaci](#)
Root namespace for the nsbaci application.
- namespace [nsbaci::services](#)
Services namespace containing all backend service implementations.

Typedefs

- using **nsbaci::services::runtime::OutputCallback** = `std::function<void(const std::string&)>`
Callback type for output operations.
- using **nsbaci::services::runtime::InputRequestCallback** = `std::function<void(const std::string&)>`
Callback type for input requests.
- using **nsbaci::services::runtime::DrawingCallback** = `std::function<void(const nsbaci::types::DrawCommand&)>`
Callback type for drawing operations.

9.46.1 Detailed Description

Interpreter class declaration for nsbaci runtime service.

This module defines the Interpreter class responsible for executing instructions within a program context.

Author

Nicolás Serrano García

Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

9.47 interpreter.h

[Go to the documentation of this file.](#)

```

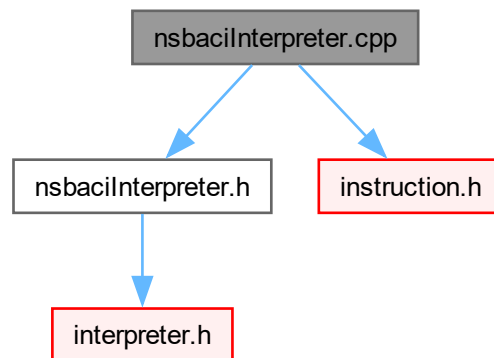
00001
00012
00013 #ifndef NSBACI_SERVICES_RUNTIME_INTERPRETER_H
00014 #define NSBACI_SERVICES_RUNTIME_INTERPRETER_H
00015
00016 #include <functional>
00017 #include <optional>
00018 #include <string>
00019
00020 #include "baseResult.h"
00021 #include "drawingTypes.h"
00022 #include "program.h"
00023 #include "thread.h"
00024
00025 struct InterpreterResult : nsbaci::BaseResult {
00026     InterpreterResult() : BaseResult() {}
00027     explicit InterpreterResult(std::vector<nsbaci::Error> errs)
00028         : BaseResult(std::move(errs)) {}
00029     explicit InterpreterResult(nsbaci::Error error)
00030         : BaseResult(std::move(error)) {}
00031
00032     InterpreterResult(InterpreterResult&& noexcept = default;
00033     InterpreterResult& operator=(InterpreterResult&& noexcept = default;
00034
00035     InterpreterResult(const InterpreterResult&) = default;
00036     InterpreterResult& operator=(const InterpreterResult&) = default;
00037
00038     bool needsInput = false;
00039     std::string inputPrompt;
00040     std::string output;
00041
00042     // Concurrency control signals
00043     bool shouldBlock = false;
00044     uint32_t blockingSemaphore = 0;
00045     bool shouldYield = false;
00046     bool cobeginStart = false;
00047     bool coendWait = false;
00048     int32_t expectedThreadCount = 0;
00049     bool createThread = false;
00050     uint32_t newThreadPC = 0;
00051     bool signalSemaphore = false;
00052     uint32_t signaledSemaphore = 0;
00053 };
00054
00055 namespace nsbaci::services::runtime {
00056
00057     using OutputCallback = std::function<void(const std::string&);>;
00058
00059     using InputRequestCallback = std::function<void(const std::string&);>;
00060
00061     using DrawingCallback = std::function<void(const nsbaci::types::DrawCommand&);>;
00062
00063     class Interpreter {
00064     public:
00065         Interpreter() = default;
00066         virtual ~Interpreter() = default;
00067
00068         virtual InterpreterResult executeInstruction(Thread& t, Program& program) = 0;
00069
00070         virtual void provideInput(const std::string& input) = 0;
00071
00072         virtual bool isWaitingForInput() const = 0;
00073
00074         virtual void setOutputCallback(OutputCallback callback) = 0;
00075
00076         virtual void setDrawingCallback(DrawingCallback callback) = 0;
00077
00078         virtual void reset() = 0;
00079     };
00080
00081 } // namespace nsbaci::services::runtime
00082
00083 #endif // NSBACI_SERVICES_RUNTIME_INTERPRETER_H

```

9.48 nsbaciInterpreter.cpp File Reference

NsbaciInterpreter class implementation for nsbaci runtime service.

Include dependency graph for nsbaciInterpreter.cpp:



Namespaces

- namespace [nsbaci](#)
Root namespace for the nsbaci application.
- namespace [nsbaci::services](#)
Services namespace containing all backend service implementations.
- namespace [nsbaci::services::runtime](#)
Runtime services namespace for nsbaci.

9.48.1 Detailed Description

NsbaciInterpreter class implementation for nsbaci runtime service.

Author

Nicolás Serrano García

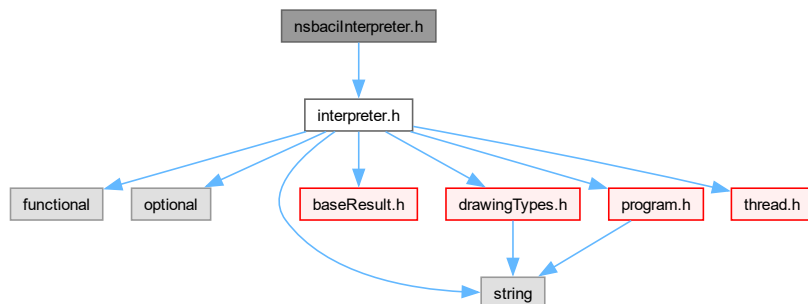
Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

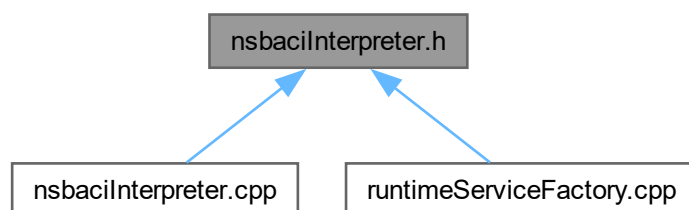
9.49 nsbaciInterpreter.h File Reference

NsbaciInterpreter class declaration for nsbaci runtime service.

Include dependency graph for nsbaciInterpreter.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [nsbaci::services::runtime::NsbaciInterpreter](#)
BACI-specific implementation of the [Interpreter](#).

Namespaces

- namespace [nsbaci::services::runtime](#)
Runtime services namespace for nsbaci.
- namespace [nsbaci](#)
Root namespace for the nsbaci application.
- namespace [nsbaci::services](#)
Services namespace containing all backend service implementations.

9.49.1 Detailed Description

NsbaciInterpreter class declaration for nsbaci runtime service.

This module provides the concrete BACI-specific implementation of the Interpreter interface for executing BACI instructions.

Author

Nicolás Serrano García

Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

9.50 nsbaciInterpreter.h

[Go to the documentation of this file.](#)

```

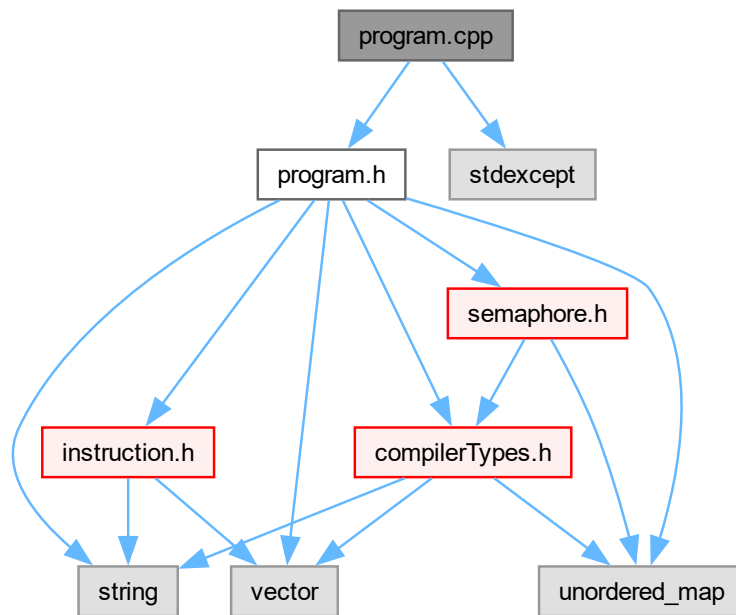
00001
00012
00013 #ifndef NSBACI_SERVICES_RUNTIME_NSBACI_INTERPRETER_H
00014 #define NSBACI_SERVICES_RUNTIME_NSBACI_INTERPRETER_H
00015
00016 #include "interpreter.h"
00017
00022 namespace nsbaci::services::runtime {
00023
00031 class NsbaciInterpreter final : public Interpreter {
00032 public:
00033     NsbaciInterpreter() = default;
00034     ~NsbaciInterpreter() override = default;
00035
00043     InterpreterResult executeInstruction(Thread& t, Program& program) override;
00044
00045     void provideInput(const std::string& input) override;
00046     bool isWaitingForInput() const override;
00047     void setOutputCallback(OutputCallback callback) override;
00048     void setDrawingCallback(DrawingCallback callback) override;
00049     void reset() override;
00050
00051 private:
00052     OutputCallback outputCallback;
00053     DrawingCallback drawingCallback;
00054     bool waitingForInput = false;
00055     std::string pendingInput;
00056     bool hasInput = false;
00057
00058     // Drawing state
00059     uint8_t currentR = 0, currentG = 0, currentB = 0, currentA = 255;
00060     int32_t currentLineWidth = 1;
00061 };
00062
00063 } // namespace nsbaci::services::runtime
00064
00065 #endif // NSBACI_SERVICES_RUNTIME_NSBACI_INTERPRETER_H

```

9.51 program.cpp File Reference

Program class implementation for nsbaci runtime service.

Include dependency graph for program.cpp:



Namespaces

- namespace `nsbaci`
Root namespace for the `nsbaci` application.
- namespace `nsbaci::services`
Services namespace containing all backend service implementations.
- namespace `nsbaci::services::runtime`
Runtime services namespace for `nsbaci`.

9.51.1 Detailed Description

Program class implementation for `nsbaci` runtime service.

Author

Nicolás Serrano García

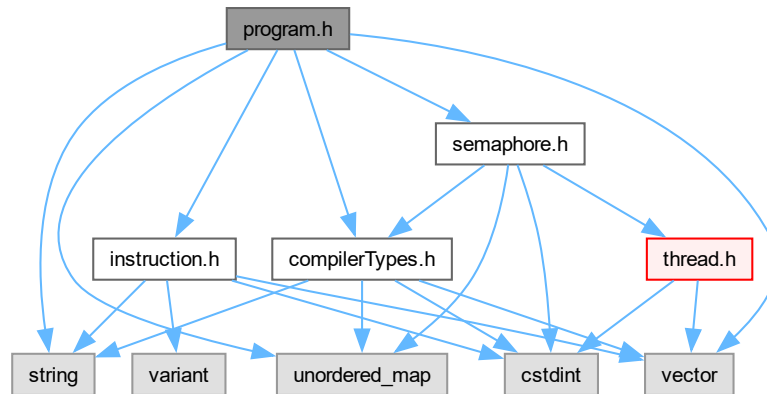
Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

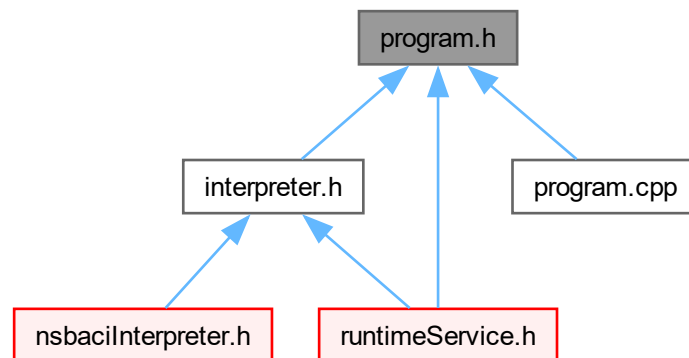
9.52 program.h File Reference

Program class declaration for nsbaci runtime service.

Include dependency graph for program.h:



This graph shows which files directly or indirectly include this file:



Classes

- class `nsbaci::services::runtime::Program`
Represents a compiled program ready for execution.

Namespaces

- namespace `nsbaci::types`
Type definitions namespace for nsbaci (runtime-specific).
- namespace `nsbaci`
Root namespace for the nsbaci application.
- namespace `nsbaci::services::runtime`
Runtime services namespace for nsbaci.
- namespace `nsbaci::services`
Services namespace containing all backend service implementations.

Typedefs

- using `nsbaci::types::StackValue` = `int32_t`
Stack value type (can hold int or address).
- using `nsbaci::types::Stack` = `std::vector<StackValue>`
Runtime stack.
- using `nsbaci::types::Memory` = `std::vector<int32_t>`
Memory block for runtime data.

9.52.1 Detailed Description

Program class declaration for nsbaci runtime service.

This module defines the Program class which holds the compiled program's instruction vector, memory tables, and other runtime data.

Author

Nicolás Serrano García

Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

9.53 program.h

[Go to the documentation of this file.](#)

```
00001
00012
00013 #ifndef NSBACI_SERVICES_RUNTIME_PROGRAM_H
00014 #define NSBACI_SERVICES_RUNTIME_PROGRAM_H
00015
00016 #include <string>
00017 #include <unordered_map>
00018 #include <vector>
00019
00020 #include "compilerTypes.h"
00021 #include "instruction.h"
00022 #include "semaphore.h"
00023
00028 namespace nsbaci::types {
00029
00031 using StackValue = int32_t;
00032
00034 using Stack = std::vector<StackValue>;
```

```

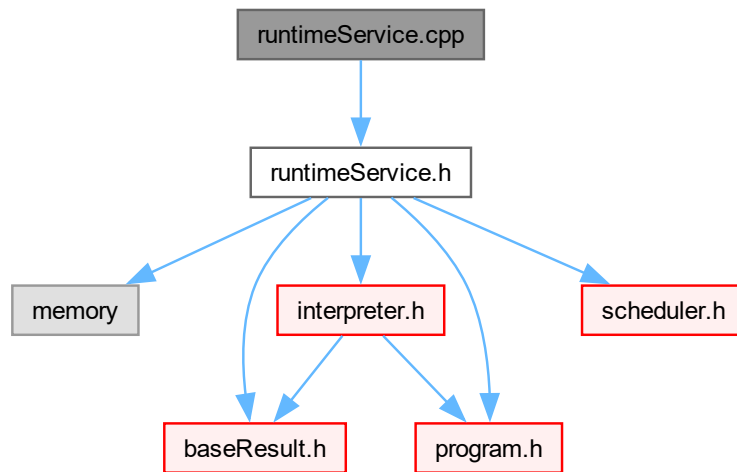
00035
00037 using Memory = std::vector<int32_t>;
00038
00039 } // namespace nsbaci::types
00040
00045 namespace nsbaci::services::runtime {
00046
00054 class Program {
00055 public:
00056     Program() = default;
00057     explicit Program(nsbaci::compiler::InstructionStream i);
00058     Program(nsbaci::compiler::InstructionStream i, nsbaci::types::SymbolTable s);
00059     ~Program() = default;
00060
00061     Program(const Program&) = delete;
00062     Program& operator=(const Program&) = delete;
00063
00064     Program(Program&&) = default;
00065     Program& operator=(Program&&) = default;
00066
00072     const nsbaci::compiler::Instruction& getInstruction(uint32_t addr) const;
00073
00078     size_t instructionCount() const;
00079
00084     nsbaci::types::Memory& memory();
00085     const nsbaci::types::Memory& memory() const;
00086
00091     const nsbaci::types::SymbolTable& symbols() const;
00092
00097     void addSymbol(nsbaci::types::SymbolInfo info);
00098
00104     int32_t readMemory(nsbaci::types::MemoryAddr addr) const;
00105
00111     void writeMemory(nsbaci::types::MemoryAddr addr, int32_t value);
00112
00118     void createSemaphore(nsbaci::types::MemoryAddr addr, int32_t initialCount);
00119
00126     bool semaphoreWait(nsbaci::types::MemoryAddr addr,
00127                        nsbaci::types::ThreadID threadId);
00128
00134     nsbaci::types::ThreadID semaphoreSignal(nsbaci::types::MemoryAddr addr);
00135
00141     bool hasSemaphore(nsbaci::types::MemoryAddr addr) const;
00142
00143 private:
00144     // Instruction stream - read-only after construction
00145     nsbaci::compiler::InstructionStream instructions;
00146     // Global symbol table
00147     nsbaci::types::SymbolTable symbolTable;
00148     // Global memory
00149     nsbaci::types::Memory globalMemory;
00150     // Semaphores
00151     nsbaci::types::SemaphoreTable semaphores;
00152 };
00153
00154 } // namespace nsbaci::services::runtime
00155
00156 #endif // NSBACI_SERVICES_RUNTIME_PROGRAM_H

```

9.54 runtimeService.cpp File Reference

Implementation unit for the RuntimeService.

Include dependency graph for runtimeService.cpp:



Namespaces

- namespace [nsbaci](#)
Root namespace for the nsbaci application.
- namespace [nsbaci::services](#)
Services namespace containing all backend service implementations.

9.54.1 Detailed Description

Implementation unit for the RuntimeService.

Author

Nicolás Serrano García

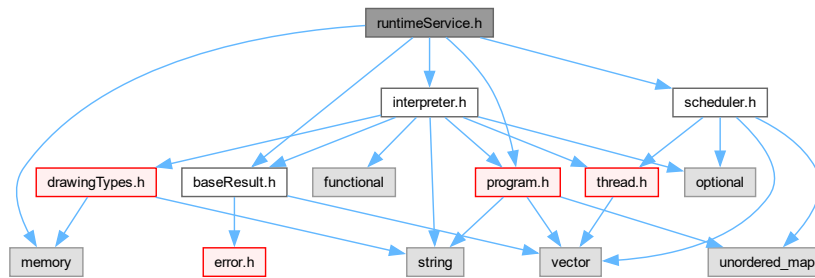
Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

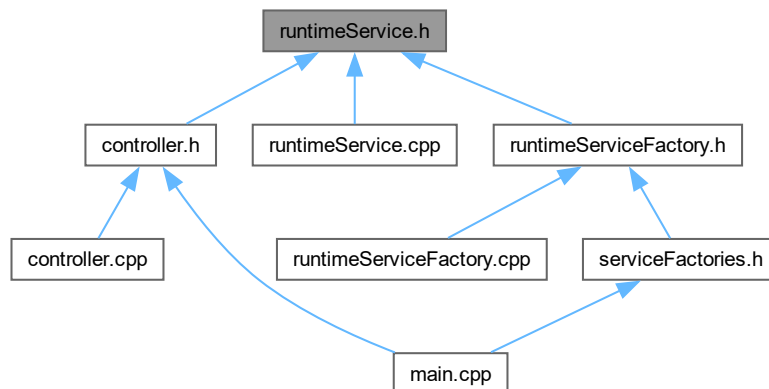
9.55 runtimeService.h File Reference

RuntimeService class declaration for nsbaci.

Include dependency graph for runtimeService.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct [nsbaci::services::RuntimeResult](#)
Result of a runtime operation (step, run, etc.).
- class [nsbaci::services::RuntimeService](#)
Service that manages program execution.

Namespaces

- namespace [nsbaci::services](#)
Services namespace containing all backend service implementations.
- namespace [nsbaci](#)
Root namespace for the nsbaci application.

Enumerations

- enum class `nsbaci::services::RuntimeState` { `nsbaci::services::Idle` , `nsbaci::services::Running` , `nsbaci::services::Paused` , `nsbaci::services::Halted` }

Possible states of the runtime service.

9.55.1 Detailed Description

RuntimeService class declaration for nsbaci.

This module defines the RuntimeService class which serves as the main interface for executing compiled nsbaci programs. It controls the interpreter and scheduler components to provide program execution with support for stepping, continuous running, and status info

Author

Nicolás Serrano García

Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

9.56 runtimeService.h

[Go to the documentation of this file.](#)

```
00001
00014
00015 #ifndef NSBACI_RUNTIMESERVICE_H
00016 #define NSBACI_RUNTIMESERVICE_H
00017
00018 #include <memory>
00019
00020 #include "baseResult.h"
00021 #include "interpreter.h"
00022 #include "program.h"
00023 #include "scheduler.h"
00024
00029 namespace nsbaci::services {
00030
00035 struct RuntimeResult : nsbaci::BaseResult {
00039     RuntimeResult() : BaseResult() {}
00040
00045     explicit RuntimeResult(std::vector<nsbaci::Error> errs)
00046         : BaseResult(std::move(errs)) {}
00047
00052     explicit RuntimeResult(nsbaci::Error error) : BaseResult(std::move(error)) {}
00053
00054     RuntimeResult(RuntimeResult&&) noexcept = default;
00055     RuntimeResult& operator=(RuntimeResult&&) noexcept = default;
00056     RuntimeResult(const RuntimeResult&) = default;
00057     RuntimeResult& operator=(const RuntimeResult&) = default;
00058
00059     bool halted = false;
00060     bool needsInput = false;
00061     std::string inputPrompt;
00062     std::string output;
00063 };
00064
00075 enum class RuntimeState {
00076     Idle,
00077     Running,
00078     Paused,
00079     Halted
00080 };
00081
00104 class RuntimeService {
```

```

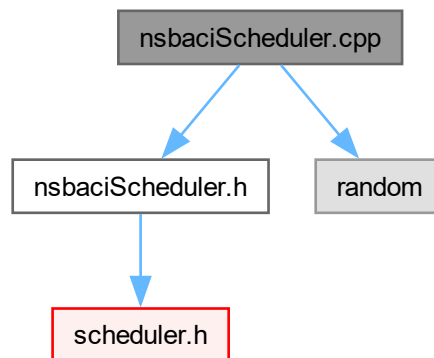
00105 public:
00113     RuntimeService() = default;
00114
00120     RuntimeService(std::unique_ptr<runtime::Interpreter> i,
00121                   std::unique_ptr<runtime::Scheduler> s);
00122
00126     ~RuntimeService() = default;
00127
00128     RuntimeService(const RuntimeService&) = delete;
00129     RuntimeService& operator=(const RuntimeService&) = delete;
00130
00131     RuntimeService(RuntimeService&&) = default;
00132     RuntimeService& operator=(RuntimeService&&) = default;
00133
00144     void loadProgram(runtime::Program&& p);
00145
00152     void reset();
00153
00162     RuntimeResult step();
00163
00172     RuntimeResult stepThread(nsbaci::types::ThreadID threadId);
00173
00186     RuntimeResult run(size_t maxSteps = 0);
00187
00193     void pause();
00194
00199     RuntimeState getState() const;
00200
00205     bool isHalted() const;
00206
00211     size_t threadCount() const;
00212
00217     const std::vector<runtime::Thread>& getThreads() const;
00218
00224     const runtime::Program& getProgram() const;
00225
00234     void provideInput(const std::string& input);
00235
00240     bool isWaitingForInput() const;
00241
00250     void setOutputCallback(runtime::OutputCallback callback);
00251
00260     void setDrawingCallback(runtime::DrawingCallback callback);
00261
00262 private:
00263     runtime::Program
00264         program;
00265     std::unique_ptr<runtime::Interpreter>
00266         interpreter;
00267     std::unique_ptr<runtime::Scheduler>
00268         scheduler;
00269     RuntimeState state = RuntimeState::Idle;
00270 };
00271
00272 } // namespace nsbaci::services
00273
00274 #endif // NSBACI_RUNTIMESERVICE_H

```

9.57 nsbaciScheduler.cpp File Reference

NsbaciScheduler class implementation for nsbaci runtime service.

Include dependency graph for nsbaciScheduler.cpp:



Namespaces

- namespace [nsbaci](#)
Root namespace for the nsbaci application.
- namespace [nsbaci::services](#)
Services namespace containing all backend service implementations.
- namespace [nsbaci::services::runtime](#)
Runtime services namespace for nsbaci.

9.57.1 Detailed Description

NsbaciScheduler class implementation for nsbaci runtime service.

Author

Nicolás Serrano García

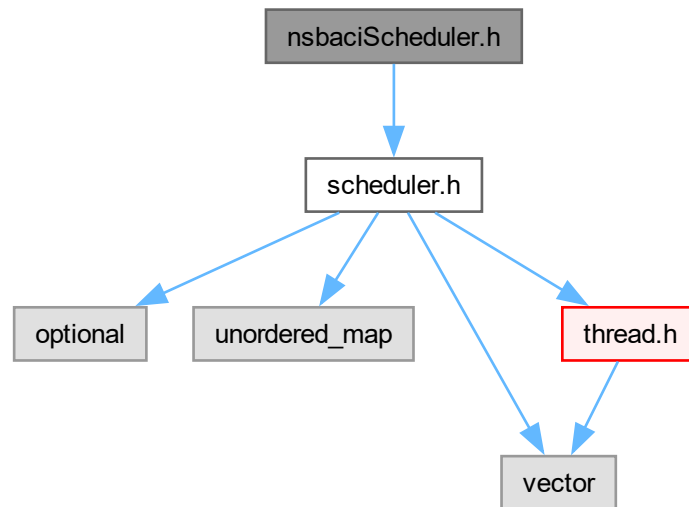
Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

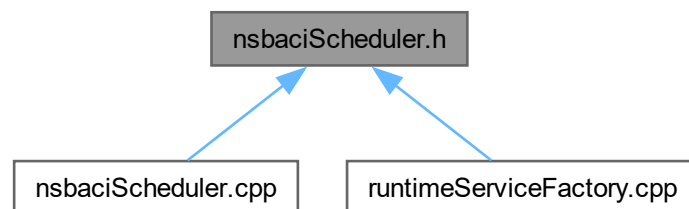
9.58 nsbaciScheduler.h File Reference

NsbaciScheduler class declaration for nsbaci runtime service.

Include dependency graph for nsbaciScheduler.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [nsbaci::services::runtime::NsbaciScheduler](#)
BACI-specific implementation of the [Scheduler](#).

Namespaces

- namespace [nsbaci::services::runtime](#)
Runtime services namespace for nsbaci.
- namespace [nsbaci](#)
Root namespace for the nsbaci application.
- namespace [nsbaci::services](#)
Services namespace containing all backend service implementations.

9.58.1 Detailed Description

NsbaciScheduler class declaration for nsbaci runtime service.

This module provides the concrete BACI-specific implementation of the Scheduler interface for managing thread scheduling in the nsbaci runtime.

Author

Nicolás Serrano García

Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

9.59 nsbaciScheduler.h

[Go to the documentation of this file.](#)

```

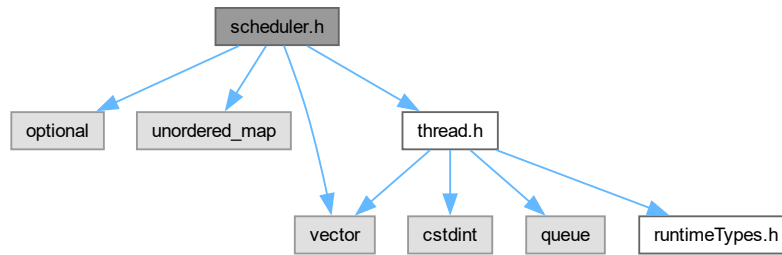
00001
00012
00013 #ifndef NSBACI_SERVICES_RUNTIME_NSBACI_SCHEDULER_H
00014 #define NSBACI_SERVICES_RUNTIME_NSBACI_SCHEDULER_H
00015
00016 #include "scheduler.h"
00017
00022 namespace nsbaci::services::runtime {
00023
00033 class NsbaciScheduler final : public Scheduler {
00034 public:
00035     NsbaciScheduler() = default;
00036     ~NsbaciScheduler() override = default;
00037
00038     Thread* pickNext() override;
00039     void addThread(Thread thread) override;
00040     void blockCurrent() override;
00041     void blockOnSemaphore(uint32_t semaphoreAddr) override;
00042     size_t unblockSemaphore(uint32_t semaphoreAddr) override;
00043     void unblock(nsbaci::types::ThreadID threadId) override;
00044     void yield() override;
00045     void terminateCurrent() override;
00046     bool hasThreads() const override;
00047     Thread* current() override;
00048     void clear() override;
00049     void unblockIO() override;
00050     const std::vector<Thread>& getThreads() const override;
00051     void blockOnCoend(int32_t expectedThreads) override;
00052     void checkCoendUnblock() override;
00053
00054 private:
00060     std::optional<size_t> findThreadIndex(nsbaci::types::ThreadID threadId) const;
00061 };
00062
00063 } // namespace nsbaci::services::runtime
00064
00065 #endif // NSBACI_SERVICES_RUNTIME_NSBACI_SCHEDULER_H

```

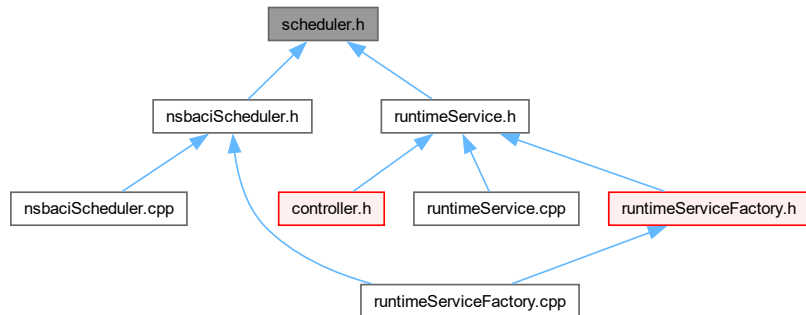
9.60 scheduler.h File Reference

Scheduler class declaration for nsbaci runtime service.

Include dependency graph for scheduler.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [nsbaci::services::runtime::Scheduler](#)
Manages thread scheduling and state transitions.

Namespaces

- namespace [nsbaci::services::runtime](#)
Runtime services namespace for nsbaci.
- namespace [nsbaci](#)
Root namespace for the nsbaci application.
- namespace [nsbaci::services](#)
Services namespace containing all backend service implementations.

9.60.1 Detailed Description

Scheduler class declaration for nsbaci runtime service.

This module defines the thread scheduler responsible for managing thread execution order and state transitions.

Author

Nicolás Serrano García

Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

9.61 scheduler.h

[Go to the documentation of this file.](#)

```

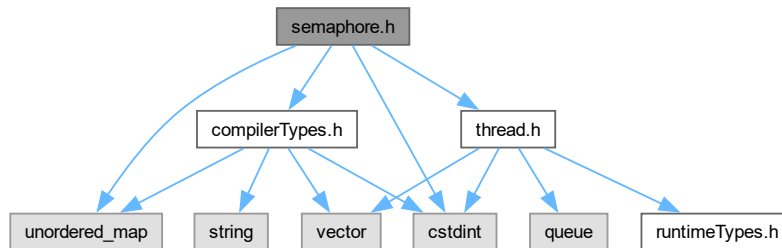
00001
00012
00013 #ifndef NSBACI_SERVICES_RUNTIME_SCHEDULER_H
00014 #define NSBACI_SERVICES_RUNTIME_SCHEDULER_H
00015
00016 #include <optional>
00017 #include <unordered_map>
00018 #include <vector>
00019
00020 #include "thread.h"
00021
00026 namespace nsbaci::services::runtime {
00027
00036 class Scheduler {
00037 public:
00038     Scheduler() = default;
00039     virtual ~Scheduler() = default;
00040
00045     virtual Thread* pickNext() = 0;
00046
00051     virtual void addThread(Thread thread) = 0;
00052
00056     virtual void blockCurrent() = 0;
00057
00062     virtual void blockOnSemaphore(uint32_t semaphoreAddr) = 0;
00063
00069     virtual size_t unblockSemaphore(uint32_t semaphoreAddr) = 0;
00070
00075     virtual void unblock(nsbaci::types::ThreadID threadId) = 0;
00076
00080     virtual void yield() = 0;
00081
00085     virtual void terminateCurrent() = 0;
00086
00091     virtual bool hasThreads() const = 0;
00092
00097     virtual Thread* current() = 0;
00098
00102     virtual void clear() = 0;
00103
00108     virtual void unblockIO() = 0;
00109
00114     virtual const std::vector<Thread>& getThreads() const = 0;
00115
00120     virtual void blockOnCoend(int32_t expectedThreads) = 0;
00121
00126     virtual void checkCoendUnblock() = 0;
00127
00128 protected:
00129     std::vector<Thread> threads;
00130     std::vector<size_t> readyQueue;
00131     std::vector<size_t> blockedQueue;
00132     std::vector<size_t> ioQueue;
00133     std::optional<size_t> runningIndex;
00134
00136     std::unordered_map<uint32_t, std::vector<size_t>> semaphoreQueues;
00137
00139     std::vector<std::pair<size_t, int32_t>> coendQueue;
00140 };
00141
00142 } // namespace nsbaci::services::runtime
00143
00144 #endif // NSBACI_SERVICES_RUNTIME_SCHEDULER_H

```

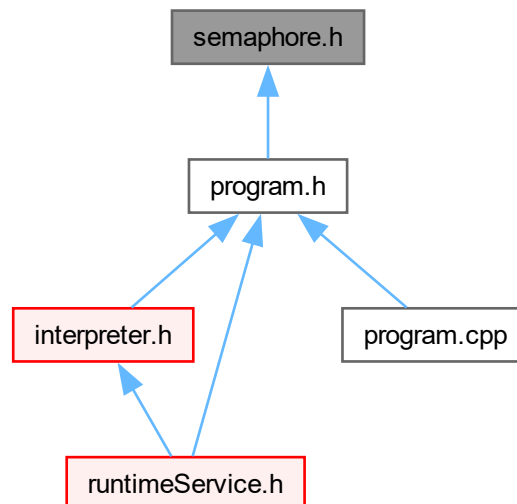
9.62 semaphore.h File Reference

Semaphore implementation for process synchronization.

Include dependency graph for semaphore.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [nsbaci::services::runtime::Semaphore](#)
Counting semaphore for process synchronization.

Namespaces

- namespace `nsbaci`
Root namespace for the nsbaci application.
- namespace `nsbaci::services`
Services namespace containing all backend service implementations.
- namespace `nsbaci::services::runtime`
Runtime services namespace for nsbaci.
- namespace `nsbaci::types`
Type definitions namespace for nsbaci (runtime-specific).

Typedefs

- using `nsbaci::types::SemaphoreTable`
Maps memory addresses to semaphores.

9.62.1 Detailed Description

Semaphore implementation for process synchronization.

9.63 semaphore.h

[Go to the documentation of this file.](#)

```

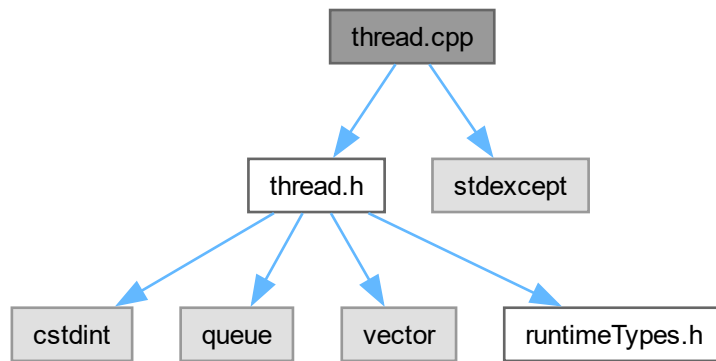
00001
00005
00006 #ifndef NSBACI_RUNTIME_SEMAPHORE_H
00007 #define NSBACI_RUNTIME_SEMAPHORE_H
00008
00009 #include <cstdint>
00010 #include <unordered_map>
00011
00012 #include "compilerTypes.h"
00013 #include "thread.h"
00014
00015 namespace nsbaci::services::runtime {
00016
00023 class Semaphore {
00024 public:
00025     explicit Semaphore(int32_t initialCount = 0) : count(initialCount) {}
00026
00030     bool wait(nsbaci::types::ThreadID currentThread);
00031
00034     nsbaci::types::ThreadID signal();
00035
00037     int32_t getCount() const { return count; }
00038
00040     bool hasWaiting() const { return !blocked.empty(); }
00041
00042 private:
00043     std::queue<nsbaci::types::ThreadID> blocked;
00044     int32_t count;
00045 };
00046
00047 } // namespace nsbaci::services::runtime
00048
00049 namespace nsbaci::types {
00050
00058 using SemaphoreTable =
00059     std::unordered_map<MemoryAddr, nsbaci::services::runtime::Semaphore>;
00060
00061 } // namespace nsbaci::types
00062
00063 #endif // NSBACI_RUNTIME_SEMAPHORE_H

```

9.64 thread.cpp File Reference

Thread class implementation for nsbaci runtime service.

Include dependency graph for thread.cpp:



Namespaces

- namespace `nsbaci`
Root namespace for the nsbaci application.
- namespace `nsbaci::services`
Services namespace containing all backend service implementations.
- namespace `nsbaci::services::runtime`
Runtime services namespace for nsbaci.

9.64.1 Detailed Description

Thread class implementation for nsbaci runtime service.

Author

Nicolás Serrano García

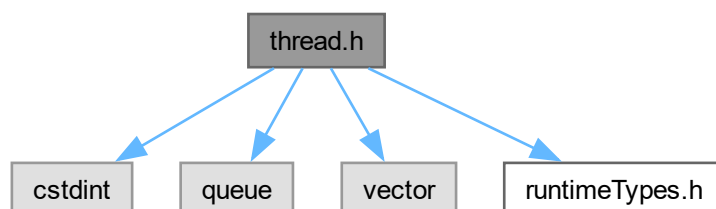
Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

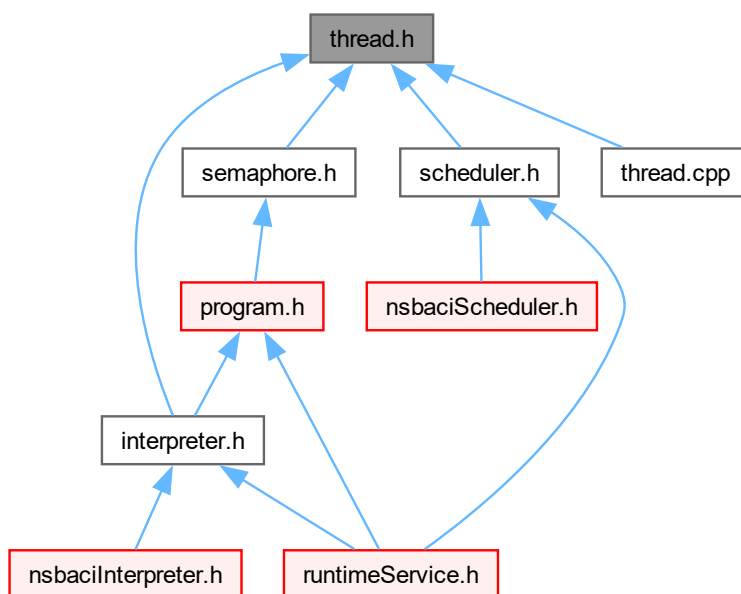
9.65 thread.h File Reference

Thread class declaration for nsbaci runtime service.

Include dependency graph for thread.h:



This graph shows which files directly or indirectly include this file:



Classes

- class `nsbaci::services::runtime::Thread`
Represents a thread in the runtime service.

Namespaces

- namespace `nsbaci::services::runtime`
Runtime services namespace for nsbaci.
- namespace `nsbaci`
Root namespace for the nsbaci application.
- namespace `nsbaci::services`
Services namespace containing all backend service implementations.
- namespace `nsbaci::types`
Type definitions namespace for nsbaci (runtime-specific).

Typedefs

- using `nsbaci::types::ThreadQueue` = `std::queue<nsbaci::services::runtime::Thread>`
Queue of threads for scheduler operations.

9.65.1 Detailed Description

Thread class declaration for nsbaci runtime service.

Author

Nicolás Serrano García

Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

9.66 thread.h

[Go to the documentation of this file.](#)

```

00001
00009
00010 #ifndef NSBACI_SERVICES_RUNTIME_THREAD_H
00011 #define NSBACI_SERVICES_RUNTIME_THREAD_H
00012
00013 #include <stdint>
00014 #include <queue>
00015 #include <vector>
00016
00017 #include "runtimeTypes.h"
00018
00023 namespace nsbaci::services::runtime {
00024
00031 class Thread {
00032 public:
00033     Thread()
00034         : id(nextThreadId++),
00035           state(nsbaci::types::ThreadState::Ready),
00036           priority(0),
00037           pc(0),
00038           bp(0),
00039           sp(0) {}
00040     ~Thread() = default;
00041
00046     nsbaci::types::ThreadID getId() const;
00047
00052     nsbaci::types::ThreadState getState() const;
00053
00058     void setState(nsbaci::types::ThreadState newState);

```



```

00059
00064     nsbaci::types::Priority getPriority() const;
00065
00070     void setPriority(nsbaci::types::Priority newPriority);
00071
00072     // ===== Stack Operations =====
00073
00077     void push(int32_t value);
00078
00082     int32_t pop();
00083
00087     int32_t top() const;
00088
00089     // ===== Program Counter =====
00090
00094     uint32_t getPC() const { return pc; }
00095
00099     void setPC(uint32_t addr) { pc = addr; }
00100
00104     void advancePC() { ++pc; }
00105
00106     // ===== Base/Stack Pointers =====
00107
00108     uint32_t getBP() const { return bp; }
00109     void setBP(uint32_t addr) { bp = addr; }
00110
00111     uint32_t getSP() const { return sp; }
00112     void setSP(uint32_t addr) { sp = addr; }
00113
00114     // ===== Call Stack =====
00115
00119     void pushReturnAddress(uint32_t addr) { callStack.push_back(addr); }
00120
00125     uint32_t popReturnAddress() {
00126         if (callStack.empty()) return 0;
00127         uint32_t addr = callStack.back();
00128         callStack.pop_back();
00129         return addr;
00130     }
00131
00135     bool callStackEmpty() const { return callStack.empty(); }
00136
00140     void pushFrame(const std::vector<int32_t>& savedLocals) {
00141         frameStack.push_back(savedLocals);
00142     }
00143
00147     std::vector<int32_t> popFrame() {
00148         if (frameStack.empty()) return {};
00149         auto frame = frameStack.back();
00150         frameStack.pop_back();
00151         return frame;
00152     }
00153
00157     bool frameStackEmpty() const { return frameStack.empty(); }
00158
00159 private:
00160     nsbaci::types::ThreadID id;
00161     nsbaci::types::ThreadState state;
00162     nsbaci::types::Priority priority;
00163
00164     // Program counter - index into instruction stream
00165     uint32_t pc;
00166     // Base pointer - for stack frames
00167     uint32_t bp;
00168     // Stack pointer
00169     uint32_t sp;
00170
00171     // Thread-local stack
00172     std::vector<int32_t> stack;
00173
00174     // Call stack for return addresses
00175     std::vector<uint32_t> callStack;
00176
00177     // Frame stack for saving local variable values during recursion
00178     std::vector<std::vector<int32_t>> frameStack;
00179
00180     static nsbaci::types::ThreadID nextThreadId;
00181
00182     // friend the scheduler
00183 };
00184
00185 } // namespace nsbaci::services::runtime
00186
00191 namespace nsbaci::types {
00192
00194     using ThreadQueue = std::queue<nsbaci::services::runtime::Thread>;
00195

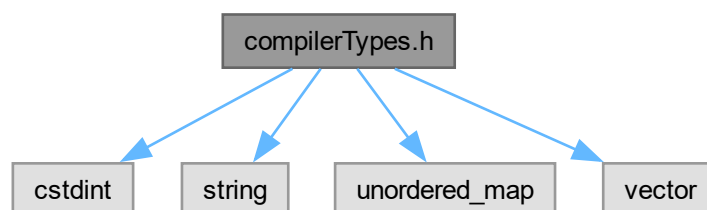
```

```
00196 } // namespace nsbaci::types
00197
00198 #endif // NSBACI_SERVICES_RUNTIME_THREAD_H
```

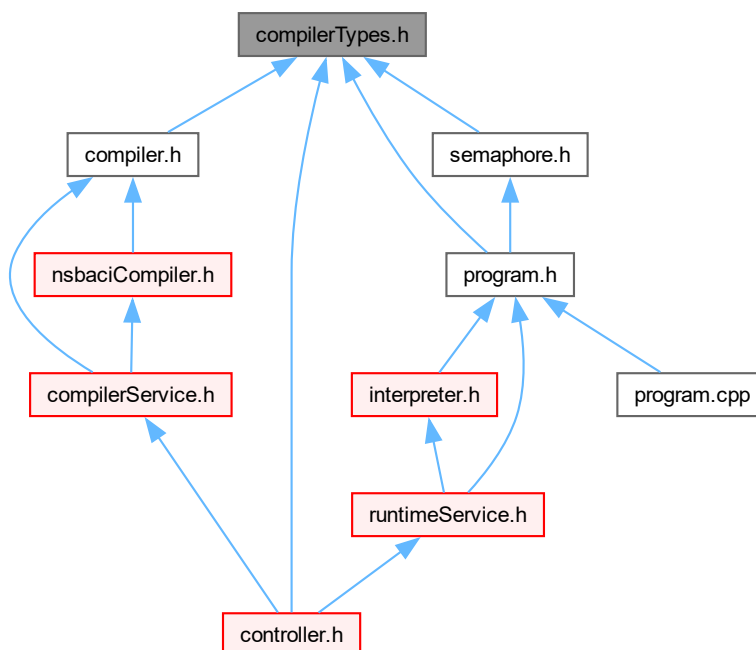
9.67 compilerTypes.h File Reference

Type definitions for compiler-related operations.

Include dependency graph for compilerTypes.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct `nsbaci::types::SymbolInfo`
Information about a variable/symbol.

Namespaces

- namespace `nsbaci::types`
Type definitions namespace for nsbaci (runtime-specific).
- namespace `nsbaci`
Root namespace for the nsbaci application.

Typedefs

- using `nsbaci::types::VarName` = `std::string`
Type alias for variable names.
- using `nsbaci::types::MemoryAddr` = `uint32_t`
Type alias for memory addresses.
- using `nsbaci::types::SymbolTable` = `std::unordered_map<VarName, SymbolInfo>`
Lookup table mapping variable names to their symbol info.

9.67.1 Detailed Description

Type definitions for compiler-related operations.

This header provides type aliases used by the CompilerService and other components that work with compilation.

Author

Nicolás Serrano García

Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

9.68 compilerTypes.h

[Go to the documentation of this file.](#)

```

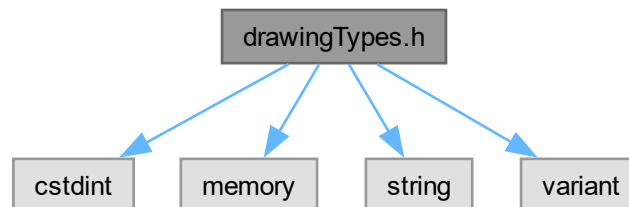
00001
00012
00013 #ifndef NSBACI_TYPES_COMPILERTYPES_H
00014 #define NSBACI_TYPES_COMPILERTYPES_H
00015
00016 #include <cstdint>
00017 #include <string>
00018 #include <unordered_map>
00019 #include <vector>
00020
00025 namespace nsbaci::types {
00026
00028 using VarName = std::string;
00029
00031 using MemoryAddr = uint32_t;
00032
00034 struct SymbolInfo {
00035     VarName name;
00036     MemoryAddr address;
00037     std::string type;
00038     bool isGlobal;
00039 };
00040
00042 using SymbolTable = std::unordered_map<VarName, SymbolInfo>;
00043
00044 } // namespace nsbaci::types
00045
00046 #endif // NSBACI_TYPES_COMPILERTYPES_H

```

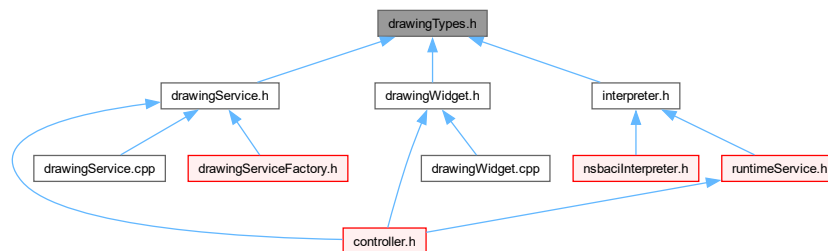
9.69 drawingTypes.h File Reference

Type definitions for drawing-related operations.

Include dependency graph for drawingTypes.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct [nsbaci::types::Color](#)
RGB color representation with values from 0-255.
- struct [nsbaci::types::Point](#)
2D point/position representation.
- struct [nsbaci::types::Size](#)
2D size representation.
- struct [nsbaci::types::Circle](#)
Circle shape with center and radius.
- struct [nsbaci::types::Rectangle](#)
Rectangle shape with position and size.
- struct [nsbaci::types::Triangle](#)
Triangle shape defined by three vertices.
- struct [nsbaci::types::Line](#)
Line segment from start to end point.
- struct [nsbaci::types::Ellipse](#)

- [Ellipse](#) shape with center and radii.
- struct [nsbaci::types::Pixel](#)
Single pixel at a position.
- struct [nsbaci::types::DrawText](#)
- struct [nsbaci::types::Drawable](#)
Complete drawable object with shape, color, and visibility.
- struct [nsbaci::types::DrawCommand](#)
Represents a single drawing command to be executed.
- struct [nsbaci::types::CanvasConfig](#)
Configuration for the drawing canvas.

Namespaces

- namespace [nsbaci::types](#)
Type definitions namespace for nsbaci (runtime-specific).
- namespace [nsbaci](#)
Root namespace for the nsbaci application.

Typedefs

- using [nsbaci::types::Shape](#)
Variant type for all drawable shapes.

Enumerations

- enum class [nsbaci::types::DrawCommandType](#) {
Clear , **SetColor** , **SetPosition** , **DrawShape** ,
Fill , **SetLineWidth** , **Refresh** }
Types of drawing commands that can be executed.
- enum class **StandardPosition** {
TopLeft , **TopCenter** , **TopRight** , **CenterLeft** ,
Center , **CenterRight** , **BottomLeft** , **BottomCenter** ,
BottomRight }

Functions

- [Point nsbaci::types::resolvePosition](#) (StandardPosition pos, [Size](#) canvasSize)
Resolve a standard position to actual coordinates.

Variables

- constexpr [Color](#) [nsbaci::types::Colors::BLACK](#) {0, 0, 0}
- constexpr [Color](#) [nsbaci::types::Colors::WHITE](#) {255, 255, 255}
- constexpr [Color](#) [nsbaci::types::Colors::RED](#) {255, 0, 0}
- constexpr [Color](#) [nsbaci::types::Colors::GREEN](#) {0, 255, 0}
- constexpr [Color](#) [nsbaci::types::Colors::BLUE](#) {0, 0, 255}
- constexpr [Color](#) [nsbaci::types::Colors::YELLOW](#) {255, 255, 0}
- constexpr [Color](#) [nsbaci::types::Colors::CYAN](#) {0, 255, 255}
- constexpr [Color](#) [nsbaci::types::Colors::MAGENTA](#) {255, 0, 255}
- constexpr [Color](#) [nsbaci::types::Colors::ORANGE](#) {255, 165, 0}
- constexpr [Color](#) [nsbaci::types::Colors::PINK](#) {255, 192, 203}
- constexpr [Color](#) [nsbaci::types::Colors::PURPLE](#) {128, 0, 128}
- constexpr [Color](#) [nsbaci::types::Colors::GRAY](#) {128, 128, 128}
- constexpr [Color](#) [nsbaci::types::Colors::LIGHT_GRAY](#) {192, 192, 192}
- constexpr [Color](#) [nsbaci::types::Colors::DARK_GRAY](#) {64, 64, 64}
- constexpr [Color](#) [nsbaci::types::Colors::BROWN](#) {139, 69, 19}

9.69.1 Detailed Description

Type definitions for drawing-related operations.

This header provides type definitions for the drawing system including colors, positions, shapes, and drawable interfaces.

Author

Nicolás Serrano García

Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

9.70 drawingTypes.h

[Go to the documentation of this file.](#)

```

00001
00012
00013 #ifndef NSBACI_TYPES_DRAWINGTYPES_H
00014 #define NSBACI_TYPES_DRAWINGTYPES_H
00015
00016 #include <cstdint>
00017 #include <memory>
00018 #include <string>
00019 #include <variant>
00020
00025 namespace nsbaci::types {
00026
00031 struct Color {
00032     uint8_t r = 0;
00033     uint8_t g = 0;
00034     uint8_t b = 0;
00035     uint8_t a = 255; // Alpha channel for transparency
00036
00037     constexpr Color() = default;
00038     constexpr Color(uint8_t red, uint8_t green, uint8_t blue, uint8_t alpha = 255)
00039         : r(red), g(green), b(blue), a(alpha) {}
00040
00041     constexpr bool operator==(const Color& other) const {
00042         return r == other.r && g == other.g && b == other.b && a == other.a;
00043     }
00044 };
00045
00046 // ===== Predefined Colors =====
00047 namespace Colors {
00048     inline constexpr Color BLACK{0, 0, 0};
00049     inline constexpr Color WHITE{255, 255, 255};
00050     inline constexpr Color RED{255, 0, 0};
00051     inline constexpr Color GREEN{0, 255, 0};
00052     inline constexpr Color BLUE{0, 0, 255};
00053     inline constexpr Color YELLOW{255, 255, 0};
00054     inline constexpr Color CYAN{0, 255, 255};
00055     inline constexpr Color MAGENTA{255, 0, 255};
00056     inline constexpr Color ORANGE{255, 165, 0};
00057     inline constexpr Color PINK{255, 192, 203};
00058     inline constexpr Color PURPLE{128, 0, 128};
00059     inline constexpr Color GRAY{128, 128, 128};
00060     inline constexpr Color LIGHT_GRAY{192, 192, 192};
00061     inline constexpr Color DARK_GRAY{64, 64, 64};
00062     inline constexpr Color BROWN{139, 69, 19};
00063 } // namespace Colors
00064
00069 struct Point {
00070     int32_t x = 0;
00071     int32_t y = 0;
00072
00073     Point() = default;
00074     Point(int32_t px, int32_t py) : x(px), y(py) {}
00075
00076     bool operator==(const Point& other) const {

```

```

00077     return x == other.x && y == other.y;
00078 }
00079
00080 Point operator+(const Point& other) const {
00081     return Point(x + other.x, y + other.y);
00082 }
00083
00084 Point operator-(const Point& other) const {
00085     return Point(x - other.x, y - other.y);
00086 }
00087 };
00088
00093 struct Size {
00094     int32_t width = 0;
00095     int32_t height = 0;
00096
00097     Size() = default;
00098     Size(int32_t w, int32_t h) : width(w), height(h) {}
00099 };
00100
00101 // ===== Shape Definitions =====
00102
00107 struct Circle {
00108     Point center;
00109     int32_t radius = 0;
00110     bool filled = false;
00111
00112     Circle() = default;
00113     Circle(Point c, int32_t r, bool fill = false)
00114         : center(c), radius(r), filled(fill) {}
00115 };
00116
00121 struct Rectangle {
00122     Point position; // Top-left corner
00123     Size size;
00124     bool filled = false;
00125
00126     Rectangle() = default;
00127     Rectangle(Point pos, Size s, bool fill = false)
00128         : position(pos), size(s), filled(fill) {}
00129     Rectangle(int32_t x, int32_t y, int32_t w, int32_t h, bool fill = false)
00130         : position(x, y), size(w, h), filled(fill) {}
00131 };
00132
00137 struct Triangle {
00138     Point p1, p2, p3;
00139     bool filled = false;
00140
00141     Triangle() = default;
00142     Triangle(Point a, Point b, Point c, bool fill = false)
00143         : p1(a), p2(b), p3(c), filled(fill) {}
00144 };
00145
00150 struct Line {
00151     Point start;
00152     Point end;
00153     int32_t thickness = 1;
00154
00155     Line() = default;
00156     Line(Point s, Point e, int32_t t = 1) : start(s), end(e), thickness(t) {}
00157 };
00158
00163 struct Ellipse {
00164     Point center;
00165     int32_t radiusX = 0;
00166     int32_t radiusY = 0;
00167     bool filled = false;
00168
00169     Ellipse() = default;
00170     Ellipse(Point c, int32_t rx, int32_t ry, bool fill = false)
00171         : center(c), radiusX(rx), radiusY(ry), filled(fill) {}
00172 };
00173
00178 struct Pixel {
00179     Point position;
00180
00181     Pixel() = default;
00182     explicit Pixel(Point p) : position(p) {}
00183     Pixel(int32_t x, int32_t y) : position(x, y) {}
00184 };
00185
00190 struct DrawText {
00191     Point position;
00192     std::string content;
00193     int32_t fontSize = 12;
00194
00195     DrawText() = default;

```

```

00196 DrawText(Point p, std::string text, int32_t size = 12)
00197     : position(p), content(std::move(text)), fontSize(size) {}
00198 };
00199
00203 using Shape =
00204     std::variant<Circle, Rectangle, Triangle, Line, Ellipse, Pixel, DrawText>;
00205
00210 struct Drawable {
00211     Shape shape;
00212     Color color;
00213     bool visible = true;
00214     int32_t zIndex = 0; // For layering
00215
00216     Drawable() = default;
00217     Drawable(Shape s, Color c, bool vis = true, int32_t z = 0)
00218         : shape(std::move(s)), color(c), visible(vis), zIndex(z) {}
00219 };
00220
00221 // ===== Drawing Commands =====
00222
00227 enum class DrawCommandType {
00228     Clear, // Clear the canvas
00229     SetColor, // Set current drawing color
00230     SetPosition, // Set current drawing position
00231     DrawShape, // Draw a shape
00232     Fill, // Fill the canvas with current color
00233     SetLineWidth, // Set line thickness
00234     Refresh // Force refresh/redraw
00235 };
00236
00241 struct DrawCommand {
00242     DrawCommandType type;
00243     Color color; // For SetColor
00244     Point position; // For SetPosition
00245     Shape shape; // For DrawShape
00246     int32_t lineWidth = 1; // For SetLineWidth
00247
00248     // Factory methods for convenience
00249     static DrawCommand clear() {
00250         DrawCommand cmd;
00251         cmd.type = DrawCommandType::Clear;
00252         cmd.color = Color(255, 255, 255); // Default white
00253         return cmd;
00254     }
00255
00256     static DrawCommand clearWithColor(Color c) {
00257         DrawCommand cmd;
00258         cmd.type = DrawCommandType::Clear;
00259         cmd.color = c;
00260         return cmd;
00261     }
00262
00263     static DrawCommand setColor(Color c) {
00264         DrawCommand cmd;
00265         cmd.type = DrawCommandType::SetColor;
00266         cmd.color = c;
00267         return cmd;
00268     }
00269
00270     static DrawCommand setPosition(Point p) {
00271         DrawCommand cmd;
00272         cmd.type = DrawCommandType::SetPosition;
00273         cmd.position = p;
00274         return cmd;
00275     }
00276
00277     static DrawCommand drawShape(Shape s, Color c) {
00278         DrawCommand cmd;
00279         cmd.type = DrawCommandType::DrawShape;
00280         cmd.shape = std::move(s);
00281         cmd.color = c;
00282         return cmd;
00283     }
00284
00285     static DrawCommand fill(Color c) {
00286         DrawCommand cmd;
00287         cmd.type = DrawCommandType::Fill;
00288         cmd.color = c;
00289         return cmd;
00290     }
00291
00292     static DrawCommand setLineWidth(int32_t width) {
00293         DrawCommand cmd;
00294         cmd.type = DrawCommandType::SetLineWidth;
00295         cmd.lineWidth = width;
00296         return cmd;
00297     }

```



```

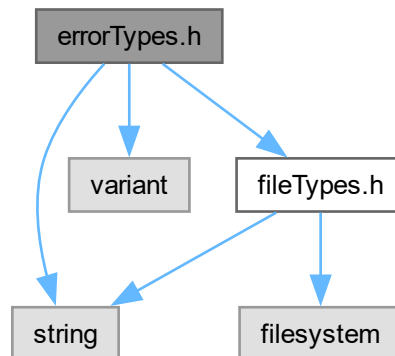
00298
00299     static DrawCommand refresh() {
00300         DrawCommand cmd;
00301         cmd.type = DrawCommandType::Refresh;
00302         return cmd;
00303     }
00304 };
00305
00306 // ===== Canvas Configuration =====
00307
00312 struct CanvasConfig {
00313     Size size{800, 600}; // Default canvas size
00314     Color backgroundColor{255, 255, 255}; // Default white background
00315     std::string title = "NSBACI Canvas";
00316 };
00317
00318 // ===== Predefined Positions =====
00319 // These are resolved at runtime based on canvas size
00320 enum class StandardPosition {
00321     TopLeft,
00322     TopCenter,
00323     TopRight,
00324     CenterLeft,
00325     Center,
00326     CenterRight,
00327     BottomLeft,
00328     BottomCenter,
00329     BottomRight
00330 };
00331
00338 inline Point resolvePosition(StandardPosition pos, Size canvasSize) {
00339     switch (pos) {
00340         case StandardPosition::TopLeft:
00341             return Point(0, 0);
00342         case StandardPosition::TopCenter:
00343             return Point(canvasSize.width / 2, 0);
00344         case StandardPosition::TopRight:
00345             return Point(canvasSize.width, 0);
00346         case StandardPosition::CenterLeft:
00347             return Point(0, canvasSize.height / 2);
00348         case StandardPosition::Center:
00349             return Point(canvasSize.width / 2, canvasSize.height / 2);
00350         case StandardPosition::CenterRight:
00351             return Point(canvasSize.width, canvasSize.height / 2);
00352         case StandardPosition::BottomLeft:
00353             return Point(0, canvasSize.height);
00354         case StandardPosition::BottomCenter:
00355             return Point(canvasSize.width / 2, canvasSize.height);
00356         case StandardPosition::BottomRight:
00357             return Point(canvasSize.width, canvasSize.height);
00358         default:
00359             return Point(0, 0);
00360     }
00361 }
00362
00363 } // namespace nsbaci::types
00364
00365 #endif // NSBACI_TYPES_DRAWINGTYPES_H

```

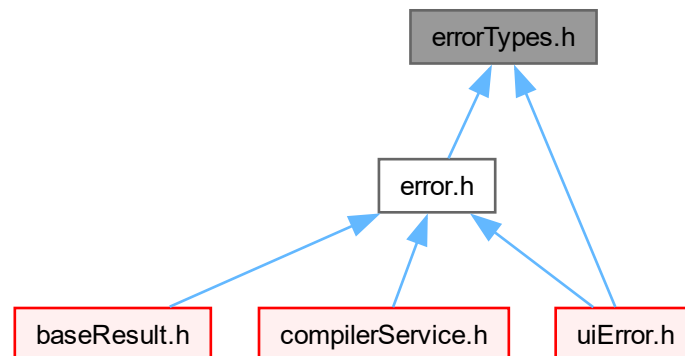
9.71 errorTypes.h File Reference

Type definitions for error-related structures.

Include dependency graph for `errorTypes.h`:



This graph shows which files directly or indirectly include this file:



Classes

- struct `nsbaci::types::ErrorBase`
Base structure containing common error properties.
- struct `nsbaci::types::CompileError`
Error payload for compilation errors.
- struct `nsbaci::types::SaveError`
Error payload for file save errors.
- struct `nsbaci::types::LoadError`
Error payload for file load errors.
- struct `nsbaci::types::RuntimeError`
Error payload for runtime errors.

Namespaces

- namespace [nsbaci::types](#)
Type definitions namespace for nsbaci (runtime-specific).
- namespace [nsbaci](#)
Root namespace for the nsbaci application.

Typedefs

- using **nsbaci::types::ErrorMessage** = std::string
- using [nsbaci::types::ErrorPayload](#)
Variant type for all possible error payloads.

Enumerations

- enum class [nsbaci::types::ErrSeverity](#) { **Warning** , **Error** , **Fatal** }
Severity levels for errors.
- enum class [nsbaci::types::ErrType](#) {
emptyPath , **invalidPath** , **invalidExtension** , **directoryNotFound** ,
fileNotFound , **notARegularFile** , **permissionDenied** , **openFailed** ,
readFailed , **writeFailed** , **compilationError** , **unknown** }
Types of errors that can occur in the application.

9.71.1 Detailed Description

Type definitions for error-related structures.

This header provides type aliases and enums used by the Error module and other components that handle errors.

Author

Nicolás Serrano García

Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

9.72 errorTypes.h

[Go to the documentation of this file.](#)

```

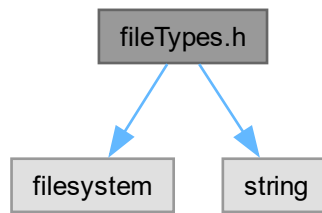
00001
00012
00013 #ifndef NSBACI_TYPES_ERRORTYPES_H
00014 #define NSBACI_TYPES_ERRORTYPES_H
00015
00016 #include <string>
00017 #include <variant>
00018
00019 #include "fileTypes.h"
00020
00025 namespace nsbaci::types {
00026
00031 enum class ErrSeverity { Warning, Error, Fatal };
00032
00037 enum class ErrType {
00038     // File path errors
00039     emptyPath,           // Path string is empty
00040     invalidPath,         // Path is malformed or invalid
00041     invalidExtension,    // File does not have .nsb extension
00042     directoryNotFound,   // Parent directory doesn't exist
00043     fileNotFound,        // File doesn't exist
00044     notARegularFile,     // Path points to directory, symlink, etc.
00045
00046     // Permission errors
00047     permissionDenied,    // No read/write access
00048
00049     // I/O errors
00050     openFailed,          // Could not open file
00051     readFailed,          // Error while reading
00052     writeFailed,         // Error while writing
00053
00054     // Compilation errors
00055     compilationError,    // Syntax or semantic error during compilation
00056
00057     // Generic
00058     unknown              // Unspecified error
00059 };
00060
00061 using ErrorMessage = std::string;
00062
00067 struct ErrorBase {
00068     // this severity serves as an indicator to what icon to use in the error, the
00069     // string that appears in the top of the dialog, additional buttons for things
00070     // like restart when the error is fatal...
00071     ErrSeverity severity;
00072     ErrorMessage message;
00073     // useful to show the reason for things. For example, if when trying to save a
00074     // file to a sensible location, if the app doesn't have privileges, the ui can
00075     // show a message of type: "/Error/ X could not be saved in Y. Reason:
00076     // Permission Denied. Try starting the app in admin mode. [Ok]"
00077     ErrType type;
00078 };
00079
00084 struct CompileError {
00085     int line = 0;
00086     int column = 0;
00087 };
00088
00093 struct SaveError {
00094     File associatedFile;
00095 };
00096
00101 struct LoadError {
00102     File associatedFile;
00103 };
00104
00109 struct RuntimeError {};
00110
00117 using ErrorPayload =
00118     std::variant<SaveError, LoadError, CompileError, RuntimeError>;
00119
00120 } // namespace nsbaci::types
00121
00122 #endif // NSBACI_TYPES_ERRORTYPES_H

```

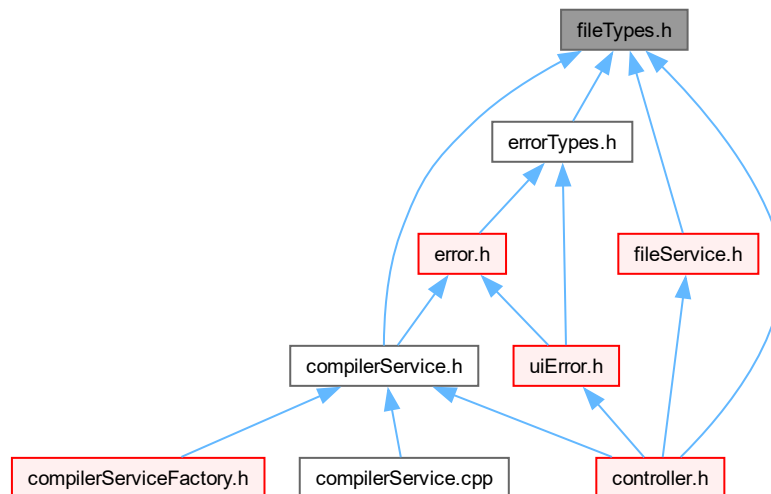
9.73 fileTypes.h File Reference

Type definitions for file-related operations.

Include dependency graph for fileType.h:



This graph shows which files directly or indirectly include this file:



Namespaces

- namespace `nsbaci::types`
Type definitions namespace for nsbaci (runtime-specific).
- namespace `nsbaci`
Root namespace for the nsbaci application.

Typedefs

- using `nsbaci::types::Text` = `std::string`
Alias for text content (file contents, source code, etc.).
- using `nsbaci::types::File` = `fs::path`
Alias for file system paths.

9.73.1 Detailed Description

Type definitions for file-related operations.

This header provides type aliases used by the FileService and other components that work with files.

Author

Nicolás Serrano García

Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

9.74 fileTypes.h

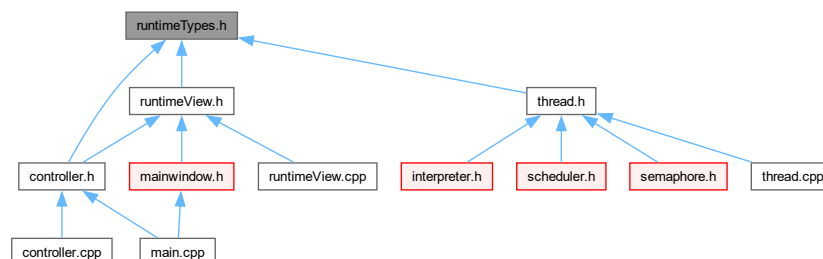
[Go to the documentation of this file.](#)

```
00001
00012
00013 #ifndef NSBACI_TYPES_FILETYPES_H
00014 #define NSBACI_TYPES_FILETYPES_H
00015
00016 #include <filesystem>
00017 #include <string>
00018
00023 namespace nsbaci::types {
00024
00025 namespace fs = std::filesystem;
00026
00030 using Text = std::string;
00031
00035 using File = fs::path;
00036
00037 } // namespace nsbaci::types
00038
00039 #endif // NSBACI_TYPES_FILETYPES_H
```

9.75 runtimeTypes.h File Reference

Type definitions for runtime-related operations.

This graph shows which files directly or indirectly include this file:



Namespaces

- namespace `nsbaci::types`
Type definitions namespace for nsbaci (runtime-specific).
- namespace `nsbaci`
Root namespace for the nsbaci application.

Typedefs

- using `nsbaci::types::ThreadID` = unsigned long long int
- using `nsbaci::types::Priority` = unsigned long int

Enumerations

- enum class `nsbaci::types::ThreadState` {
`nsbaci::types::Ready` , `nsbaci::types::Running` , `nsbaci::types::Blocked` , `nsbaci::types::Waiting` ,
`nsbaci::types::IO` , `nsbaci::types::Terminated` }

9.75.1 Detailed Description

Type definitions for runtime-related operations.

This header provides type aliases used by the RuntimeService and other components that work with runtime execution.

Author

Nicolás Serrano García

Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

9.76 runtimeTypes.h

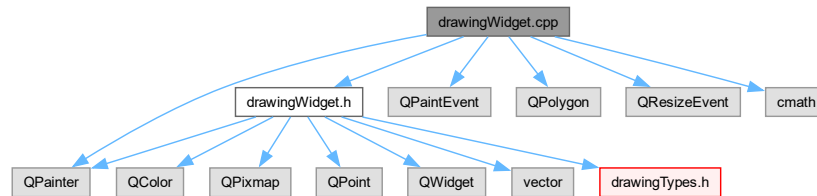
[Go to the documentation of this file.](#)

```
00001
00012
00013 #ifndef NSBACI_TYPES_RUNTIMEYPES_H
00014 #define NSBACI_TYPES_RUNTIMEYPES_H
00015
00020 namespace nsbaci::types {
00021
00022 using ThreadID = unsigned long long int;
00023
00024 using Priority = unsigned long int;
00025
00026 enum class ThreadState {
00027     Ready,
00028     Running,
00029     Blocked,
00030     Waiting,
00031     IO,
00032     Terminated
00033 };
00034
00035 } // namespace nsbaci::types
00036
00037 #endif // NSBACI_TYPES_RUNTIMEYPES_H
```

9.77 drawingWidget.cpp File Reference

DrawingWidget class implementation for nsbaci.

Include dependency graph for drawingWidget.cpp:



Namespaces

- namespace `nsbaci`
Root namespace for the nsbaci application.
- namespace `nsbaci::ui`
User interface namespace for nsbaci.

9.77.1 Detailed Description

DrawingWidget class implementation for nsbaci.

Author

Nicolás Serrano García

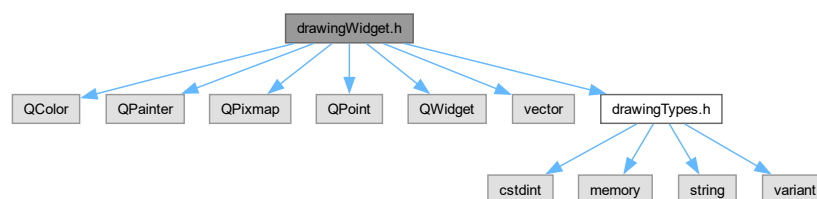
Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

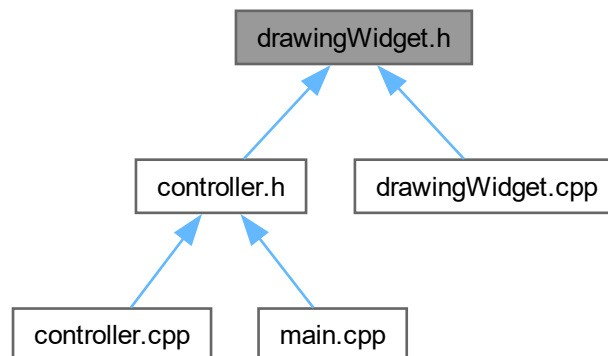
9.78 drawingWidget.h File Reference

DrawingWidget class declaration for nsbaci.

Include dependency graph for drawingWidget.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [nsbaci::ui::DrawingWidget](#)
Qt widget that provides a drawing canvas.

Namespaces

- namespace [nsbaci::ui](#)
User interface namespace for nsbaci.
- namespace [nsbaci](#)
Root namespace for the nsbaci application.

9.78.1 Detailed Description

DrawingWidget class declaration for nsbaci.

This widget provides a canvas for drawing shapes and graphics. It receives drawing commands from the Drawing↔Service via signals.

Author

Nicolás Serrano García

Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

9.79 drawingWidget.h

[Go to the documentation of this file.](#)

```

00001
00012
00013 #ifndef NSBACI_UI_DRAWINGWIDGET_H
00014 #define NSBACI_UI_DRAWINGWIDGET_H
00015
00016 #include <QColor>
00017 #include <QPainter>
00018 #include <QPixmap>
00019 #include <QPoint>
00020 #include <QWidget>
00021 #include <vector>
00022
00023 #include "drawingTypes.h"
00024
00029 namespace nsbaci::ui {
00030
00039 class DrawingWidget : public QWidget {
00040     Q_OBJECT
00041
00042 public:
00047     explicit DrawingWidget(QWidget* parent = nullptr);
00048
00052     ~DrawingWidget() override = default;
00053
00058     nsbaci::types::Size getCanvasSize() const;
00059
00064     QSize minimumSizeHint() const override;
00065
00070     QSize sizeHint() const override;
00071
00072 public slots:
00073     // ===== Drawing Command Slots =====
00074
00079     void onDrawCommand(const nsbaci::types::DrawCommand& command);
00080
00085     void onDrawRequested(const nsbaci::types::Drawable& drawable);
00086
00091     void onClearRequested(const nsbaci::types::Color& color);
00092
00096     void onRefreshRequested();
00097
00102     void onCanvasSizeChanged(const nsbaci::types::Size& size);
00103
00104     // ===== Direct Drawing Slots =====
00105
00109     void setColor(uint8_t r, uint8_t g, uint8_t b, uint8_t a = 255);
00110
00114     void clear();
00115
00119     void drawCircle(int32_t centerX, int32_t centerY, int32_t radius,
00120                     bool filled = false);
00121
00125     void drawRectangle(int32_t x, int32_t y, int32_t width, int32_t height,
00126                        bool filled = false);
00127
00131     void drawTriangle(int32_t x1, int32_t y1, int32_t x2, int32_t y2, int32_t x3,
00132                       int32_t y3, bool filled = false);
00133
00137     void drawLine(int32_t x1, int32_t y1, int32_t x2, int32_t y2);
00138
00142     void drawEllipse(int32_t centerX, int32_t centerY, int32_t radiusX,
00143                      int32_t radiusY, bool filled = false);
00144
00148     void drawPixel(int32_t x, int32_t y);
00149
00153     void drawText(int32_t x, int32_t y, const QString& text,
00154                   int32_t fontSize = 12);
00155
00159     void setLineWidth(int32_t width);
00160
00164     void reset();
00165
00166 protected:
00171     void paintEvent(QPaintEvent* event) override;
00172
00177     void resizeEvent(QResizeEvent* event) override;
00178
00179 private:
00185     void drawShapeInternal(const nsbaci::types::Shape& shape,
00186                           const nsbaci::types::Color& color);
00187
00191     static QColor toQColor(const nsbaci::types::Color& color);

```

```

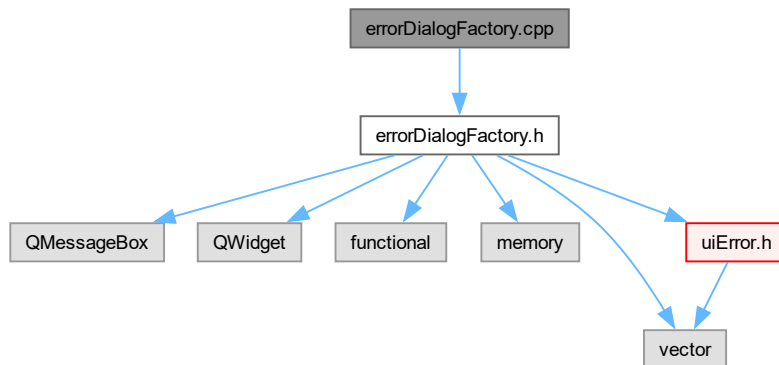
00192
00196 void ensureCanvas();
00197
00198 QPixmap canvas_; // The drawing buffer
00199 QColor currentColor_{Qt::black}; // Current drawing color
00200 QColor backgroundColor_{Qt::white}; // Background color
00201 int lineWidth_ = 1; // Current line width
00202 nsbaci::types::Size canvasSize_{800, 600}; // Canvas dimensions
00203 };
00204
00205 } // namespace nsbaci::ui
00206
00207 #endif // NSBACI_UI_DRAWINGWIDGET_H

```

9.80 errorDialogFactory.cpp File Reference

Implementation of ErrorDialogFactory.

Include dependency graph for errorDialogFactory.cpp:



Namespaces

- namespace `nsbaci`
Root namespace for the nsbaci application.
- namespace `nsbaci::ui`
User interface namespace for nsbaci.

9.80.1 Detailed Description

Implementation of ErrorDialogFactory.

Author

Nicolás Serrano García

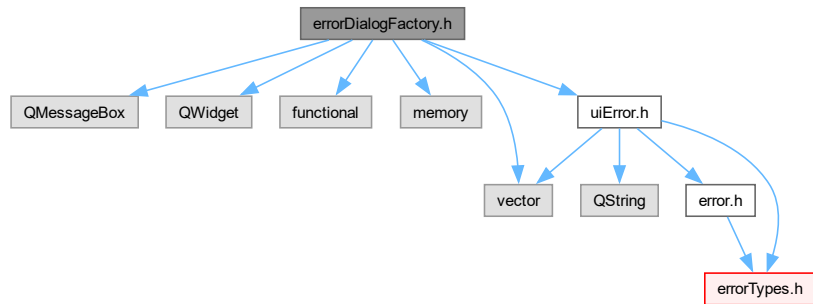
Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

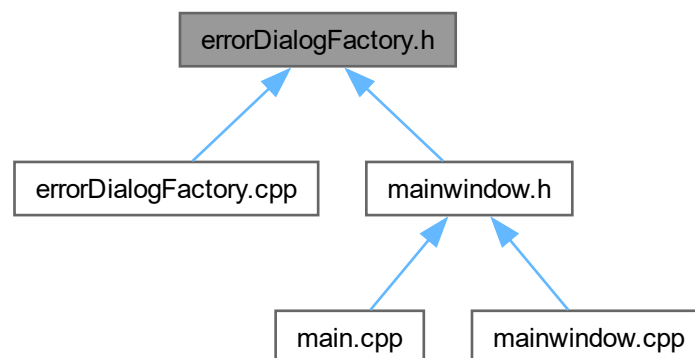
9.81 errorDialogFactory.h File Reference

Factory for creating error dialogs from `UIError` objects.

Include dependency graph for `errorDialogFactory.h`:



This graph shows which files directly or indirectly include this file:



Classes

- class `nsbaci::ui::ErrorDialogFactory`
Factory for creating error dialogs from `UIError` objects.

Namespaces

- namespace `nsbaci`
Root namespace for the `nsbaci` application.
- namespace `nsbaci::ui`
User interface namespace for `nsbaci`.

9.81.1 Detailed Description

Factory for creating error dialogs from UIError objects.

This module provides a factory that converts UIError objects into ready-to-show QMessageBox dialogs. It is private to the UI layer.

9.81.1.1 Overview

The ErrorDialogFactory operates in two modes:

9.81.1.1.1 1. Deferred Mode (Factory Pattern)

Returns a **DialogInvoker** - a callable (`std::function`) that encapsulates all dialog data but does NOT show the dialog immediately. The caller decides when to invoke it. This is similar to a "lazy evaluation" or "thunk" pattern.

```
// Create the invoker (dialog is NOT shown yet)
auto dialogInvoker = ErrorDialogFactory::getDialogFromUIError(error, this);

// ... do other work, validation, logging, etc.

// Show the dialog when ready (blocks until user clicks)
QMessageBox::StandardButton clicked = dialogInvoker();

// React to user choice
if (clicked == QMessageBox::Close) {
    QApplication::quit();
}
```

9.81.1.1.2 2. Immediate Mode (Convenience)

Shows the dialog immediately and returns the result. Internally, this creates an invoker and calls it right away. Use this when you don't need to defer the dialog display.

```
// Show immediately - blocks until user clicks
ErrorDialogFactory::showError(error, this);

// Or capture the result
auto clicked = ErrorDialogFactory::showError(error, this);
```

9.81.1.2 Why Return Callables?

The DialogInvoker pattern provides:

- **Separation of concerns:** Factory knows HOW to build, caller knows WHEN to show
- **Deferred execution:** Prepare dialogs ahead of time, show when appropriate
- **User response handling:** The return value indicates which button was clicked
- **Flexibility:** Batch prepare multiple dialogs, show conditionally, etc.

The callable captures all necessary data by value, so the original UIError can be destroyed before the dialog is shown.

Author

Nicolás Serrano García

Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

9.82 errorDialogFactory.h

[Go to the documentation of this file.](#)

```

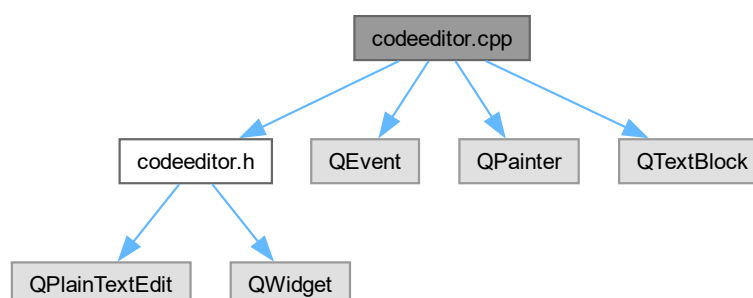
00001
00065
00066 #ifndef NSBACI_ERRORDIALOGFACTORY_H
00067 #define NSBACI_ERRORDIALOGFACTORY_H
00068
00069 #include <QMessageBox>
00070 #include <QWidget>
00071 #include <functional>
00072 #include <memory>
00073 #include <vector>
00074
00075 #include "uiError.h"
00076
00077 namespace nsbaci::ui {
00078
00090 class ErrorDialogFactory {
00091 public:
00102     using DialogInvoker = std::function<QMessageBox::StandardButton()>;
00103
00114     static DialogInvoker getDialogFromUIError(const UIError& error,
00115                                             QWidget* parent = nullptr);
00116
00124     static std::vector<DialogInvoker> getDialogsFromUIErrors(
00125         const std::vector<UIError>& errors, QWidget* parent = nullptr);
00126
00135     static DialogInvoker getSuccessDialog(const QString& title,
00136                                         const QString& message,
00137                                         QWidget* parent = nullptr);
00138
00148     static void showErrors(const std::vector<UIError>& errors,
00149                          QWidget* parent = nullptr);
00150
00158     static QMessageBox::StandardButton showError(const UIError& error,
00159                                                  QWidget* parent = nullptr);
00160
00168     static void showSuccess(const QString& title, const QString& message,
00169                          QWidget* parent = nullptr);
00170
00171 private:
00175     static QMessageBox::Icon iconFromSeverity(
00176         nsbaci::types::ErrSeverity severity);
00177 };
00178
00179 } // namespace nsbaci::ui
00180
00181 #endif // NSBACI_ERRORDIALOGFACTORY_H

```

9.83 codeeditor.cpp File Reference

Implementation of the [CodeEditor](#) class with line numbers.

Include dependency graph for codeeditor.cpp:



9.83.1 Detailed Description

Implementation of the [CodeEditor](#) class with line numbers.

Author

Nicolás Serrano García

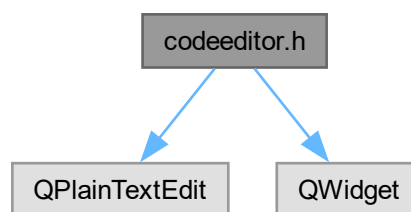
Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

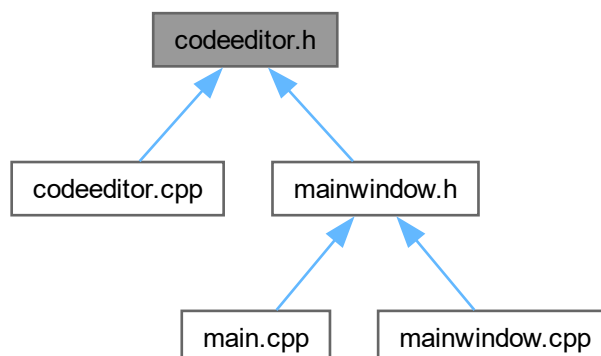
9.84 codeeditor.h File Reference

[CodeEditor](#) class with line numbers for nsbaci.

Include dependency graph for codeeditor.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [CodeEditor](#)
- class [LineNumberArea](#)

9.84.1 Detailed Description

[CodeEditor](#) class with line numbers for nsbaci.

Author

Nicolás Serrano García

Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

9.85 codeeditor.h

[Go to the documentation of this file.](#)

```

00001
00009
00010 #ifndef CODEEDITOR_H
00011 #define CODEEDITOR_H
00012
00013 #include <QPlainTextEdit>
00014 #include <QWidget>
00015
00016 class LineNumberArea;
00017
00018 class CodeEditor : public QPlainTextEdit {
00019     Q_OBJECT
00020
00021 public:
00022     explicit CodeEditor(QWidget* parent = nullptr);
00023
00024     void lineNumberAreaPaintEvent(QPaintEvent* event);
00025     int lineNumberAreaWidth();
00026
00028     void setLineNumbersVisible(bool visible);
00029     bool lineNumbersVisible() const;
00030
00033     void setLightTheme(bool light);
00034
00035 protected:
00036     void resizeEvent(QResizeEvent* event) override;
00037     void changeEvent(QEvent* event) override;
00038
00039 private slots:
00040     void updateLineNumberAreaWidth(int newBlockCount);
00041     void highlightCurrentLine();
00042     void updateLineNumberArea(const QRect& rect, int dy);
00043
00044 private:
00045     LineNumberArea* lineNumberArea;
00046     bool showLineNumbers = true;
00047     bool lightTheme = false;
00048 };
00049
00050 class LineNumberArea : public QWidget {
00051 public:
00052     explicit LineNumberArea(CodeEditor* editor)
00053         : QWidget(editor), codeEditor(editor) {}
00054
00055     QSize sizeHint() const override {
00056         return QSize(codeEditor->lineNumberAreaWidth(), 0);
00057     }
00058
00059 protected:

```


9.86 mainwindow.cpp File Reference

Include dependency graph for mainwindow.cpp:

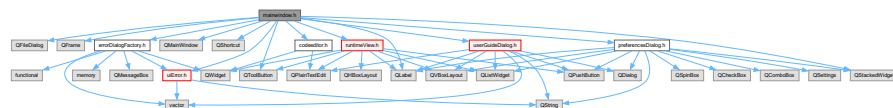


Author

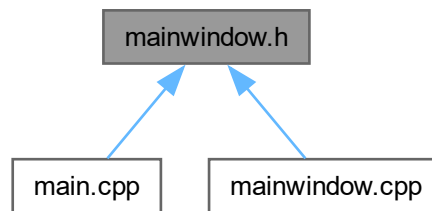
Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

Include dependency graph for mainwindow.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [MainWindow](#)

9.87.1 Detailed Description

Main window class declaration for nsbaci.

Author

Nicolás Serrano García

Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

9.88 mainwindow.h

[Go to the documentation of this file.](#)

```
00001
00009
00010 #ifndef MAINWINDOW_H
00011 #define MAINWINDOW_H
00012
00013 #include <QFileDialog>
00014 #include <QFrame>
00015 #include <QLabel>
00016 #include <QMainWindow>
00017 #include <QShortcut>
00018 #include <QStackedWidget>
00019 #include <QToolButton>
00020
00021 #include "codeeditor.h"
00022 #include "errorDialogFactory.h"
00023 #include "preferencesDialog.h"
00024 #include "runtimeView.h"
00025 #include "uiError.h"
00026 #include "userGuideDialog.h"
00027
00028 QT_BEGIN_NAMESPACE
00029 namespace Ui {
00030 class MainWindow;
```

```

00031 }
00032 QT_END_NAMESPACE
00033
00034 class MainWindow : public QMainWindow {
00035     Q_OBJECT
00036
00037 public:
00038     explicit MainWindow(QWidget* parent = nullptr);
00039     ~MainWindow() override = default;
00040
00041 signals:
00042     void saveRequested(const QString& filePath, const QString& contents);
00043     void openRequested(const QString& filePath);
00044     void compileRequested(const QString& contents);
00045     void runRequested();
00046
00047     // Runtime control signals
00048     void stepRequested();
00049     void stepThreadRequested(nsbaci::types::ThreadID threadId);
00050     void runContinueRequested();
00051     void pauseRequested();
00052     void resetRequested();
00053     void stopRequested();
00054     void inputProvided(const QString& input);
00055
00056 public slots:
00057     void setEditorContents(const QString& contents);
00058     void setStatusMessage(const QString& message);
00059     void setCurrentFile(const QString& fileName, bool modified = false);
00060
00061     // Controller response slots
00062     void onSaveSucceeded();
00063     void onSaveFailed(std::vector<nsbaci::UIError> errors);
00064     void onLoadSucceeded(const QString& contents);
00065     void onLoadFailed(std::vector<nsbaci::UIError> errors);
00066     void onCompileSucceeded();
00067     void onCompileFailed(std::vector<nsbaci::UIError> errors);
00068
00069     // Runtime slots
00070     void onRunStarted(const QString& programName);
00071     void onRuntimeStateChanged(bool running, bool halted);
00072     void onThreadsUpdated(const std::vector<nsbaci::ui::ThreadInfo>& threads);
00073     void onVariablesUpdated(
00074         const std::vector<nsbaci::ui::VariableInfo>& variables);
00075     void onOutputReceived(const QString& output);
00076     void onInputRequested(const QString& prompt);
00077
00078 private slots:
00079     // File menu
00080     void onNew();
00081     void onSave();
00082     void onSaveAs();
00083     void onOpen();
00084     void onExit();
00085     void onCompile();
00086     void onRun();
00087
00088     // Edit menu
00089     void onUndo();
00090     void onRedo();
00091     void onCut();
00092     void onCopy();
00093     void onPaste();
00094     void onSelectAll();
00095
00096     // View menu
00097     void onToggleSidebar();
00098     void onToggleFullscreen();
00099
00100     // Edit menu (preferences)
00101     void onPreferences();
00102
00103     // Help menu
00104     void onUserGuide();
00105     void onAbout();
00106
00107     // Editor
00108     void onTextChanged();
00109
00110     // Runtime view
00111     void onStopRuntime();
00112
00113 private:
00114     // Stacked widget for switching views
00115     QStackedWidget* centralStack = nullptr;
00116
00117     // Editor view container

```

```

00118     QWidget* editorView = nullptr;
00119
00120     // File info bar
00121     QFrame* fileInfoBar = nullptr;
00122     QLabel* fileNameLabel = nullptr;
00123     QLabel* fileModifiedIndicator = nullptr;
00124     QLabel* compileStatusIndicator = nullptr;
00125
00126     // Central widget
00127     CodeEditor* codeEditor = nullptr;
00128
00129     // Sidebar
00130     QFrame* sideBar = nullptr;
00131     QToolButton* compileButton = nullptr;
00132     QToolButton* runButton = nullptr;
00133
00134     // Runtime view
00135     nsbaci::ui::RuntimeView* runtimeView = nullptr;
00136
00137     // File actions
00138     QAction* actionNew = nullptr;
00139     QAction* actionSave = nullptr;
00140     QAction* actionSaveAs = nullptr;
00141     QAction* actionOpen = nullptr;
00142     QAction* actionExit = nullptr;
00143
00144     // Edit actions
00145     QAction* actionUndo = nullptr;
00146     QAction* actionRedo = nullptr;
00147     QAction* actionCut = nullptr;
00148     QAction* actionCopy = nullptr;
00149     QAction* actionPaste = nullptr;
00150     QAction* actionSelectAll = nullptr;
00151
00152     // View actions
00153     QAction* actionToggleSidebar = nullptr;
00154     QAction* actionFullscreen = nullptr;
00155
00156     // Edit actions (preferences)
00157     QAction* actionPreferences = nullptr;
00158
00159     // Build actions
00160     QAction* actionCompile = nullptr;
00161     QAction* actionRun = nullptr;
00162
00163     // Help actions
00164     QAction* actionUserGuide = nullptr;
00165     QAction* actionAbout = nullptr;
00166
00167     // State
00168     QString currentFileName; // Just the filename for display
00169     QString currentFilePath; // Full path for saving
00170     bool isModified = false;
00171     bool hasName = false;
00172     bool isCompiled =
00173         false; // True when program is compiled and code hasn't changed
00174
00175     // Current preferences
00176     nsbaci::ui::Preferences appPrefs;
00177
00178 private:
00179     void createCentralWidget();
00180     void createEditorView();
00181     void createRuntimeView();
00182     void createMenuBar();
00183     void createStatusBar();
00184     void setupShortcuts();
00185     void applyStyleSheet();
00186     void applyPreferences(const nsbaci::ui::Preferences& prefs);
00187     QString darkStyleSheet() const;
00188     QString lightStyleSheet() const;
00189
00190     void switchToEditor();
00191     void switchToRuntime();
00192 };
00193 #endif // MAINWINDOW_H

```

9.89 preferencesDialog.cpp File Reference

Implementation of the PreferencesDialog class.

Include dependency graph for preferencesDialog.cpp:



Namespaces

- namespace [nsbaci](#)
Root namespace for the nsbaci application.
- namespace [nsbaci::ui](#)
User interface namespace for nsbaci.

9.89.1 Detailed Description

Implementation of the PreferencesDialog class.

Author

Nicolás Serrano García

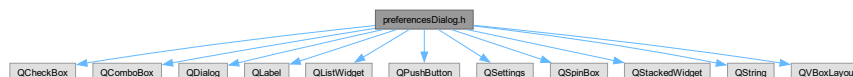
Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

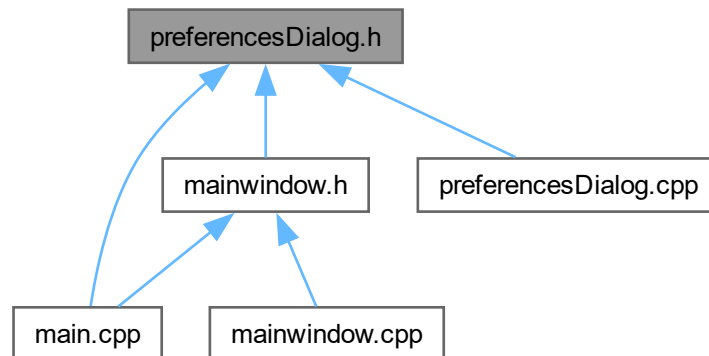
9.90 preferencesDialog.h File Reference

Preferences dialog for nsbaci application settings.

Include dependency graph for preferencesDialog.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct [nsbaci::ui::Preferences](#)
Holds all user-configurable preferences.
- class [nsbaci::ui::PreferencesDialog](#)
A dialog that allows the user to configure application preferences.

Namespaces

- namespace [nsbaci](#)
Root namespace for the nsbaci application.
- namespace [nsbaci::ui](#)
User interface namespace for nsbaci.

Enumerations

- enum class [nsbaci::ui::Theme](#) { **Dark** , **Light** }
Enumerates the available application themes.

9.90.1 Detailed Description

Preferences dialog for nsbaci application settings.

Author

Nicolás Serrano García

Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

9.91 preferencesDialog.h

[Go to the documentation of this file.](#)

```

00001
00009
00010 #ifndef PREFERENCESDIALOG_H
00011 #define PREFERENCESDIALOG_H
00012
00013 #include <QCheckBox>
00014 #include <QComboBox>
00015 #include <QDialog>
00016 #include <QLabel>
00017 #include <QListWidget>
00018 #include <QPushButton>
00019 #include <QSettings>
00020 #include <QSpinBox>
00021 #include <QStackedWidget>
00022 #include <QString>
00023 #include <QVBoxLayout>
00024
00025 namespace nsbaci::ui {
00026
00030 enum class Theme { Dark, Light };
00031
00038 struct Preferences {
00039     // Appearance
00040     Theme theme = Theme::Dark;
00041     int editorFontSize = 11;
00042
00043     // Editor
00044     int tabWidth = 4;
00045     bool showLineNumbers = true;
00046     bool wordWrap = false;
00047
00048     // General
00049     bool restoreLastFile = false;
00050     QString lastFilePath;
00051 };
00052
00060 class PreferencesDialog : public QDialog {
00061     Q_OBJECT
00062
00063 public:
00064     explicit PreferencesDialog(const Preferences& current,
00065                               QWidget* parent = nullptr);
00066     ~PreferencesDialog() override = default;
00067
00072     Preferences result() const;
00073
00074     // ---- Static persistence helpers ----
00075
00077     static Preferences load();
00078
00080     static void save(const Preferences& prefs);
00081
00082 signals:
00087     void preferencesChanged(const nsbaci::ui::Preferences& prefs);
00088
00089 private slots:
00090     void onCategoryChanged(int currentRow);
00091     void onAccept();
00092     void onRestoreDefaults();
00093
00094 private:
00095     void setupUi();
00096     QWidget* createAppearancePage();
00097     QWidget* createEditorPage();
00098     QWidget* createGeneralPage();
00099     void applyDialogStyle();
00100     void populateFromPreferences(const Preferences& prefs);
00101
00102     // UI elements
00103     QListWidget* categoryList = nullptr;
00104     QStackedWidget* pages = nullptr;
00105
00106     // Appearance widgets
00107     QComboBox* themeCombo = nullptr;
00108     QSpinBox* fontSizeSpin = nullptr;
00109
00110     // Editor widgets
00111     QComboBox* tabWidthCombo = nullptr;
00112     QCheckBox* lineNumbersCheck = nullptr;
00113     QCheckBox* wordWrapCheck = nullptr;
00114
00115     // General widgets

```

```

00116     QCheckBox* restoreLastFileCheck = nullptr;
00117
00118     QPushButton* okButton = nullptr;
00119     QPushButton* cancelButton = nullptr;
00120     QPushButton* defaultsButton = nullptr;
00121
00122     // Snapshot of the preferences we were opened with
00123     Preferences currentPrefs;
00124 };
00125
00126 } // namespace nsbaci::ui
00127
00128 #endif // PREFERENCESDIALOG_H

```

9.92 runtimeView.cpp File Reference

Implementation of the RuntimeView widget.

Include dependency graph for runtimeView.cpp:



Namespaces

- namespace `nsbaci`
Root namespace for the nsbaci application.
- namespace `nsbaci::ui`
User interface namespace for nsbaci.

9.92.1 Detailed Description

Implementation of the RuntimeView widget.

Author

Nicolás Serrano García

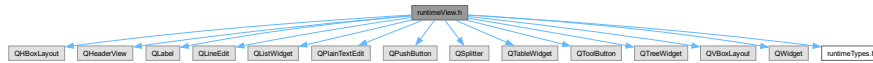
Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

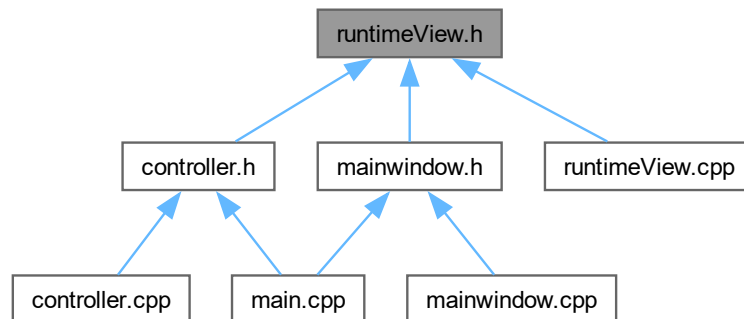
9.93 runtimeView.h File Reference

RuntimeView widget declaration for nsbaci.

Include dependency graph for runtimeView.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct [nsbaci::ui::ThreadInfo](#)
Information about a thread for display.
- struct [nsbaci::ui::VariableInfo](#)
Information about a variable for display.
- class [nsbaci::ui::RuntimeView](#)
Widget displaying runtime execution state.

Namespaces

- namespace [nsbaci](#)
Root namespace for the nsbaci application.
- namespace [nsbaci::ui](#)
User interface namespace for nsbaci.

9.93.1 Detailed Description

RuntimeView widget declaration for nsbaci.

This widget displays runtime information when executing a program: variables, threads, I/O console, and execution controls.

Author

Nicolás Serrano García

Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

9.94 runtimeView.h

[Go to the documentation of this file.](#)

```

00001
00012
00013 #ifndef NSBACI_RUNTIMEVIEW_H
00014 #define NSBACI_RUNTIMEVIEW_H
00015
00016 #include <QHBoxLayout>
00017 #include <QHeaderView>
00018 #include <QLabel>
00019 #include <QLineEdit>
00020 #include <QListWidget>
00021 #include <QPlainTextEdit>
00022 #include <QPushButton>
00023 #include <QSplitter>
00024 #include <QTableWidget>
00025 #include <QToolButton>
00026 #include <QTreeWidget>
00027 #include <QVBoxLayout>
00028 #include <QWidget>
00029
00030 #include "runtimeTypes.h"
00031
00032 namespace nsbaci::ui {
00033
00034 struct ThreadInfo {
00035     nsbaci::types::ThreadID id;
00036     nsbaci::types::ThreadState state;
00037     size_t pc;
00038     QString currentInstruction;
00039 };
00040
00041 struct VariableInfo {
00042     QString name;
00043     QString type;
00044     QString value;
00045     size_t address;
00046 };
00047
00048 class RuntimeView : public QWidget {
00049     Q_OBJECT
00050
00051 public:
00052     explicit RuntimeView(QWidget* parent = nullptr);
00053     ~RuntimeView() override = default;
00054
00055 signals:
00056     // Execution control signals
00057     void stepRequested();
00058     void stepThreadRequested(nsbaci::types::ThreadID threadId);
00059     void runRequested();
00060     void pauseRequested();
00061     void resetRequested();
00062     void stopRequested();
00063
00064     // I/O signals

```

```

00083 void inputProvided(const QString& input);
00084
00085 public slots:
00086 // Update display
00087 void updateThreads(const std::vector<ThreadInfo>& threads);
00088 void updateVariables(const std::vector<VariableInfo>& variables);
00089 void updateCurrentInstruction(const QString& instruction);
00090 void updateExecutionState(bool running, bool halted);
00091
00092 // I/O
00093 void appendOutput(const QString& text);
00094 void requestInput(const QString& prompt);
00095 void clearConsole();
00096
00097 // State
00098 void onProgramLoaded(const QString& programName);
00099 void onProgramHalted();
00100
00101 private slots:
00102 void onStepClicked();
00103 void onRunClicked();
00104 void onPauseClicked();
00105 void onResetClicked();
00106 void onStopClicked();
00107 void onInputSubmitted();
00108 void onThreadSelected(QTreeWidgetItem* item, int column);
00109
00110 private:
00111 void createUI();
00112 void createToolBar();
00113 void createThreadPanel();
00114 void createVariablePanel();
00115 void createConsolePanel();
00116 void applyStyleSheet();
00117
00118 // Toolbar
00119 QWidget* toolbar = nullptr;
00120 QToolButton* stepButton = nullptr;
00121 QToolButton* runButton = nullptr;
00122 QToolButton* pauseButton = nullptr;
00123 QToolButton* resetButton = nullptr;
00124 QToolButton* stopButton = nullptr;
00125 QLabel* statusLabel = nullptr;
00126
00127 // Thread panel
00128 QTreeWidgetItem* threadTree = nullptr;
00129
00130 // Variable panel
00131 QTableWidgetItem* variableTable = nullptr;
00132
00133 // Console panel
00134 QPlainTextEdit* consoleOutput = nullptr;
00135 QLineEdit* consoleInput = nullptr;
00136 QPushButton* inputSubmitButton = nullptr;
00137 QLabel* inputPromptLabel = nullptr;
00138
00139 // State
00140 bool isRunning = false;
00141 bool isHalted = false;
00142 bool waitingForInput = false;
00143 nsbaci::types::ThreadID selectedThread = 0;
00144 };
00145
00146 } // namespace nsbaci::ui
00147
00148 #endif // NSBACI_RUNTIMEVIEW_H

```

9.95 userGuideDialog.cpp File Reference

Implementation of the UserGuideDialog class.

Include dependency graph for userGuideDialog.cpp:



Namespaces

- namespace `nsbaci`
Root namespace for the nsbaci application.
- namespace `nsbaci::ui`
User interface namespace for nsbaci.

9.95.1 Detailed Description

Implementation of the UserGuideDialog class.

Author

Nicolás Serrano García

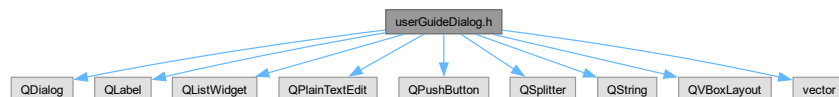
Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

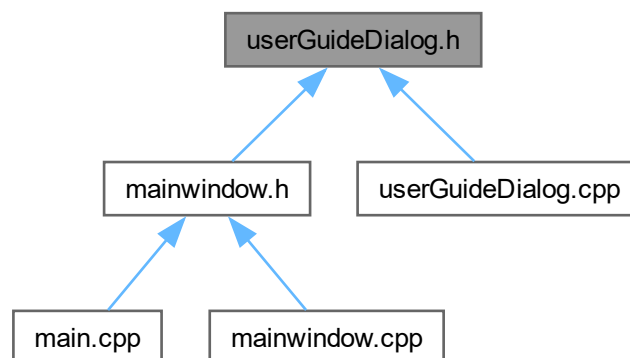
9.96 userGuideDialog.h File Reference

User Guide dialog with integrated code examples for nsbaci.

Include dependency graph for userGuideDialog.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct [nsbaci::ui::ExampleEntry](#)
Data for a single example entry in the User Guide.
- class [nsbaci::ui::UserGuideDialog](#)
A dialog that provides an integrated user guide with code examples.

Namespaces

- namespace [nsbaci](#)
Root namespace for the nsbaci application.
- namespace [nsbaci::ui](#)
User interface namespace for nsbaci.

9.96.1 Detailed Description

User Guide dialog with integrated code examples for nsbaci.

Author

Nicolás Serrano García

Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

9.97 userGuideDialog.h

[Go to the documentation of this file.](#)

```

00001
00009
00010 #ifndef USERGUIDEDIALOG_H
00011 #define USERGUIDEDIALOG_H
00012
00013 #include <QDialog>
00014 #include <QLabel>
00015 #include <QListWidget>
00016 #include <QPlainTextEdit>
00017 #include <QPushButton>
00018 #include <QSplitter>
00019 #include <QString>
00020 #include <QVBoxLayout>
00021
00022 #include <vector>
00023
00024 namespace nsbaci::ui {
00025
00029 struct ExampleEntry {
00030     QString title;
00031     QString category;
00032     QString resourcePath;
00033     QString description;
00034 };
00035
00043 class UserGuideDialog : public QDialog {
00044     Q_OBJECT
00045
00046 public:
00047     explicit UserGuideDialog(QWidget* parent = nullptr);
00048     ~UserGuideDialog() override = default;
00049

```

```
00050 signals:
00055     void loadExampleRequested(const QString& code);
00056
00057 private slots:
00058     void onExampleSelected(QListWidgetItem* current, QListWidgetItem* previous);
00059     void onLoadInEditor();
00060     void onOpenOnlineDocs();
00061
00062 private:
00063     void setupUi();
00064     void populateExamples();
00065     void applyDialogStyle();
00066     QString loadExampleCode(const QString& resourcePath) const;
00067
00068     // UI elements
00069     QListWidget* exampleList = nullptr;
00070     QLabel* titleLabel = nullptr;
00071     QLabel* descriptionLabel = nullptr;
00072     QPlainTextEdit* codeViewer = nullptr;
00073     QPushButton* loadButton = nullptr;
00074     QPushButton* onlineDocsButton = nullptr;
00075     QPushButton* closeButton = nullptr;
00076
00077     // Data
00078     std::vector<ExampleEntry> examples;
00079 };
00080
00081 } // namespace nsbaci::ui
00082
00083 #endif // USERGUIDEDIALOG_H
```